

Nota

Questi lucidi non sono da considerare come sostitutivi né dei testi di riferimento né degli appunti presi alle lezioni, e non sono quindi assolutamente sufficienti per la preparazione dell'esame. Sono piuttosto da considerare una traccia dettagliata degli argomenti trattati a lezione. Bisogna anche tener conto del fatto che parte del materiale inserito nei lucidi è protetto da copyright e può essere reso disponibile ad uso personale e solo per scopi didattici. Non è quindi possibile ridistribuire questo materiale e fornire ad altri la propria password di accesso.

Algoritmi e Strutture Dati

AA. 2025-26

Algoritmi e Strutture Dati

- Orario: 2 ore martedì, 3 ore giovedì
- Programma: = 90 % anni scorsi
- Libro adottato: Cormen et al. *Introduzione agli algoritmi e strutture dati* 3a Ed. McGraw-Hill (4a Ed. OK)
- Verifica:
 - ❶ 1 prova intermedia: 8 Novembre?
 - ❷ 2 prova intermedia: durante una prova scritta degli appelli di Gennaio - Febbraio (dettagli su moodle)
 - ❸ Verifica: prova scritta (o due prove intermedie) + orale
- Ric. studenti: informazioni nella pagina del docente (Cercachi UNIFI) ed eventuali modifiche negli avvisi sulla pagina del corso di studi

Algoritmi e Strutture Dati vs Lab. Algoritmi

Laboratorio di Algoritmi

- Non ci sono lezioni frontali (lavoro a casa)
- Alcuni argomenti saranno trattati nelle lezioni di algoritmi (probabilmente alla fine del corso)
- Esercizi pratici e semplici programmi Python
 - ▶ Consegna su Moodle nella pagina di Algoritmi e Strutture Dati

Informazioni Generali

- Comunicazioni: sito web, pinco.pallino at edu.unifi.it durante ricevimento studenti
- Precedenze d'esame
 - ▶ Per **partecipare alla prova scritta** è necessario aver superato il corso integrato "Fondamenti di Informatica / Programmazione" ed "Analisi Matematica I" (o "Elementi di Analisi Matematica")
 - ▶ Dettagli nella pagina web del corso

Verifica Laboratorio di Algoritmi

- Laboratorio di Algoritmi (a partire dall'appello successivo)
 - ① Esercizi da svolgere a casa: esperimenti con relazione su esercizi indicati dal docente
 - ② Orale: condivisione dell'ambiente di sviluppo online e discussione del codice
 - ③ Dettagli nella pagina Moodle del corso
- Chiave di accesso Moodle:
 - ▶ ASD: CDM-A

Algoritmo - in generale

Descrizione di una sequenza di azioni che un esecutore deve compiere per risolvere un problema

Esempio (ricetta)

Problema: cucinare un dolce

Input: ingredienti

Esecutore: cuoco

Algoritmo: ricetta

Output: piatto cucinato

Altro esempio (assegnare supplenti per scuola)

Problema: determinare quale docente viene assegnato ad un determinato posto

Input: dati sui docenti e sulle posizioni

Esecutore: "l'algoritmo"

Algoritmo: varie alternative

Output: assegnazioni

Algoritmo - definizione più formale

Una procedura di calcolo ben definita che prende un valore (o un insieme di valori) in **ingresso** e genera un valore (o un insieme di valori) come **uscita**

Risolve uno specifico **problema computazionale** per **ogni** possibile istanza

Problema computazionale

- La sua **descrizione** specifica in termini generali la relazione ingresso/uscita desiderata
- L'**algoritmo** descrive un modo per ottenere tale relazione
- Un **programma** infine implementa l'algoritmo
- Sono tre cose distinte e **non vanno confuse!!**

Problemi computazionali

Problema computazionale: descrizione

- La **descrizione** specifica in termini generali la relazione ingresso/uscita desiderata
- Viene descritto in genere in termini **matematici**

Problema computazionale: algoritmo

- L'**algoritmo** descrive un modo per ottenere tale relazione
- Viene descritto in genere con **pseudo-codice**

Problema computazionale: programma

- Un **programma** infine implementa l'algoritmo
- Viene descritto in genere con un programma in un linguaggio di **alto livello** (es. C, C++, Python)

Problemi computazionali: esempi

Minimo di un insieme (S)

$$m = \min(S)$$

$$\Rightarrow m \in S : \forall s \in S : m \leq s$$

Input: S **Output:** m

Ricerca in un insieme

$$i = \text{Ricerca}(S, v)$$

$$\Rightarrow i \in \{1, \dots, n\} : S_i = v; 0 \text{ altrimenti}$$

Input: S **Output:** i

Ordinamento

Input: una sequenza di n numeri $\langle a_1, a_2, \dots, a_n \rangle$

Output una permutazione $\langle a'_1, a'_2, \dots, a'_n \rangle$ della sequenza di input tale che $\langle a'_1 \leq a'_2 \leq \dots \leq a'_n \rangle$

Algoritmi: esempi

Algoritmi ?!? per alcuni degli esempi

- **Minimo:**

Confronta ogni elemento con tutti gli altri; l'elemento che è minore di tutti è il minimo

- **Ricerca:**

Confronta v con tutti gli elementi di S e restituisci la posizione corrispondente se trovato;
restituisci 0 se nessun elemento corrisponde

- **Ordinamento:**

Prendi il valore più piccolo lo metti da parte, poi passi al successivo fino a terminare tutti i valori

Problemi ...

Descrizione

- Algoritmi descritti in linguaggio naturale non sono precisi
 - ▶ Serve un linguaggio più formale indipendente dall'implementazione (programma)

Valutazione algoritmi

- Esistono algoritmi “*migliori*” di quelli proposti?
 - ▶ Come valuto “*migliore*”?

Caratteristiche comuni a molti problemi algoritmici

- **Molte soluzioni** tra cui scegliere la “migliore”
- **Molte applicazioni** per ogni algoritmo

Corso **focalizzato** su:

1. **Analisi** degli algoritmi

- Studio *teorico* delle prestazioni e dell'utilizzo di risorse degli algoritmi
 - ▶ l'algoritmo risolve il problema in modo **efficiente**?
 - ▶ potrei fare meglio?

2. Studio della **correttezza** degli algoritmi

- Un algoritmo termina con l'output corretto *per ogni* istanza di input? (**risolve** il problema computazionale)?
 - ▶ alcuni problemi non possono essere risolti
 - ▶ alcuni problemi vengono risolti in modo approssimato

3. **Utilizzo** degli algoritmi

- Posso risolvere problemi con "piccole modifiche" di algoritmi noti
- *Scelta* di algoritmi alternativi
- Conoscere le interfacce di ingresso-uscita degli algoritmi

Dato vs tipo di dato

Dato

- Valore che una variabile può assumere

Tipo di dato

- Modello matematico, costituito da:
 - ▶ un insieme di valori che la variabile può assumere
 - ▶ un insieme di operazioni ammesse su questi valori
- Si trascurano i dettagli implementativi
- Tipi di dati **primitivi** forniti dal linguaggio
 - ▶ int (+, -, *, /, %), boolean (!, &&, ||)
 - ▶ matrice bidimensionale (memorizzata “per righe”)

Strutture di dati

- Estende il concetto di "tipo di dato" elementare a dati più complessi
- Un modo per memorizzare e organizzare i dati e semplificarne l'accesso e la modifica
- Include:
 - ▶ organizzazione dati in memoria
 - ▶ parte algoritmica (accesso e modifica)

Strutture di dati

Organizzazione in memoria dei dati

Come "organizzo i byte" per memorizzare i dati

Insieme di operatori per:

- **accedere** alla struttura
 - ▶ leggere elementi
 - ▶ cercare elementi
 - ▶ ...
- **manipolare** la struttura:
 - ▶ aggiungere elementi
 - ▶ rimuovere elementi
 - ▶ modificare elementi
 - ▶ modificare la struttura
 - ▶ ...

Strutture dati elementari (nei linguaggi di alto livello es. C)

- **Vettore di memoria**: elementi dello stesso tipo
- **Array**: vettore su più dimensioni
- **Record**: elementi di tipo diverso con nome (struct del C)

Altre strutture dati

- **Insieme**
 - ▶ insieme ordinato (sequenza, lista)
 - ▶ insieme dinamico
- **Pila** (stack)
- **Coda** (queue)
 - ▶ coda con priorità
- **Dizionario**
- **Grafo**
 - ▶ albero
- Talvolta presenti in linguaggi di alto livello (es. dizionario in Python)

Insiemi - insiemi ordinati

Differenze tra **insiemi** in informatica e in matematica

- Un insieme matematico **non** cambia
(per aggiungere l'elemento e all'insieme S :
 $S' = S \cup e$)
- In informatica: **insieme dinamico** (cresce, cambia nel tempo)
- Ogni elemento è rappresentato da un oggetto con:
 - ▶ **attributi** e
 - ▶ **dati satelliti**
- Diversi algoritmi possono usare diverse operazioni sugli insiemi
 - ▶ la migliore implementazione dipende dalle operazioni richieste

Possibili operazioni per insieme S

- $\text{Size}(S)$
- $\text{Contains}(S, x)$ (x è un elemento)
- $\text{Search}(S, k)$ (k è una chiave)
- $\text{Insert}(S, x)$
- $\text{Delete}(S, x)$
- Operazioni tipiche su insiemi
 - ▶ $\text{Union}(A, B)$
 - ▶ $\text{Intersection}(A, B)$
 - ▶ $\text{Difference}(A, B)$
- Non sono tutte, non necessariamente si chiamano così

Insiemi ordinati (sequenza)

- **Ordine** nella sequenza è importante

Possibili operazioni per sequenza S

- First(S)
- Last(S)
- Next(S, pos)
- Prev(S, pos)
- Insert(S, pos, x)
- Delete(S, pos)
- Read(S, pos)
- Write(S, pos, x)
- Min(S) / Max(S)
- Non sono tutte, non necessariamente si chiamano così

Stringa

- Tipo particolare di insieme ordinato (contiene caratteri alfanumerici)
- Operazioni specifiche:
 - ▶ Concat(S,str)
 - ▶ Prefix(S,len)
 - ▶ Suffix(S,len)

- Varie implementazioni

- ▶ Terminato con un NULL, come in C:

c	a	t	×
---	---	---	---

- ▶ Indicando prima la lunghezza, come in Pascal:

3	c	a	t
---	---	---	---

- ▶ Con lunghezza fissa:

c	a	t	×		
---	---	---	---	--	--

- Vantaggi e svantaggi?

Stack e Code

- Insiemi dinamici in cui
 - ▶ la posizione di Insert
 - ▶ l'elemento rimosso da Deletesono **predeterminati**

Stack

- *Last In, First Out* (LIFO)

Coda

- *First In, First Out* (FIFO)

Stack

Operazioni

- $\text{IsEmpty}(S)$
 - $\text{Push}(S, v)$
 - $\text{Pop}(S)$
 - $\text{Top}(S)$
-
- Uso:
 - ▶ Passaggio dei parametri e indirizzo di ritorno per chiamate di funzione
 - ▶ Invertire una sequenza...

Coda

Operazioni

- `IsEmpty(S)`
- `Enqueue(S, v)`
- `Dequeue(S)`
- `Top(S)`
- Uso:
 - ▶ In SO gestione dei processi in attesa di utilizzare una risorsa
 - ▶ A volte coda con priorità →

Coda con priorità

Operazioni

- `Insert(Q, x)`
- `Delete(Q, x)`
- `FindMin(Q)` (Max)
- `ExtractMin(Q)` (Max)

Implementazione

- Ad esempio con **heap binario**
- Lo vedremo poi...

Dizionario (Map)

- Relazione univoca (associa ad ogni elemento di A un solo elemento di B , ma non viceversa)
- Un insieme D di coppie (*chiave*, *valore*)
 - ▶ Relazione $R : A \rightarrow B$
 - ▶ Insieme A è il dominio (chiavi)
 - ▶ Insieme B è il codominio (valori)

Operazioni

- `Search(D, k)`
- `Insert(D, k, v)`
- `Delete(D, k)`

Grafi e alberi

- Rappresentazione grafica di una interrelazione tra oggetti/concetti
- Albero è un caso particolare di grafo

Grafi

- Insiemi di nodi e insieme di archi che connettono i nodi (es: il Web)
- Operazioni:
 - ▶ *Visita*: ispezione completa di tutti i nodi di un albero o di un grafo
 - ▶ Altre operazioni in seguito

Albero ordinato (definizione formale in seguito)

- un tipo particolare di grafo
- un nodo è designato come **radice**
- i rimanenti nodi, se esistono, sono partizionati in insiemi ordinati e disgiunti, anch'essi alberi ordinati

Alberi

Ci sono vari concetti che sono indicati come “albero”

Disambiguazione Wikipedia:

- pianta perenne dotata di fusto
- simbolo araldico e religioso (negli stemmi)
- **una struttura dati usata per organizzare un insieme di dati**
- **in matematica, un grafo non orientato connesso e aciclico**
- genealogico
- negli apparati meccanici, un organo atto a trasferire un moto
- componente della distribuzione dei motori a quattro tempi
- parte di una nave o imbarcazione a vela

Possibili implementazioni di alcune strutture dati

Messaggio principale

- ci sono vari modi per realizzare una struttura dati
- in certi casi è preferibile un modo ...
- ... in altri un altro approccio
- la scelta dipende dall'uso
- ... ma bisogna saper scegliere

Implementazione **Array**

Esempio in C: `int A[10][20]`

Memorizzazione “standard” (per righe)

Data la matrice $A[R][C]$

$A[i][j]$ è memorizzato in: $i \cdot C + j$

(primo elemento in posizione 0)

Matrici sparse

Memorizzo le triplette (i, j, val) solo se $A[i][j] \neq 0$

Spazio occupato da matrice $n \times n$ (interi 4 byte)

- occupa $4 \cdot n^2$ byte se rappresentata per righe
- e se rappresentata per colonne?
- occupa $v \cdot 3 \cdot 4$ byte se è sparsa e contiene v valori
- conviene se $v \cdot 3 < n^2$
- come memorizzo le triplette? Con il dizionario

Quanto può essere grande una matrice?

- esempio: matrice dei link delle pagine Web (vedi poi per grafi)
- stima: 1 miliardo di pagine Web
- matrice avrebbe $10^9 \cdot 10^9 = 10^{18}$ elementi
- se ogni elemento occupa un byte si ha un exabyte
(...mega, giga, tera, peta, exa...)
- (5 exa: stima del totale di parole mai pronunciate da essere umano...)

Implementazione **Insiemi** (e dizionari)

- Insieme → Collezione di oggetti
- Dizionario → Insieme di associazioni chiave-valore
- Implementazioni ...

Sappiamo **implementare** insiemi?

- Consideriamo **solo** alcune strutture dati viste a Fondamenti di Informatica:
 - ▶ Tupla, Vettore di memoria, Lista concatenata (ordinata o no)

Insiemi realizzati con **vettori booleani**

- Insieme
 - ▶ interi $1 \dots n$
 - ▶ collezione di n possibili oggetti memorizzati in un vettore
- Rappresentazione dell'insieme:
 - ▶ vettore booleano con n elementi
- **Vantaggi**
 - ▶ semplice!
 - ▶ efficiente verificare se un elemento appartiene all'insieme
 - ▶ efficienti Unione e Intersezione
- **Svantaggi**
 - ▶ occupazione di memoria $O(n)$, indipendente dalla dimensione dell'insieme
 - ▶ molte operazioni inefficienti $O(n)$
- Nota: $O(n) \approx$ proporzionale con n (poi una definizione formale)

Insiemi realizzati con liste concatenate **non ordinate**

- **Vantaggi**

- ▶ occupazione di memoria proporzionale alla dimensione dell'insieme
- ▶ operazioni di inserimento: $O(1)$

- **Svantaggi**

- ▶ operazioni di ricerca e cancellazione: $O(n)$
- ▶ operazioni di unione, intersezione e differenza: $O(n\ m)$

Insiemi realizzati con liste concatenate **ordinate**

- **Vantaggi**

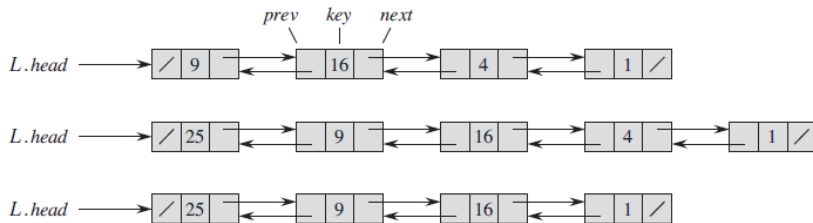
- ▶ Occupazione di memoria proporzionale alla dimensione dell'insieme
- ▶ Operazioni di unione, intersezione e differenza = $O(n)$

- **Svantaggi**

- ▶ Operazioni di ricerca, inserimento e cancellazione: $O(n)$

Insieme ordinato (sequenza)

Implementazione con **lista concatenata**



Usate sempre questo tipo di rappresentazione per le liste concatenate!!

Varianti:

- Bidirezionale / monodirezionale
- Con sentinella / senza sentinella
- Circolare / non circolare

Realizzazione di insiemi con strutture dati **complesse**

- Saranno presentate nel corso
- Alberi di ricerca (bilanciati)
 - ▶ ricerca, inserimento, cancellazione: $O(\log n)$
 - ▶ viene mantenuto l'ordinamento
 - ▶ elencare tutti gli elementi: $O(n)$
- Tabelle hash
 - ▶ ricerca, inserimento, cancellazione: $O(1)$
 - ▶ viene perso l'ordinamento
 - ▶ elencare tutti gli elementi: $O(m)$, dove m è la dimensione della tabella

Implementazione Insiemi (e dizionari)

- Insieme $\rightarrow$ Collezione di oggetti
- Dizionario $\rightarrow$ Insieme di associazioni chiave-valore
- Implementazioni ...

	search	insert	delete	min	scan	sorted
Vet. bool	$O(1)$	$O(1)$	$O(1)$	$O(N)$	$O(N)$	Si
Lista non ordinata	$O(n)$	$O(1) *$	$O(n)$	$O(n)$	$O(n)$	No
Lista ordinata	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	Si
Hash (medio) mem. interna	$O(1)$	$O(1)$	$O(1)$	$O(m)$	$O(m)$	No
Hash (medio) mem. esterna	$O(1)$	$O(1)$	$O(1)$	$O(m+n)$	$O(m+n)$	No
ARN	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Si

* sapendo che l'elemento da inserire non è già presente

N numero massimo di elementi **memorizzabili**, n elementi **memorizzati**

m dimensione tabella hash. $m \geq n$ con memorizzazione interna

Pila / Coda

Stack

Possibili implementazioni:

- liste bidirezionali
- vettore di memoria

Coda

Possibili implementazioni

- Liste
 - ▶ puntatore head per estrazione
 - ▶ puntatore tail per inserimento
- Array circolari

Dizionario

Implementazioni

- **array ordinato**
- **array non ordinato mantenuto con la tecnica del raddoppiamento-dimezzamento**
- **struttura circolare doppiamente collegata**
- albero binario di ricerca
- albero rosso-nero
- tavola ad accesso diretto
- tavola hash con indirizzamento aperto
- tavola hash con liste di collisione
- tavola hash perfetta
- albero B
- tecniche map-reduce

Albero

Possibili realizzazioni...

- Realizzazione con vettore dei figli. Rischio di sprecare memoria se molti nodi hanno grado minore del grado massimo
- Realizzazione con puntatori padre/primo-figlio/fratello
Ha una lista dei figli (fratelli)
- Realizzazione con vettore dei padri
L'albero è rappresentato da un vettore i cui elementi contengono l'indice del padre (unico)

Ne parleremo abbondantemente il seguito

Grafo

Rappresentazioni

- con lista di archi
- con liste di adiacenza
- con liste di incidenza
- con matrice di adiacenza
- con matrice di incidenza

Ne parleremo abbondantemente il seguito

Sommario

Abbiamo visto...

- **Algoritmi**

- ▶ Utilità e definizioni di algoritmi
- ▶ Obiettivi del corso relativi allo studio di algoritmi

- **Strutture dati**

- ▶ Stretto legame tra
 - ★ Memorizzazione dati
 - ★ Accesso ed utilizzo
- ▶ Caratteristiche principali strutture dati
- ▶ Diverse implementazioni con diverse peculiarità
- ▶ Richiamo strutture dati note

Insertion sort e oltre...

AA. 2025-26

Il problema dell' **ordinamento**

Input: una sequenza di n numeri $A = \langle a_1, a_2, \dots, a_n \rangle$

Output: una permutazione

$A' = \langle a'_1, a'_2, \dots, a'_n \rangle$ di A t.c. :

$a'_1 \leq a'_2 \leq \dots \leq a'_n$

- Sequenze memorizzate in **array**
- Numeri $\rightarrow$ **chiavi**

Descrivere algoritmi

- Italiano (Inglese)
- Pseudo-codice

Insertion sort

Buon algoritmo per **pochi** elementi

Ordina un *mazzo di carte*:

- in una mano ordino le carte (all'inizio è “vuota”)
- prendo una carta e la inserisco nella posizione corretta
 - ▶ confrontandola con ogni carta nella mano
- in ogni momento le carte nella mano sono ordinate

Pseudocodice per INSERTION-SORT

- Input: $A[1..n]$
 - ▶ primo elemento: 1
 - ▶ In C, C++, Java, Python primo elemento: 0
- A è **ordinato sul posto**

INSERTION-SORT(A, n)

```
1  for  $j \leftarrow 2$  to  $n$ 
2       $key \leftarrow A[j]$ 
3      // Inserisci  $A[j]$  nella sequenza ordinata  $A[1..j-1]$ 
4       $i \leftarrow j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] \leftarrow A[i]$ 
7           $i \leftarrow i - 1$ 
8       $A[i+1] \leftarrow key$ 
```

Correttezza

- Un algoritmo si dice **corretto** se, per ogni istanza di input, termina con l'output **giusto**
- Mostrare che un algoritmo funziona con un determinato input **non** è una dimostrazione di correttezza!

- Per algoritmi **iterativi** si usa spesso un **invariante di ciclo**
- Per INSERTION-SORT:

“All’inizio di ogni iterazione del ciclo for il sottoarray $A[1..j-1]$ è ordinato ed è formato dagli stessi elementi che erano originariamente in $A[1..j-1]$, ma ordinati”

Correttezza con invariante di ciclo

- Bisogna **dimostrare tre cose** sull'invariante (oltre ad enunciare l'invariante):
 - ▶ **Inizializzazione**: IC è vera prima della prima iterazione del ciclo
 - ▶ **Conservazione**: se IC è vera prima di un'iterazione del ciclo, rimane vera prima della successiva iterazione
 - ▶ **Conclusione**: quando il ciclo termina l'invariante ci fornisce una proprietà **utile** per mostrare che l'algoritmo è corretto

Analogo all'**induzione matematica**

- Inizializzazione
- Conservazione
- Conclusione non c'è nell'*induzione matematica*:
il passo induttivo è *usato all'infinito*

Induzione matematica

Voglio mostrare che una **proprietà** $P(n)$ vale per **qualunque** $n \in \mathbb{N}$

- $U = \{n \in \mathbb{N} : \text{vale } P(n)\}$
- Passo base: **dimostro** che vale $P(1)$ (o $P(0)$)
- Passo induttivo **suppongo** che $P(n)$ valga per un generico n e dimostro che vale anche $P(n+1)$
($n \in U \Rightarrow n+1 \in U$)
- Concludo che l'insieme U coincide con tutto l'insieme dei numeri naturali

Esempio: serie aritmetica

$$P(n) = 1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2}$$

- Passo base:

$P(n)$ vera per $n = 1$:

$$P(1) = \frac{1 \cdot (1+1)}{2} = \frac{1 \cdot 2}{2} = 1$$

- Passo induttivo:

$$P(n+1) = 1 + 2 + \dots + n + (n+1) = P(n) + (n+1) =$$

$$\frac{n(n+1)}{2} + (n+1) = \frac{n(n+1) + 2(n+1)}{2} = \frac{(n+1)(n+2)}{2} =$$

$$\frac{(n+1)((n+1)+1)}{2} = P(n+1)$$

- Conclusione:

$$1 + 2 + \dots + n = \frac{n(n+1)}{2} \quad \forall n \in \mathbb{N}.$$

IC: *“All’inizio di ogni iterazione del ciclo for il sottoarray $A[1..j-1]$ è ordinato ed è formato dagli stessi elementi che erano originariamente in $A[1..j-1]$, ma ordinati”*

INSERTION-SORT(A, n)

```
1  for  $j \leftarrow 2$  to  $n$ 
2       $key \leftarrow A[j]$ 
3      // Insert  $A[j]$  in sorted sequence  $A[1..j-1]$ 
4       $i \leftarrow j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] \leftarrow A[i]$ 
7           $i \leftarrow i - 1$ 
8       $A[i+1] \leftarrow key$ 
```

Invariante di ciclo

IC: *“All’inizio di ogni iterazione del ciclo for il sottoarray $A[1..j-1]$ è ordinato ed è formato dagli stessi elementi che erano originariamente in $A[1..j-1]$, ma ordinati”*

- Inizializzazione (vera prima della prima iterazione):

...

...

- Conservazione (se vera prima di iterazione j vera anche alla fine):

...

...

- Terminazione (vera dopo la fine dell’ultima iterazione):

...

...

Python

```
for j in range(1,len(A)):
    print(A)
    key = A[j]
    i = j-1
    while i >= 0 and A[i] > key:
        A[i+1] = A[i]
        i = i-1
    A[i+1] = key
```

Complessità di un algoritmo

Analisi delle **risorse** impiegate da un algoritmo per risolvere un problema, in funzione della dimensione e del tipo di input

Risorse

- **Tempo:** per completare l'algoritmo
 - ▶ misurato con il cronometro
 - ▶ misurato contando il numero di operazioni elementari
- **Spazio:** quantità di memoria utilizzata
- **Banda:** quantità di bit spediti (algoritmi distribuiti)

Perché stimare la complessità in tempo degli algoritmi?

- per stimare il tempo impiegato per un dato input
- per stimare il più grande input gestibile in tempi ragionevoli
- per confrontare l'efficienza di algoritmi diversi
- per ottimizzare le parti più importanti

Problemi...

- Come definire la dimensione dell' input?
- Come misurare il tempo?

Dimensione input

Costo **logaritmico**

- numero di bit necessari per rappresentare l'input
 - ▶ Es.: moltiplicazione numeri di n bit

Costo **uniforme**

- Numero di elementi nell'input
 - ▶ Es.: minimo in un vettore di n elementi
- Potrebbero esserci più valori da considerare (es. numero di *nodi ed archi* di un grafo)

Definizione di tempo

Tempo $\equiv$ wall-clock time

Dipende da:

- bravura del programmatore
- linguaggio di programmazione
- codice generato dal compilatore
- processore, memoria (cache, primaria, secondaria), sistema operativo, processi in esecuzione

Tempo $\equiv$ numero operazioni elementari

Operazione elementare: eseguita in tempo “costante”.

Esempi (operazioni elementari?)

- $a* = 2$?
- $\text{Math.cos}(d)$?
- $\text{min}(A, n)$?

Modello di calcolo

Rappresentazione astratta di un calcolatore

- **Astrazione**
- **Realismo**
- **Potenza matematica**

Random Access Machine (RAM)

Singolo processore

- istruzioni eseguite **in successione** (no concorrenti)
- istruzioni comuni, tempo costante (no **ordinamento**)

Memoria

- quantità infinita di celle di dimensione finita
- accesso in tempo costante (RAM)

Dati

- interi e virgola mobile (no “matrice”)
- non ci preoccupiamo della *precisione*
- parole di dimensioni limitate

Analisi di insertion sort

INSERTION-SORT(A, n)

```

1  for  $j \leftarrow 2$  to  $n$ 
2       $key \leftarrow A[j]$ 
3      // Insert  $A[j]$  in sorted sequence  $A[1..j-1]$ 
4       $i \leftarrow j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] \leftarrow A[i]$ 
7           $i \leftarrow i - 1$ 
8       $A[i+1] \leftarrow key$ 

```

Cost Times

c_1	n
c_2	$n - 1$
0	$n - 1$
c_4	$n - 1$
c_5	$\sum_{j=2}^n t_j$
c_6	$\sum_{j=2}^n (t_j - 1)$
c_7	$\sum_{j=2}^n (t_j - 1)$
c_8	$n - 1$

t_j : numero di volte che viene eseguito il test del while

Tempo di esecuzione dell'algoritmo:

- Linea i impiega c_i

$$\text{costo} = \sum_{\text{istruzioni}} c_i \cdot (\text{volte che viene eseguita})$$

$T(n)$ = tempo esecuzione di INSERTION-SORT =

$$= c_1 \cdot n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

- $T(n)$ dipende da t_j che dipendono da valori in input
 - t_j : numero di volte che viene eseguito il test del while

Caso migliore

Array già ordinato

- Sempre $A[i] \leq \text{key} \Rightarrow \forall j \ t_j = 1$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

- $\Rightarrow T(n) = a \cdot n + b$
con costanti a e b (dipendono da c_i)
 $\Rightarrow T(n)$ è una **funzione lineare** di n

Caso peggiore

Array **ordinato al contrario**

- Sempre $A[i] > key$ nel test del while
- $\Rightarrow$ Confronta key con tutti i $j - 1$ elementi alla sinistra di j
- Un altro test dopo i $j - 1$ test $\Rightarrow t_j = j$

$$\sum_{j=2}^n t_j = \sum_{j=2}^n j = \left(\sum_{j=1}^n j \right) - 1 = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^n (t_j - 1) = \sum_{j=2}^n (j - 1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

Avendo posto $k = j - 1$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right)$$

$$+ c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) =$$

$$T(n) = c_1 n + (c_2 + c_4 + c_8)(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + (c_6 + c_7)\left(\frac{n(n-1)}{2}\right) =$$

$$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n - (c_2 + c_4 + c_5 + c_8)$$

- $T(n) = an^2 + bn + c$ con costanti a , b , c
- $T(n)$ è una **funzione quadratica** di n

Caso peggiore

- Tempo di esecuzione più lungo per ogni input di dim n :
 - ▶ **Limite superiore** per ogni input
 - ▶ Non tanto raro
 - ★ Es: ricerca di un elemento assente
 - ▶ **Caso medio** spesso **altrettanto cattivo del peggiore**

Caso medio

- Media del tempo di esecuzione **su tutti i possibili input**

Caso medio per insertion sort

- Input: n numeri a caso
- In "media" $A[j] <$ di metà degli elementi in $A[1..j-1]$
- $\Rightarrow t_j = j/2$
- Circa metà del caso peggiore, ma sempre una funzione quadratica di n

Tasso di crescita: astrazione semplificativa

Guarda solo il **termine principale** di una formula ignorando costanti

insertion sort

$$T(n) = an^2 + bn + c \Rightarrow an^2 \Rightarrow n^2$$

- Il tempo di esecuzione **cresce come** n^2
non è **uguale** a n^2
- *Tasso di crescita:* $n^2 \rightarrow \Theta(n^2)$
- Usato per confrontare algoritmi

Algoritmi iterativi e ricorsivi

Algoritmi iterativi

- Insertion sort è **incrementale**: ordinato $A[1..j-1]$, si sistema $A[j]$
 $\Rightarrow A[1..j]$ è ordinato

Algoritmi ricorsivi: **divide et impera**

- **divide** il problema in sottoproblemi
- sottoproblemi risolti **ricorsivamente** (**impera**)
- soluzioni dei sottoproblemi **combinare** per risolvere il problema originale
- **caso base**: sottoproblemi piccoli si risolvono direttamente

Fattoriale

Il **fattoriale** di un numero naturale n è il prodotto dei numeri interi positivi minori o uguali a n :

- Iterativo:

$$n! = \prod_{k=1}^n k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

- oppure:

$$n! = n \cdot (n-1) \cdots 2 \cdot 1$$

- Ricorsivo:

$$n! = \begin{cases} 1 & \text{se } n = 1 \\ n \cdot (n-1)! & \text{se } n > 1 \end{cases}$$

Fattoriale

Iterativo

FATT(n)

```
1  if  $n = 1$   
2      return 1  
3   $Fat \leftarrow 1$   
4  for  $i \leftarrow 2$  to  $n$   
5       $Fat \leftarrow Fat \cdot i$   
6  return  $Fat$ 
```

Alternativa

FATT(n)

```
1   $Fat \leftarrow 1$   
2  for  $i \leftarrow 1$  to  $n$   
3       $Fat \leftarrow Fat \cdot i$   
4  return  $Fat$ 
```

Fattoriale

Altra alternativa

FATT(n)

```
1   $Fat \leftarrow 1$   
2  for  $i \leftarrow n$  downto 1  
3       $Fat \leftarrow Fat \cdot i$   
4  return  $Fat$ 
```

Ricorsivo

FATT(n)

```
1  if  $n = 1$   
2      return 1  
3  else  
4      return  $n \cdot \text{FATT}(n - 1)$ 
```

Fattoriale

Fattoriale iterativo

FATT(n)

```
1  if  $n = 1$ 
2      return 1
3   $Fat \leftarrow 1$ 
4  for  $i \leftarrow 2$  to  $n$ 
5       $Fat \leftarrow Fat \cdot i$ 
6  return  $Fat$ 
```

Possiamo "studiarlo"?

- E' corretto?
 - ▶ invariante di ciclo
- Quanto tempo impiega?
 - ▶ ciclo su n valori
 - ▶ operazioni costanti
 - ▶ $\rightarrow$ costa $c \cdot n = n$

Fattoriale

Fattoriale ricorsivo

FATT(n)

```
1  if  $n = 1$   
2      return 1  
3  else return  $n \cdot \text{FATT}(n - 1)$ 
```

Possiamo "studiarlo"?

- E' corretto?
 - ▶ sulla base della definizione ricorsiva
- Quanto tempo impiega?
 - ▶ costo costante ad ogni chiamata
 - ▶ quante chiamate?
 - ▶ mi aiuta l'**albero di ricorsione**
 - ▶ costa $c \cdot n = n$

Merge sort: ordina $A[p..r]$

Divide - Impera - Combina

- **Divide** $A[p..r]$ in $A[p..q]$ e $A[q+1..r]$,
 q è il punto di mezzo di $A[p..r]$
- **Impera** ordinando ricorsivamente $A[p..q]$ e $A[q+1..r]$
- **Combina** fondendo i due sotto-array ordinati $A[p..q]$ e $A[q+1..r]$
in un singolo array ordinato $A[p..r]$

MERGE-SORT(A, p, r)

```
1  if  $p < r$                                 // Controlla il caso base
2       $q \leftarrow \lfloor (p+r)/2 \rfloor$           // Divide
3      MERGE-SORT( $A, p, q$ )                  // Impera
4      MERGE-SORT( $A, q+1, r$ )                // Impera
5      MERGE( $A, p, q, r$ )                    // Combina
```

Chiamata iniziale: MERGE-SORT($A, 1, n$)

MERGE(A, p, q, r)

```
1   $n_1 \leftarrow q - p + 1$ 
2   $n_2 \leftarrow r - q$ 
3  // Crea array  $L[1..n_1 + 1]$  e  $R[1..n_2 + 1]$ 
4  for  $i \leftarrow 1$  to  $n_1$ 
5       $L[i] \leftarrow A[p + i - 1]$ 
6  for  $j \leftarrow 1$  to  $n_2$ 
7       $R[j] \leftarrow A[q + j]$ 
8   $L[n_1 + 1] \leftarrow \infty$ 
9   $R[n_2 + 1] \leftarrow \infty$ 
10  $i \leftarrow 1$ 
11  $j \leftarrow 1$ 
12 for  $k \leftarrow p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] \leftarrow L[i]$ 
15          $i \leftarrow i + 1$ 
16     else  $A[k] \leftarrow R[j]$ 
17          $j \leftarrow j + 1$ 
```

Tempo di esecuzione

- Primi due loop: $\Theta(n1 + n2) = \Theta(n)$
- Ultimo *for*: n iterazioni ciascuna in tempo costante
 $\Rightarrow \Theta(n)$
- **Totale:** $\Theta(n)$

Correttezza?

Esercizio: *Invariante di ciclo per mostrare la correttezza di MERGE*

Analizzare algoritmi divide et impera

Ricorrenza

- Equazione di ricorrenza descrive il tempo di esecuzione totale
 $T(n)$ = tempo esecuzione per problema di dimensione n

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{altrimenti} \end{cases}$$

- Se $n \leq c$ per qualche costante $c \Rightarrow$
caso base risolto in **tempo costante**: $\Theta(1)$
- Altrimenti **divide** in a sottoproblemi di dimensione $1/b$
- $D(n)$: tempo per **dividere**
- $C(n)$: tempo per **combinare**

Analizzare merge sort

- Supponiamo che n sia una potenza di 2
 $\Rightarrow$ due sottoproblemi di dimensione $n/2$
- Caso base: $n = 1$
- Se $n \geq 2$, i tempi per i passi divide et impera sono:
 - ▶ **Divide**: Calcola $q \Rightarrow D(n) = \Theta(1)$
 - ▶ **Impera**: Risolve 2 sottoproblemi di dim $n/2 \Rightarrow 2 \cdot T(n/2)$
 - ▶ **Combina**: fonde un array con n elementi
- $D(n) + C(n) = \Theta(1) + \Theta(n) = \Theta(n)$

Ricorrenza

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Risolvere la ricorrenza di merge-sort

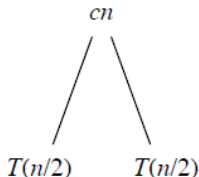
- Dal teorema dell'esperto (lo vedremo poi) soluzione:
 $T(n) = \Theta(n \lg n)$
- $\lg n$ indica $\log_2 n$
- Più veloce di insertion sort ($\theta(n^2)$ caso peggiore)
- Su **input piccoli** insertion sort può essere più veloce
- Per input grandi abbastanza merge sort in genere più veloce

Senza il teorema

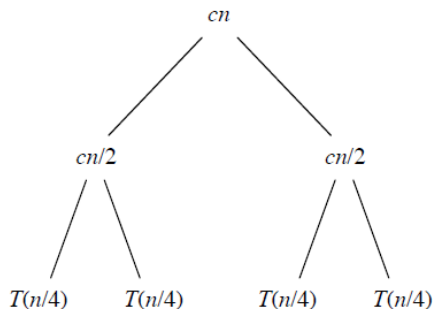
- costante c : massimo tra
tempo di esecuzione per il *caso base* e
tempo per *divide* e *combina* per ogni elemento
- **Ricorrenza**:

$$T(n) = \begin{cases} c & \text{se } n = 1 \\ 2T(n/2) + c n & \text{se } n > 1 \end{cases}$$

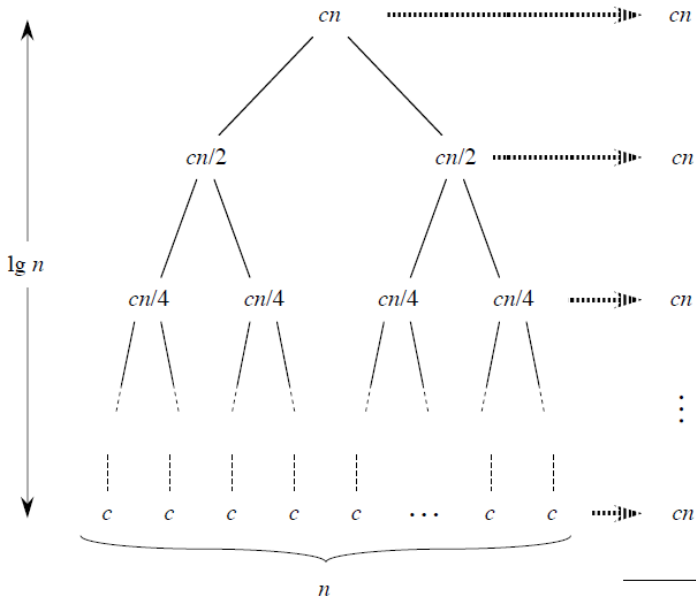
- **Albero di ricorsione** mostra espansioni successive
- Problema originario: $c \cdot n + 2$ sotto-problemi $T(n/2)$:



- Ogni sotto-problema di dim $n/2$ costa $cn/2 + 2$ sottoproblemi $T(n/4)$:



Si continua ad espandere fino ad arrivare a problemi di dimensione 1:



- Ogni livello ha costo $c \cdot n$:
 - ▶ 1 livello: $c \cdot n$
 - ▶ 2 livello: 2 sottoproblemi con costo $c \cdot n/2$ ($2 \cdot c \cdot n/2 = c \cdot n$)
 - ▶ 3 livello: 4 sottoproblemi con costo $c \cdot n/4$ ($4 \cdot c \cdot n/4 = c \cdot n$)
 - ▶ $\rightarrow$ costo per livello costante
- $\lg n + 1$ livelli (altezza: $\lg n$)
 - ▶ Ad ogni livello dimezzo i dati:
al livello i dimensione $\frac{n}{2^i}$ (radice ha dim. n)
 - ▶ Caso base (foglia) con un solo elemento:
 $\frac{n}{2^i} = 1 \Rightarrow n = 2^i \Rightarrow i = \log_2 n = \lg n$
- **Costo totale**
 $\lg n + 1$ livelli, ognuno costa $cn \Rightarrow$ totale $c n \lg n + c n$
 $c \Rightarrow \Theta(n \lg n)$

Crescita delle funzioni

Esercizi...

Ricerca in un array (2.1-3)

- **Input:** una sequenza di n numeri $A = \langle a_1, a_2, \dots, a_n \rangle$ e un valore v
- **Output:** un indice i tale che $v = A[i]$ o *NIL* se v non si trova in A
- Scrivere lo **pseudocodice** dell'algoritmo semplice (che scorre tutti i valori)
- Utilizzando un'invariante di ciclo dimostrare che l'algoritmo è corretto

Ricerca Binaria 2.3-5

- Se la sequenza A è ordinata possiamo *confrontare il punto centrale della sequenza con v ed escludere metà sequenza da ulteriori controlli*
- Scrivere pseudocodice iterativo e ricorsivo
- Dimostrare che nel caso peggiore è $\Theta(\lg n)$
- Chiamata iniziale con parametri $A, v, 1, n$

Selection sort 2.2-2

Algoritmo per ordinare n numeri memorizzati nell'array A

- Trovare il più piccolo elemento di A e scambiarlo con l'elemento in $A[1]$
- trovare il secondo elemento più piccolo di A e scambiarlo con $A[2]$
- continuando per i primi $n - 1$ elementi di A

Esercizio

- Scrivere lo pseudocodice.
- Quale invariante di ciclo conserva questo algoritmo?
- Perché basta eseguirlo solo per i primi $n - 1$ elementi e non per n ?
- Esprimere nella notazione Θ i tempi di esecuzione dei casi migliore e peggiore dell'algoritmo

- Come modificare quasi tutti gli algoritmi in modo da avere un buon tempo di esecuzione nel caso migliore?
- Tante possibili risposte.

Crescita delle funzioni

- Come aumenta il tempo di esecuzione di un algoritmo al crescere della dimensione dell'input **al limite**?
- Per input piccoli il tempo di esecuzione potrebbe essere diverso

Notazione asintotica

- Astraendo termini di ordine inferiore e fattori costanti
- Un modo per confrontare la “dimensione” di funzioni:

$$\begin{array}{ll}
 O \approx \leq & \Omega \approx \geq \\
 \Theta \approx = & \\
 o \approx < & \omega \approx >
 \end{array}$$

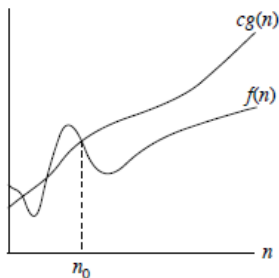
Nota

- Ogni funzione nelle definizioni deve essere **asintoticamente non negativa**

Notazione O

$$O(g(n)) = \{f(n) : \exists c \in \mathbb{R}^+, n_0 \in \mathbb{N} \text{ t.c. } 0 \leq f(n) \leq c \cdot g(n) \quad \forall n \geq n_0\}$$

$O(g(n))$ è un **insieme di funzioni**



- $g(n)$ è un **limite asintotico superiore** per $f(n)$
- Se $f(n) \in O(g(n))$, si scrive $f(n) = O(g(n))$

Esempio

$$2n^2 = O(n^3), \text{ con } c = 1 \text{ e } n_0 = 2$$

Esempi di funzioni in $O(n^2)$

$$n^2$$

$$n^2 + n$$

$$n^2 + 1000n$$

$$1000n^2 + 1000n$$

anche

$$n$$

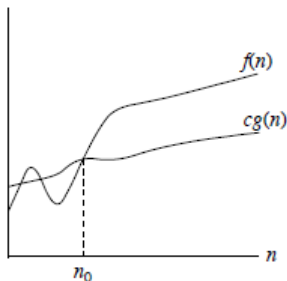
$$n/1000$$

$$n^{1.9999}$$

$$\frac{n^2}{\lg \lg \lg n}$$

Notazione Ω

$$\Omega(g(n)) = \{f(n) : \exists c \in \mathbb{R}^+, n_0 \in \mathbb{N} \text{ t.c. } 0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0\}$$

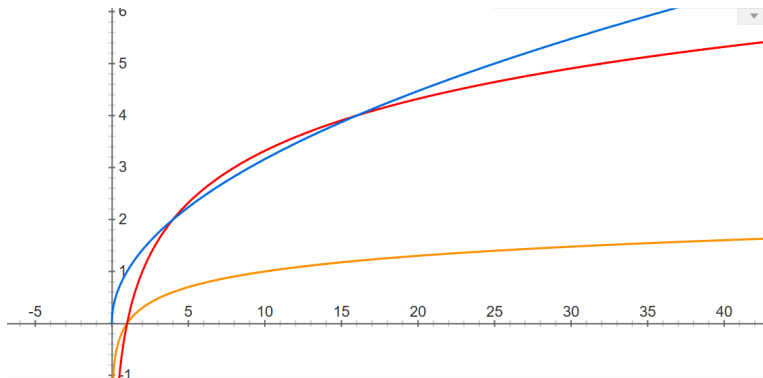


$g(n)$ è un **limite asintotico inferiore** per $f(n)$

Esempio

$\sqrt{n} = \Omega(\lg n)$ con $c = 1$ e $n_0 = 16$

$\sqrt{16} = 4$, $\lg 16 = 4$ ($2^4 = 16$)



blu: $\sqrt{n}$, arancione: $\log(n)$, rosso: $\lg(n)$

Esempi di funzioni in $\Omega(n^2)$

$$n^2$$

$$n^2 + n$$

$$n^2 - n$$

$$1000n^2 + 1000n$$

$$1000n^2 - 1000n$$

anche

$$n^3$$

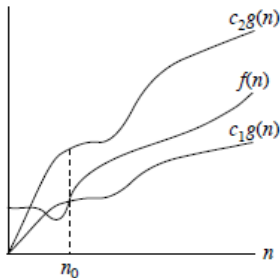
$$n^{2.0001}$$

$$n^2 \lg \lg n$$

$$2^{2^n}$$

Notazione Θ

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2 \in \mathbb{R}^+, n_0 \in \mathbb{N} \text{ t.c.} \\ 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \forall n \geq n_0\}$$



$g(n)$ è un **limite asintoticamente stretto** per $f(n)$

Teorema:

$$f(n) = \Theta(g(n)) \text{ sse } f(n) = O(g(n)) \text{ e } f(n) = \Omega(g(n))$$

Notazione asintotica in equazioni

Nota: Le costanti e i termini di ordine inferiore non sono importanti

Nel lato destro

$O(n^2)$ indica una **qualche funzione anonima nell'insieme** $O(n^2)$

- $f(n) = O(n^2) \rightarrow$ vuol dire $f(n) \in O(n^2)$
- $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$
vuol dire $2n^2 + 3n + 1 = 2n^2 + f(n)$ per qualche $f(n) \in \Theta(n)$
(in particolare: $f(n) = 3n + 1 \Rightarrow$ elimino dettagli dalle formule)

Nel lato sinistro

- Indipendentemente da come scelgo le funzioni anonime a sinistra, posso scegliere quelle a destra per rendere l'equazione valida
- $2n^2 + \Theta(n) = \Theta(n^2)$ vuol dire:
 $\forall$ funzione $f(n) \in \Theta(n) \exists$ una funzione $g(n) \in \Theta(n^2)$ t.c.
 $2n^2 + f(n) = g(n)$

Notazione o

$$o(g(n)) = \{f(n) : \forall c > 0 \in \mathbb{R} \exists n_0 > 0 \in \mathbb{N} \text{ t.c. } 0 \leq f(n) < c \cdot g(n) \forall n \geq n_0\}$$

oppure

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Esempi di funzioni in $o(n^2)$

$$n^{1.9999} = o(n^2)$$

$$n^2 / \lg n = o(n^2)$$

$$n^2 \neq o(n^2)$$

$$n^2 / 1000 \neq o(n^2)$$

Notazione ω

$$\omega(g(n)) = \{f(n) : \forall c > 0 \in \mathbb{R} \exists n_0 > 0 \in \mathbb{N} \text{ t.c.} \\ 0 \leq cg(n) < f(n) \forall n \geq n_0\}$$

Oppure

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

Esempi di funzioni in $\omega(n^2)$

$$n^{2.0001} = \omega(n^2)$$

$$n^2 \lg n = \omega(n^2)$$

$$n^2 \neq \omega(n^2)$$

Confronto di funzioni: proprietà relazionali

Transitività :

$$f(n) = \Theta(g(n)) \text{ e } g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$$

Lo stesso per O , Ω , o e ω

Riflessività :

$$f(n) = \Theta(f(n))$$

Analogamente per O e Ω

Simmetria:

$$f(n) = \Theta(g(n)) \quad \text{sse} \quad g(n) = \Theta(f(n))$$

Simmetria trasposta:

$$f(n) = O(g(n)) \quad \text{sse} \quad g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \quad \text{sse} \quad g(n) = \omega(f(n))$$

Assenza di tricotomia

Numeri Reali

- Per i numeri reali vale la proprietà di tricotomia
- Dati $a, b \in \mathbb{R}$ vale una e sola delle seguenti relazioni
 - ▶ $a < b$
 - ▶ $a = b$
 - ▶ $a > b$

Per il confronto tra funzioni?

- Intuitivamente:
 - ▶ O equivale a $\leq$
 - ▶ Ω equivale a $\geq$
 - ▶ ...
- ma non vale la tricotomia
 - ▶ Esempio: $n^{1+\sin n}$ e n ($1 + \sin n$ oscilla tra 0 e 2)

Logaritmi

Notazioni:

- $\lg n = \log_2 n$ (logaritmo binario)
- $\ln n = \log_e n$ (logaritmo naturale)
- $\lg^k n = (\lg n)^k$
- $\lg \lg n = \lg(\lg n)$ (composizione)
- Le funzioni logaritmiche si applicano soltanto al termine successivo nella formula,
 $\lg n + k$ vuol dire $(\lg n) + k$ e non $\lg(n + k)$

Nell'espressione $\log_b a$

- Se b è costante l'espressione è strettamente crescente quando a cresce
- Se a è costante l'espressione è strettamente decrescente quando b cresce

Identità utili

Per tutti i reali $a > 0$, $b > 0$, $c > 0$ e n (con base del logaritmo $\neq 1$):

$$a = b^{\log_b a}$$

$$\log_c(ab) = \log_c a + \log_c b$$

$$\log_b a^n = n \log_b a$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

$$\log_b(1/a) = -\log_b a$$

$$\log_b a = \frac{1}{\log_a b}$$

$$a^{\log_b c} = c^{\log_b a}$$

Note sulla crescita di funzioni

Quanto cresce $\log \log n$?

- Moolto poco
- Supponiamo di considerare la base 10. Si ha $\log \log 10^{100} = 2$
- Ma quanto è grande 10^{100} ?
- E' un googol (da cui deriva il nome google)
- $70! = 1.198$ googol.
- Il numero **stimato** di **atomi** nell'universo visibile è compreso tra 10^{72} e 10^{87}

Ricorrenze

Ricorrenze

Algoritmo generico divide-et-impera

- *Caso base:*
 - ▶ problema P (dim 1) in $\Theta(1)$ passi
- *Caso ricorsivo:*
 - ▶ dividere problema P (dim $n > 1$)
 - ★ in a sotto-problemi P (dim n/b)
 - ▶ combinare le soluzioni dei sottoproblemi in $f(n)$
- $$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ aT(n/b) + f(n) & \text{se } n > 1 \end{cases}$$

Obiettivo

ottenere una formula chiusa per $T(n)$

Ricorrenza

Equazione (o disequazione) che descrive una funzione in termini del suo valore con input più piccoli

In genere in termini di:

- uno o più casi base e
- se stessa, con argomenti più piccoli

Esempi

Vedremo

- soluzioni esatte a ricorrenze "esatte", poi
- con funzioni asintotiche nella definizione della ricorrenza e nella soluzione della ricorrenza

Fattoriale

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ T(n-1) + 1 & \text{se } n > 1 \end{cases}$$

Soluzione: $T(n) = n$

Dimostrazione per induzione

Suppongo che sia vero per $k < n$ ($T(k) = k$ per $k < n$) e **verifico**:

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= (n-1) + 1 = n \end{aligned}$$

Metodi di soluzione

- **Metodo di sostituzione** ipotizzo una soluzione e mostro che è corretta con *induzione matematica*
- **Metodo dell'albero di ricorsione.** I **nodi** rappresentano i costi ai vari livelli della ricorsione
- **Metodo dell'esperto** fornisce soluzione per:

$$T(n) = aT(n/b) + f(n)$$

- Ricorrenza e soluzione spesso espresse in notazione asintotica
 - ▶ Es: $T(n) = 2T(n/2) + \Theta(n)$; soluzione $T(n) = \Theta(n \lg n)$.
- Se serve soluzione esatta (non asintotica) devo dettagliare le condizioni al contorno

Metodo di sostituzione

Due passi principali

- 1 Ipotizzare la soluzione
- 2 Con induzione matematica trovo le costanti e **dimosto** che la soluzione è corretta

Soluzione Esatta

Ricorrenza con una funzione esatta $\Rightarrow$ soluzione è esatta

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2T(\frac{n}{2}) + n & \text{se } n > 1 \end{cases}$$

❶ **Ipotesi:** $T(n) = n \lg n + n$

❷ **Induzione:**

Caso base $n = 1 \Rightarrow n \lg n + n = 1 = T(n)$

Passo induttivo: Ipotesi induttiva $T(k) = k \lg k + k \quad \forall k < n$
uso ipotesi induttiva per $k = n/2$

$$\begin{aligned} T(n) &= 2T(\frac{n}{2}) + n = 2(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}) + n \text{ (ipotesi induttiva)} \\ &= n \lg \frac{n}{2} + n + n = n(\lg n - \lg 2) + n + n \\ &= n \lg n - n + n + n = n \lg n + n \end{aligned}$$

Soluzione Asintotica (usata più spesso)

$$T(n) = \begin{cases} O(1) & \text{se } n \text{ sufficientemente piccolo} \\ 2T(\frac{n}{2}) + \Theta(n) & \text{altrimenti} \end{cases}$$

- Soluzione con notazione asintotica: $T(n) = \Theta(n \lg n)$
- Non si mostra in dettaglio il caso base
 - ▶ $T(n)$ è costante $\forall$ n costante
 - ▶ Interessa soluzione asintotica $\Rightarrow$
posso sempre scegliere un caso base $O(1)$

Risolvere $T(n) = 2T(n/2) + \Theta(n)$ col metodo di sostituzione

- Dare un **nome** alla costante per $\Theta(n)$
- Mostrare il limite superiore (O) e inferiore (Ω) separatamente

Limite superiore di $T(n) = 2T(\frac{n}{2}) + O(n)$

- $T(n) \leq 2T(n/2) + cn$ per qualche costante positiva c
(definizione di $O()$)
- **Ipotesi** $T(n) = O(n \lg n)$
(cioè $T(n) \leq d \cdot n \cdot \lg n$ per $d > 0$ costante)
- Se trovo d costante t.c. la ricorrenza è verificata $\Rightarrow$ verifico che $T(n) = O(n \lg n)$
- Nella sostituzione non ho funzioni asintotiche, ma funzioni che dipendono da d e c che quindi posso risolvere in forma esatta

Sostituzione

Ipotesi che $T(n) = O(n \lg n)$ cioè che

$T(k) \leq d \cdot k \cdot \lg k$ per qualche costante positiva d (e $k < n$)

$$T(n) \leq 2T\left(\frac{n}{2}\right) + c n$$

$$\leq 2\left(d \frac{n}{2} \lg \frac{n}{2}\right) + c n$$

$$= d n \lg \frac{n}{2} + c n$$

$$= d n \lg n - d n + c n$$

$$\leq d n \lg n$$

valido se $-dn + cn \leq 0$, cioè $d \geq c$

$$\Rightarrow T(n) = O(n \lg n)$$

Limite inferiore

$T(n) \geq 2T(n/2) + cn$ per qualche costante positiva c
(definizione di $\Omega()$)

Ipotesi: $T(n) \geq d n \lg n$ per qualche costante positiva d

Sostituzione

$$\begin{aligned}
 T(n) &\geq 2T\left(\frac{n}{2}\right) + cn \\
 &\geq 2\left(d\frac{n}{2} \lg \frac{n}{2}\right) + cn \\
 &= d n \lg \frac{n}{2} + cn \\
 &= d n \lg n - d n + cn \\
 &\geq d n \lg n
 \end{aligned}$$

valido se $-dn + cn \geq 0$, cioè $d \leq c$

Quindi

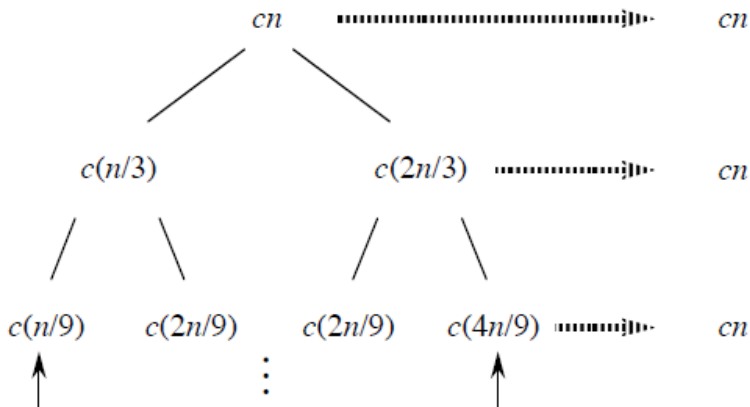
$T(n) = \Omega(n \lg n) \Rightarrow T(n) = \Theta(n \lg n)$
($d = c$ per O e Ω non sono sempre uguali)

Metodo dell'albero di ricorsione

- Intuisco un'ipotesi che verifico col metodo di sostituzione
- Nodo $\rightarrow$ costo di qualche sotto problema
- Sommo i costi all'interno di ogni livello e poi di tutti i livelli

Esempio

- $T(n) = T(n/3) + T(2n/3) + \Theta(n)$
- Limite **superiore** (O): $T(n) \leq T(n/3) + T(2n/3) + cn$
- Limite **inferiore**: (Ω) $T(n) \geq T(n/3) + T(2n/3) + cn$
- Albero:



Caratteristiche albero

- $\log_3 n$ **livelli pieni**

Altezza a sx (x) t.c.

$$3^x = n \Rightarrow x = \log_3 n$$

- Altezza albero: $\log_{3/2} n$
- Ogni livello $\leq cn$
- Idea per **limite superiore**: $\leq dn \log_{3/2} n = O(n \lg n)$
- Idea per **limite inferiore**: $\geq dn \log_3 n = \Omega(n \lg n)$
- ... molto complicato fare i conti esatti con l'albero!

1. Limite superiore - Ipotesi: $T(n) \leq dn \lg n$

$$\begin{aligned}
 T(n) &\leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn \leq d\left(\frac{n}{3}\right) \lg\left(\frac{n}{3}\right) + d\left(\frac{2n}{3}\right) \lg\left(\frac{2n}{3}\right) + cn \\
 &= \left(d\left(\frac{n}{3}\right) \lg n - d\left(\frac{n}{3}\right) \lg 3\right) + \left(d\left(\frac{2n}{3}\right) \lg n - d\left(\frac{2n}{3}\right) \lg\left(\frac{3}{2}\right)\right) + cn \\
 &= d n \lg n - d\left(\left(\frac{n}{3}\right) \lg 3 + \left(\frac{2n}{3}\right) \lg\left(\frac{3}{2}\right)\right) + cn \\
 &= d n \lg n - d\left(\left(\frac{n}{3}\right) \lg 3 + \left(\frac{2n}{3}\right) \lg 3 - \left(\frac{2n}{3}\right) \lg 2\right) + cn \\
 &= d n \lg n - dn\left(\lg 3 - \frac{2}{3}\right) + cn \leq \\
 &\leq d n \lg n
 \end{aligned}$$

$$\text{se } -dn\left(\lg 3 - \frac{2}{3}\right) + cn \leq 0 \text{ quindi per } d \geq \frac{c}{\lg 3 - \frac{2}{3}}$$

$$\Rightarrow T(n) = O(n \lg n)$$

2. Limite inferiore - Ipotesi: $T(n) \geq d n \lg n$

- **Sostituzione:** Stessa del limite superiore, ma cambiando $\leq$ con $\geq$
- Corretto se
 - ▶ $0 < d \leq \frac{c}{\lg 3 - 2/3}$
- Quindi, $T(n) = \Omega(n \lg n)$
- Poichè
 - ▶ $T(n) = O(n \lg n)$
 - ▶ $T(n) = \Omega(n \lg n)$
- $\Rightarrow$
 $T(n) = \Theta(n \lg n)$

Metodo dell'esperto

Teorema dell'esperto

- Sia $T(n)$ una funzione definita sui naturali dalla ricorrenza:
$$T(n) = aT(n/b) + f(n)$$

(n/b indica $\lfloor n/b \rfloor$ o $\lceil n/b \rceil$)
- $a \geq 1, b > 1$ costanti e $f(n) > 0$ asintoticamente

Note:

- $a \geq 1$
- $b > 1$
- $f(n) > 0$ per $n > n_0$

Allora $T(n)$ può essere asintoticamente limitata nei seguenti modi:

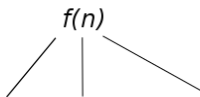
- **Caso 1:** Se $f(n) = O(n^{\log_b a - \epsilon})$ per qualche costante $\epsilon > 0$
allora: $T(n) = \Theta(n^{\log_b a})$
- **Caso 2:** Se $f(n) = \Theta(n^{\log_b a})$
allora: $T(n) = \Theta(n^{\log_b a} \lg n)$
- **Caso 3:** Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ per qualche costante $\epsilon > 0$ e
 $f(n)$ t.c. $af(n/b) \leq cf(n)$ per qualche costante $c < 1$ e $\forall n \geq n_0$
allora: $T(n) = \Theta(f(n))$

- Confronta $n^{\log_b a}$ con $f(n)$
- Soluzione dipende dalla **più grande**
- I tre casi **non coprono tutte** le funzioni possibili
- Condizione di regolarità : voglio essere sicuro che quando scendo nell'albero $f()$ diventa più piccola

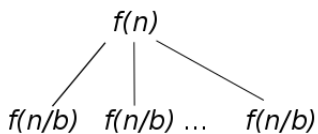
Intuizione

del teorema dell'esperto
a partire dall'albero di ricorsione

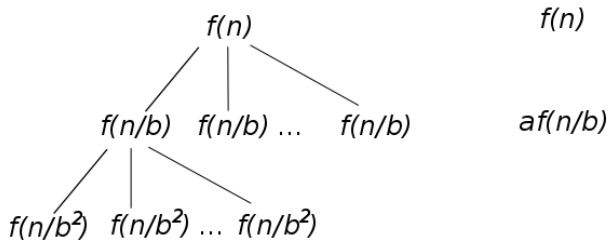
$$T(n) = aT(n/b) + f(n)$$



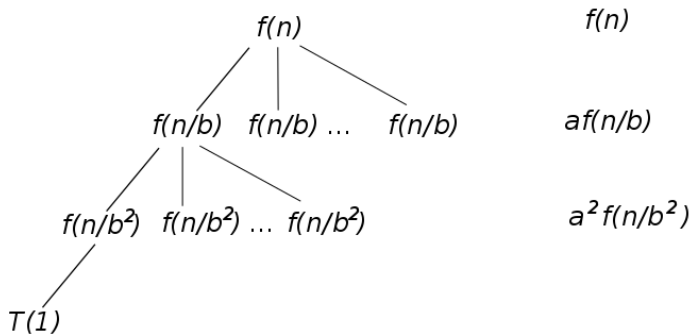
$$T(n) = aT(n/b) + f(n)$$

 $f(n)$

$$T(n) = aT(n/b) + f(n)$$



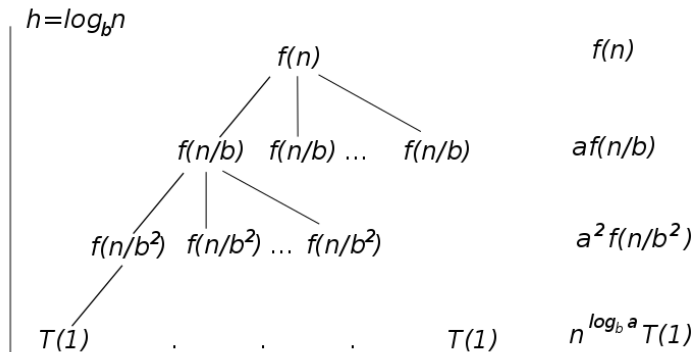
$$T(n) = aT(n/b) + f(n)$$



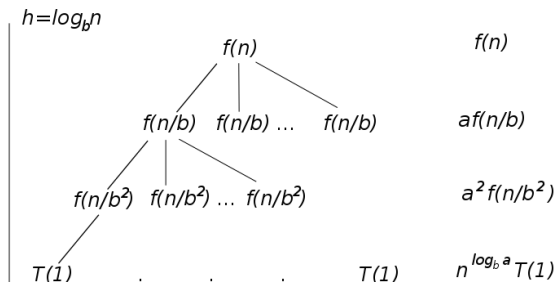
Quante foglie?

- Altezza albero: quando $n/b^h = 1 \Rightarrow n = b^h \Rightarrow h = \log_b n$
- Foglie = $a^h = a^{\log_b n} = n^{\log_b a}$

$$T(n) = aT(n/b) + f(n)$$



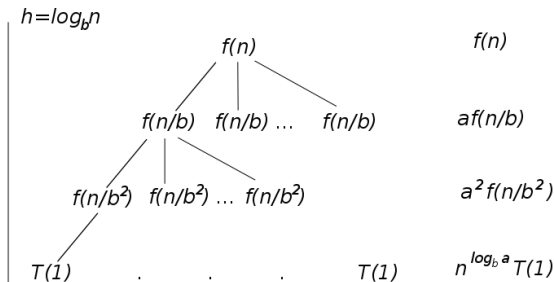
Caso 1



$$\exists \epsilon > 0 : f(n) = O(n^{\log_b a - \epsilon})$$

- Costo cresce dalla radice alle foglie
- Costo dominato dalle foglie
- $\Rightarrow \Theta(n^{\log_b a})$

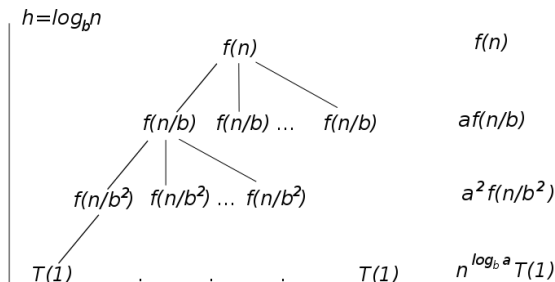
Caso 2



$$f(n) = \Theta(n^{\log_b a})$$

- Il costo è all'incirca lo stesso in ognuno dei $\log_b n$ livelli
- $\Rightarrow \Theta(f(n) \lg_n) = \Theta(n^{\log_b a} \lg_n)$

Caso 3



$$\exists \epsilon > 0 : f(n) = \Omega(n^{\log_b a + \epsilon})$$

- Costo decrementa dalla radice alle foglie
- Costo dominato dalla radice
- $\Rightarrow \Theta(f(n))$

Condizione di regolarità del caso 3

- Vale sempre se $f(n) = n^k$ e $f(n) = \Omega(n^{\log_b a + \epsilon})$ per qualche costante $\epsilon > 0$

- $\Rightarrow$ OK se $f(n)$ è un polinomio

- **Dimostrazione**

▶ $f(n) = \Omega(n^{\log_b a + \epsilon})$ e $f(n) = n^k \Rightarrow$

$$n^k > n^{\log_b a + \epsilon} = n^{\log_b a} \cdot n^\epsilon > n^{\log_b a} \Rightarrow$$

$$k > \log_b a$$

- ▶ i due lati come esponenti con base b :

$$b^k > b^{\log_b a} = a \Rightarrow a/b^k < 1$$

- ▶ a, b, k costanti $\Rightarrow$ scelgo $c = a/b^k \Rightarrow c < 1$ costante

▶ $a f(n/b) = a(n/b)^k = (a/b^k)n^k = cf(n)$

$\Rightarrow$ condizione di regolarità soddisfatta

Esempi

- $T(n) = 4T(\frac{n}{2}) + n$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n$$

Caso 1: $f(n) = O(n^{2-\epsilon})$ con $\epsilon = 1 \Rightarrow T(n) = \Theta(n^2)$

- $T(n) = 4T(\frac{n}{2}) + n^2$

$$\Rightarrow n^{\log_b a} = n^2; f(n) = n^2$$

Caso 2: $f(n) = \Theta(n^2) \Rightarrow T(n) = \Theta(n^2 \lg n)$

- $T(n) = 4T(\frac{n}{2}) + n^3$

$$\Rightarrow n^{\log_b a} = n^2; f(n) = n^3$$

Caso 3: $f(n) = \Omega(n^{2+\epsilon})$ con $\epsilon = 1$

e condizione regolarità $4(\frac{n}{2})^3 \leq c n^3$ con $c = \frac{1}{2}$

$$\Rightarrow T(n) = \Theta(n^3)$$

Ricerca binaria

Trovare un elemento in un array ordinato:

- **Divide:** Verificare l'elemento di mezzo
- **Impera:** Cercare ricorsivamente in un sotto-array
- **Combina:** Banale

Ricorrenza per ricerca binaria:

$$T(n) = 1T(n/2) + \Theta(1)$$

- Da teorema dell'esperto:
 $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow$
Caso 2 $\Rightarrow T(n) = \Theta(\lg n)$

Potenza intera

Problema: Calcolare a^n , dove $n \in \mathbb{N}$

Algoritmo **intuitivo**: $\Theta(n)$

Algoritmo divide et impera

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{se } n \text{ è pari} \\ a^{n/2} \cdot a^{n/2} \cdot a & \text{se } n \text{ è dispari} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\lg n)$$

Cenni di calcolo delle probabilità

(distribuzione di probabilità discreta)

Calcolo delle probabilità

Probabilità

- La probabilità è definita su uno **spazio degli eventi** S
 S è un insieme di **eventi elementari** (esito di un esperimento)
- Per il lancio di due monete (ognuna T o C):
 $S = \{TT, TC, CT, CC\}$

Eventi

- Un **evento** è un *sottoinsieme* di S (es. due lanci uguali: $\{TT, CC\}$)
 - ▶ Un evento si realizza quando si realizza uno qualsiasi degli eventi elementari che lo compongono
- A ciascun evento elementare si assegna una probabilità (reale compreso tra 0 e 1)
 - ▶ la somma delle probabilità degli eventi elementari è 1
 - ▶ la probabilità di un evento è la somma delle probabilità degli eventi elementari che lo costituiscono

Definizioni di probabilità

Interpretazione classica

- Si assegna la stessa probabilità a tutti gli eventi elementari
- $\Rightarrow$ la probabilità di un evento E è il rapporto tra il numero di eventi elementari che lo compongono (*casi favorevoli*) e il numero totale di eventi elementari (*casi possibili*)

$$P(E) = \frac{\text{casi favorevoli}}{\text{casi possibili}}$$

- ▶ bisogna scegliere opportunamente gli eventi elementari
 - ▶ molto usata nel nostro corso
- Non è detto che gli eventi elementari siano equiprobabili (probabilità di prendere 30L ad un esame...)

Definizioni di probabilità

Interpretazione frequentista

- Per assegnare la probabilità ad un evento E si ripete l'esperimento n volte e si osserva il numero di volte in cui E si è verificato

$$P(E) = \text{frequenza relativa} = \frac{\text{casi favorevoli su } n}{n}$$

- Basata sulla **legge empirica del caso**: *la frequenza relativa di un evento tende a stabilizzarsi all'aumentare del numero delle prove effettuate*
 - ▶ è una legge empirica, non un teorema!
 - ▶ è un caso particolare della *legge dei grandi numeri*
- La probabilità di un evento E è il limite delle frequenze relative dei casi favorevoli quando il numero delle prove tende all'infinito

$$P(E) = \lim_{n \rightarrow \infty} \frac{\text{casi favorevoli su } n}{n}$$

Definizioni di probabilità

Interpretazione assiomatica

- Una **distribuzione di probabilità** $P()$ è una **relazione** che associa gli eventi di S ai numeri reali t.c. sono soddisfatti gli **assiomi di probabilita'**:
 - ① $P(E) \geq 0 \ \forall \text{ evento } E$
 - ② $P(S) = 1$
 - ③ $P(A \cup B) = P(A) + P(B)$ se A e B sono **mutuamente esclusivi**
- $P(E)$ è la probabilità dell'evento E (possibilità che un evento incerto si manifesti)

Leggi della Probabilità

Probabilità dell'evento contrario

- $\bar{E}$ è l'evento contrario di E
(tutti gli eventi elementari che non sono in E)
- La somma di tutti gli eventi elementari è 1 $\Rightarrow$
 $P(\bar{E}) = 1 - P(E)$

Leggi della Probabilità

Probabilità totale per eventi incompatibili

- A e B sono incompatibili se non hanno eventi elementari in comune ($A \cap B = \emptyset$)
- Probabilità che si verifichi uno dei due eventi (probabilità totale):

$$P(A \cup B) = P(A) + P(B)$$

Probabilità totale per eventi compatibili

- A e B sono compatibili se hanno eventi elementari in comune (devo contarli una volta sola)
- $$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$
- Esempio: estrazione di una carta da un mazzo con 40 carte
 - ▶ $A =$ "estrai una carta di cuori"; $B =$ "estrai una figura"
 - ▶ $A \cup B =$ "estrai una carta di cuori o una figura"
 - ▶ $A \cap B =$ "estrai una figura di cuori"
 - ▶
$$P(A \cup B) = \frac{10}{40} + \frac{12}{40} - \frac{3}{40}$$

Leggi della Probabilità

Probabilità composta per eventi indipendenti

- Due eventi sono **indipendenti** se il realizzarsi di uno **non influenza** la probabilità del realizzarsi dell'altro
- probabilità che si realizzino entrambi (probabilità composta):
$$P(A \cap B) = P(A) \cdot P(B)$$

Probabilità composta per eventi dipendenti

- indico con $P(B|A)$ la probabilità **condizionata** di B dato A (probabilità che si verifichi B supponendo che si sia verificato A)
- $P(A \cap B) = P(A) \cdot P(B|A)$

Probabilità condizionata e indipendenza

Definizione di probabilità condizionata

- Si ha *a priori qualche informazione sull'esito di un esperimento*
- $P(A|B) = \frac{P(A \cap B)}{P(B)}$
- Due eventi sono **indipendenti** se $P(A \cap B) = P(A) \cdot P(B)$
 - ▶ In tal caso $P(A|B) = P(A)$

Teorema di Bayes

- Dalla definizione di probabilità condizionata e dalla legge commutativa ($A \cap B = B \cap A$) si ha:
- $P(A \cap B) = P(B)P(A|B) = P(A)P(B|A)$
- Quindi $P(A|B) = \frac{P(A)P(B|A)}{P(B)}$

Variabili Casuali

Variabile casuale (o stocastica, o aleatoria)

- E' una variabile che può assumere valori diversi in dipendenza di un qualche fenomeno casuale
 - ▶ usiamo le lettere maiuscole per indicare la variabile, e le lettere minuscole per i suoi possibili valori

Esempi

- $X = \{ \text{numero di teste in tre lanci di monete} \}$
 - ▶ un suo possibile valore (esito) sarà: $x = 2$
- $X = \{ \text{risultato del lancio di un dado} \}$
 - ▶ la variabile casuale ha 6 possibili valori: 1, 2, 3, 4, 5, 6 (equiprobabili)
- esempio più complesso: risultato del lancio di due dati
 - ▶ valori compresi tra 2 e 12 **NON** equiprobabili

Distribuzione di probabilità

- I possibili esiti di un esperimento e le loro probabilità sono sintetizzati in distribuzioni di probabilità
- Una variabile casuale si riferisce all'esito di un fenomeno casuale
- A ogni possibile esito è associata una probabilità di verificarsi
- La distribuzione di probabilità di una variabile casuale elenca **i suoi possibili valori e le relative probabilità**

Distribuzione di probabilità

per variabili casuali discrete

- La variabile casuale X è detta **discreta** se assume un numero finito o un'infinità numerabile di valori $\{x_1, x_2, \dots, x_n\}$ (è definita su un insieme discreto)
- La distribuzione di probabilità di una variabile casuale discreta X assegna una probabilità $P(X = x_i)$ (o $P(x)$) a ogni possibile valore assunto $x \in \{x_1, x_2, \dots, x_n\}$
 - ▶ $\forall x \ 0 \leq P(x) \leq 1$
 - ▶ $\sum_{i=1}^n P(X = x_i) = 1$
- Anche chiamata funzione di massa (o funzione massa di probabilità o densità discreta): $p(x) = P(X = x)$

Sintesi di una distribuzione

- Per descrivere le caratteristiche di una distribuzione di probabilità si possono usare sintesi numeriche
 - ▶ ad esempio media, mediana, quartili, varianza e deviazione standard (o scarto quadratico medio)
- Le misure che ricorrono maggiormente sono:
 - ▶ la **media** per descrivere la tendenza centrale della distribuzione;
 - ▶ la **deviazione standard** e la **varianza** per descrivere la sua variabilità

Valore atteso

di una distribuzione discreta

- Il valor medio (o valore atteso) per una variabile casuale indica *cosa ci si aspetta* in media in una lunga serie di osservazioni
- Il valore atteso della variabile casuale discreta X è definito come:

$$\mu = E[X] = \sum_{i=1}^n x_i P(x_i)$$

Esempio: lancio di due monete

- variabile casuale $X = \{\text{numero di teste}\} = \{0, 1, 2\}$
- Valore atteso di X
 - ▶ $P(0) = 0.25, P(1) = 0.50, P(2) = 0.25$
 - ▶ $\rightarrow \mu = E[X] = \sum_{i=1}^3 x_i P(x_i) = 0 \cdot P(0) + 1 \cdot P(1) + 2 \cdot P(2) = 1$
 - ▶ ci aspettiamo che in media si ottenga una testa in due lanci consecutivi

Linearità del valore atteso

- Se $E[X]$ e $E[Y]$ sono definiti si ha: $E[X + Y] = E[X] + E[Y]$
- Se X è una variabile casuale, qualsiasi funzione $g(x)$ definisce una nuova variabile casuale $g(X)$

$$E[g(X)] = \sum_x g(x)P\{X = x\}$$

- In particolare se $g(x) = ax$ si ha $E[aX] = aE[X]$ (i valori attesi sono lineari)

Variabilità distribuzione discreta

Data la definizione del valor medio allora si avrà per le misure di variabilità:

- **Varianza** di una variabile casuale discreta

$$\sigma^2 = E[X - \mu]^2 = \sum_{i=1}^n (x_i - \mu)^2 P(x_i)$$
- **Deviazione standard** di una variabile casuale discreta

$$\sigma = \sqrt{\sum_{i=1}^n (x_i - \mu)^2 P(x_i)}$$

Distribuzione Uniforme Discreta

Variabile aleatoria Uniforme Discreta

- Se S è finito e $\forall s \in S$ si ha $P\{s\} = 1/|S|$
- Esempio: **moneta perfetta**
- La funzione densità di probabilità è:

$$f_Y(y) = \begin{cases} \frac{1}{N} & \text{se } y = 1 \dots N \\ 0 & \text{altrimenti} \end{cases}$$

Variabili casuali indicatrici

- Dato uno spazio dei campioni S e un evento A , si definisce la variabile casuale indicatrice:

$$I\{A\} = \begin{cases} 1 & \text{se si verifica } A \\ 0 & \text{se non si verifica } A \end{cases}$$

Lemma

Per un evento A , sia $X_A = I\{A\}$

Allora $E[X_A] = Pr\{A\}$

Dim

$\bar{A}$ complemento di A :

$$\begin{aligned} E[X_A] &= E[I\{A\}] \\ &= 1 \cdot Pr\{A\} + 0 \cdot Pr\{\bar{A}\} \text{ (def valore atteso)} \\ &= Pr\{A\} \end{aligned}$$

Valore atteso: media ponderata dei possibili valori di $I\{A\}$

Metodo per conversione tra **probabilità** e **valori attesi**

Esempi

Numero atteso di teste lanciando una moneta una volta

- Moneta imparziale
- Spazio dei campioni: $S = \{T, C\}$
- $Pr\{T\} = Pr\{C\} = 1/2$
- Variabile casuale indicatrice:
 $X_T = I\{T\}$
 X_T conta il numero di teste in un lancio
- Dal lemma:
 $E[X_T] = Pr\{T\} = 1/2$

Numero atteso di teste in n lanci

- Questo numero NON è una probabilità !
- X : variabile casuale per il numero di teste in n lanci
- Variabile casuale indicatrice

- ▶ $X_i = I\{i\text{-mo lancio è evento } T\}$

- ▶ $X = \sum_{i=1}^n X_i$

- ▶ Numero atteso di teste (T):

$$E[X] = E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n Pr\{T\} = \sum_{i=1}^n 1/2 = n/2$$

- Linearità valori attesi:
valore atteso della somma = somma dei valori attesi

Analisi Probabilistica e Algoritmi Randomizzati

Il problema delle assunzioni

Scenario

- Agenzia per assumere un impiegato
- L'agenzia intervista un candidato al giorno
- Dopo il colloquio si decide subito se assumere (e si licenzia l'attuale impiegato)
 - ▶ **Costo colloquio:** c_c per candidato
 - ▶ **Costo assunzione:** c_a per candidato
- Supponiamo $c_a > c_c$ (forse $c_a \gg c_c$)
- Assumiamo sempre il miglior candidato visto

Obiettivo

- Determinare il costo

Pseudocodice

- Candidati numerati $1, \dots, n$
- Il primo candidato è sempre assunto

HIRE-ASSISTANT(n)

$best \leftarrow 0$ // Candidato 0 e' fittizio

for $i \leftarrow 1$ **to** n

 colloquio con candidato i

if (candidato i migliore di $best$)

$best \leftarrow i$

 assumi candidato i

Paradigma **comune** (non solo assunzioni)

- Trovare min (max) in una sequenza

Correttezza

- Con invariante di ciclo

Costo

- n **candidati** ne assumo m
- "Costo": $O(n \cdot c_c + m \cdot c_a)$
- Dipende da **ordine** dei candidati (m varia)

Caso peggiore

Assumo **tutti** gli n candidati (arrivano in ordine crescente)

Costo $\Theta(nc_c + nc_a) = \Theta(n c_a)$

($c_a > c_c$)

Caso migliore?

- Il primo assunto è il miglior candidato

Analisi probabilistica

- I candidati arrivano in ordine **casuale**:
 - ▶ Assegno un **rango** ad ogni candidato (posizione in una ipotetica graduatoria):
 $pos(i)$ è la posizione del candidato i
 $pos(i) \in 1 \dots n$ e $pos(i) \neq pos(j)$ sse $i \neq j$
 - ▶ $\langle pos(1), pos(2), \dots, pos(n) \rangle$ è una **permutazione** della lista dei candidati $\langle 1, 2, \dots, n \rangle$
 - ▶ Ognuna delle $n!$ permutazioni ha **uguale probabilità**
- **Idea** dell'analisi probabilistica:
Uso conoscenza su distribuzione degli input per analizzare l'algoritmo

Analisi del problema dell'assunzione

- Candidati arrivano in ordine **casuale**
- X è una variabile casuale:
numero di volte che si assume una nuova persona
- **Variabili casuali indicatrici** $X_1, X_2, \dots, X_n$:

$$X_i = I\{\text{candidato } i \text{ è assunto}\} = \begin{cases} 1 & \text{se candidato assunto} \\ 0 & \text{se candidato non assunto} \end{cases}$$

- $X = X_1 + X_2 + \dots + X_n$
- Lemma $\Rightarrow E[X_i] = Pr\{\text{candidato } i \text{ è assunto}\}$
- Devo **calcolare** $Pr\{\text{candidato } i \text{ è assunto}\}$

$Pr\{ \text{candidato } i \text{ è assunto} \}$

- Candidato i assunto sse è migliore dei precedenti $1, 2, \dots, i-1$
- Candidati in ordine casuale
 $\Rightarrow$ ciascuno dei primi i candidati
 ha la stessa probabilità di essere il migliore
- $Pr\{ \text{candidato } i \text{ è migliore fino ad ora} \} = 1/i$

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i] \\ &= \sum_{i=1}^n 1/i = \ln n + O(1) \end{aligned}$$

- **Serie armonica:** $\sum_{k=1}^n \frac{1}{k} = \ln n + O(1)$
- **Costo atteso di assunzione:** $O(c_a \ln n)$
 molto meglio del caso peggiore $O(n c_a)$

Algoritmi randomizzati

E' corretta l'analisi vista?

- Posso non conoscere la distribuzione delle interviste (la permutazione dei candidati)
- Esempio: alcune sequenze di interviste **non capitano mai**.
- Non posso essere sicuro che ognuna delle $n!$ permutazioni capiti con uguale probabilità

Cambio lo scenario

- Uso la randomizzazione **dentro l'algoritmo** per **imporre** una distribuzione agli input:
 - ▶ L'agenzia manda in anticipo la lista degli n candidati
 - ▶ Ogni giorno scelgo un candidato a caso dalla lista
- **Forzo** un ordine casuale

Algoritmi randomizzati

- Un algoritmo è randomizzato se il suo **comportamento** è determinato in parte da valori prodotti con un **generatore di numeri (pseudo-)casuali**
- $\text{RANDOM}(a, b)$
ritorna un intero r t.c. $a \leq r \leq b$ e
ciascuno dei $b - a + 1$ valori possibili di r è ugualmente probabile
- RANDOM è implementato con un generatore **pseudo-casuale**:
un metodo *deterministico* che ritorna numeri che
 - ▶ superano test statistici e
 - ▶ "appaiono" casuali

Problema di assunzione randomizzato

- Non intervisto sempre i candidati nell'ordine presentato, ma **permuto casualmente** quest'ordine
- **Randomizzazione** nell'algoritmo non nella distribuzione d'ingresso

RANDOMIZED-HIRE-ASSISTANT(n)

Permutare casualmente la lista di candidati

HIRE-ASSISTANT(n)

Costo atteso di RANDOMIZED-HIRE-ASSISTANT: $O(c_a \ln n)$

Proof: Avendo permutato l'array di input si ha una situazione identica all'analisi probabilistica di HIRE-ASSISTANT deterministico

Differenza sottile

- Il costo di esecuzione non dipende da un particolare input
- Il caso peggiore ha la stessa probabilità degli altri

Permutazione casuale array

- Permuta **sul posto** i dati
- Voglio **permutazione casuale uniforme**, ogni permutazione dei numeri da 1 a n (sono $n!$) è ugualmente probabile

RANDOMIZE-IN-PLACE(A)

$n \leftarrow A.length$

for $i \leftarrow 1$ **to** n

 scambia $A[i] \leftrightarrow A[\text{RANDOM}(i, n)]$

Tempo

$O(1)$ per ogni iterazione $\Rightarrow O(n)$ totale

Quicksort

Quicksort

Caratteristiche principali

- Caso peggiore: $\Theta(n^2)$
- Tempo di esecuzione atteso: $\Theta(n \lg n)$
- Costanti in $\Theta(n \lg n)$ piccole
- Ordina sul posto

Per ordinare $A[p..r]$:

- **Divide:** $A[p..r]$, in $A[p..q-1]$ e $A[q+1..r]$, t.c.
 - ▶ ogni elemento in $A[p..q-1] \leq A[q]$ e
 - ▶ $A[q] \leq$ ogni elemento in $A[q+1..r]$
- **Impera:** Ordina i due sottoarray con chiamate a QUICKSORT
- **Combina:** sottoarray ordinati sul posto

Chiamata iniziale: $\text{QUICKSORT}(A, 1, n)$

$\text{QUICKSORT}(A, p, r)$

```

1  if  $p < r$ 
2       $q \leftarrow \text{PARTITION}(A, p, r)$ 
3       $\text{QUICKSORT}(A, p, q - 1)$ 
4       $\text{QUICKSORT}(A, q + 1, r)$ 

```

q è il pivot

$\text{PARTITION}(A, p, r)$

```

1   $x \leftarrow A[r]$ 
2   $i \leftarrow p - 1$ 
3  for  $j \leftarrow p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i \leftarrow i + 1$ 
6           $A[i] \leftrightarrow A[j]$ 
7   $A[i + 1] \leftrightarrow A[r]$ 
8  return  $i + 1$ 

```

Correttezza di PARTITION

Invariante di ciclo

- ① $A[r] \rightarrow \text{pivot}$
- ② $\forall A[x] \in A[p..i] : A[x] \leq \text{pivot}$
- ③ $\forall A[x] \in A[i + 1..j - 1] : A[x] > \text{pivot}$
- ④ $\forall A[x] \in A[j..r - 1] : A[x]$ non è stato esaminato

Correttezza di PARTITION

Invariante di ciclo

- **Inizializzazione:**

$A[r]$ è il pivot, $A[p..i]$ e $A[i + 1..j - 1]$ sono vuoti

- **Conservazione:**

Se $A[j] \leq \text{pivot}$, scambia $A[j]$ e $A[i + 1]$ e $i++$ e $j++$

Se $A[j] > \text{pivot}$ solo $j++$

- **Conclusione:** Ciclo termina con $j = r$

$A[p..i] < \text{pivot}$; $A[i + 1..r - 1] > \text{pivot}$; $A[r] = \text{pivot}$

Linea 7 sposta il pivot nella posizione corretta $A[i + 1] \leftrightarrow A[r]$

Analisi di QUICKSORT

Tempi di esecuzione

- **Tempo** di PARTITION:
 $\Theta(n)$
- **Tempo** di QUICKSORT
dipende dal partizionamento dei sottoarray:
 - ▶ Se **bilanciati** $\Rightarrow$ veloce come mergesort
 - ▶ Se **sbilanciati** $\Rightarrow$ lento come insertion sort
 - ▶ Caso medio analogo a mergesort, ma costanti più piccole

Intuizione caso peggiore

- Sottoarray completamente sbilanciati (0 in uno e $n - 1$ nell'altro)

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \\ &= \Theta(n^2) \end{aligned}$$

- Come insertion sort
- Quando capita?
 - ▶ Array **ordinato** (insertion sort: $O(n)$)

Caso migliore

- Sottoarray sempre bilanciati $\Rightarrow$
- Ognuno $\leq n/2$ elementi

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n) \end{aligned}$$

Intuizione caso medio (1/2)

- Partizionamento con proporzionalità costante
 - ▶ Suppongo PARTITION divida sempre in 9-a-1:

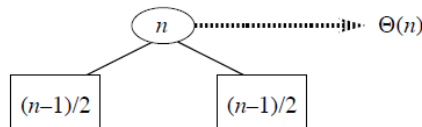
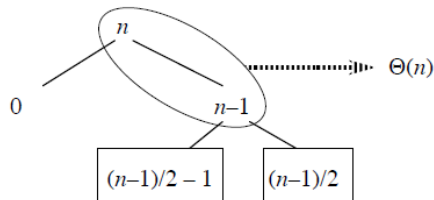
$$\begin{aligned} T(n) &= T(9n/10) + T(n/10) + \Theta(n) \\ &= O(n \lg n) \end{aligned}$$

- Simile a $T(n) = T(n/3) + T(2n/3) + O(n)$
- $\log_{10} n$ livelli pieni
 $\log_{10/9} n$ livelli non vuoti
- $\Omega(\log_{10} n)$ e $O(\log_{10/9} n)$
- Altezza albero di ricorsione: $\Theta(\lg n)$
- Intuizione: il tempo di esecuzione medio di Quicksort è molto più vicino al caso migliore che al peggiore

Intuizione caso medio (2/2)

Una volta sono fortunato una volta no...

- Alternano tagli buoni e cattivi



- Ogni "nodo" a sx ha $\Theta(n) + \Theta(n-1) = \Theta(n)$
- A dx $\Theta(n)$
- Entrambe $O(n \lg n)$ anche se costante a sx $>$ costante a dx

Quicksort randomizzato

- Ipotesi **caso medio**: tutte le permutazioni di ingresso sono ugualmente probabili
 - ▶ Non è sempre vero!!
- **Randomizzo** QUICKSORT
- Permuto casualmente l'input?
- No! Scelgo un **elemento a caso** come pivot

RANDOMIZED-PARTITION(A, p, r)

- 1 $i \leftarrow \text{RANDOM}(p, r)$
- 2 scambia $A[r] \leftrightarrow A[i]$
- 3 **return** PARTITION(A, p, r)

RANDOMIZED-QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$ 
3      RANDOMIZED-QUICKSORT( $A, p, q - 1$ )
4      RANDOMIZED-QUICKSORT( $A, q + 1, r$ )
```

- Randomizzazione **impedisce** a specifici input di causare sempre il **caso peggiore**
- Esempio:
array ordinato è un caso peggiore per QUICKSORT,
non per RANDOMIZED-QUICKSORT!

Analisi di quicksort

Analizziamo:

- Tempo di esecuzione del caso peggiore di QUICKSORT e RANDOMIZED-QUICKSORT (stesso)
- Tempo atteso di RANDOMIZED-QUICKSORT

Caso peggiore

Ad ogni livello peggior taglio:

Ricorrenza per caso peggiore

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q - 1)) + \Theta(n)$$

Ipotesi: $T(n) \leq cn^2$ (vediamo O)

$$T(n) \leq \max_{0 \leq q \leq n-1} (cq^2 + c(n-q-1)^2) + c_1 \cdot n$$

$$= c \cdot \max_{0 \leq q \leq n-1} (q^2 + (n-q-1)^2) + c_1 \cdot n$$

max con $q = 0$ o $q = n-1$ (scelgo $q = 0$)

$$\max_{0 \leq q \leq n-1} (q^2 + (n-q-1)^2) \leq (n-1)^2 = n^2 - 2n + 1$$

$\Rightarrow$

$$T(n) \leq cn^2 - c(2n-1) + c_1 \cdot n \leq cn^2$$

$$\text{se } c(2n-1) \geq c_1 \cdot n \Rightarrow c \text{ t.c. } c(2n-1) > c_1 \cdot n$$

$\Rightarrow$ caso peggiore è $O(n^2)$ E' anche $\Omega(n^2)$

$\Rightarrow$ caso peggiore $\Theta(n^2)$

Caso medio

In realtà **tempo di esecuzione atteso**

- Costo **dominante**: PARTITION
- Ogni volta PARTITION rimuove il pivot
⇒ PARTITION chiamato al più n volte
- Costo PARTITION : **costante** + iterazioni in ciclo for
(= numero di confronti)
- X = numero **totale** di confronti in tutte le chiamate a PARTITION
- Costo totale $O(n + X)$
- Calcoliamo un limite asintotico per X

Quicksort

QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2       $q \leftarrow \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )

```

PARTITION(A, p, r)

```

1   $x \leftarrow A[r]$ 
2   $i \leftarrow p - 1$ 
3  for  $j \leftarrow p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i \leftarrow i + 1$ 
6           $A[i] \leftrightarrow A[j]$ 
7   $A[i + 1] \leftrightarrow A[r]$ 
8  return  $i + 1$ 

```

Calcoliamo un limite al numero totale di confronti X

- Rinomino elementi di A : $z_1, z_2, \dots, z_n$, t.c. $z_i \leq z_j$ se $i < j$
- $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ elementi compresi tra z_i e z_j , inclusi
- z_i e z_j confrontati al massimo una volta
 - ▶ elementi confrontati solo col pivot
 - ▶ pivot non presente in chiamate successive
- $X_{ij} = I\{z_i \text{ confrontato con } z_j\}$ (in qualunque momento)
- $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$

$$E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ e' confrontato con } z_j\}$$

$\Pr\{z_i \text{ confrontato con } z_j\}$

- Numeri **in partizioni diverse mai** confrontati
 - ▶ Se $z_i < pivot < z_j \Rightarrow z_i$ e z_j mai confrontati
- Se z_i o z_j scelto per primo in Z_{ij} allora z_i o z_j confrontato con tutti gli altri elementi di Z_{ij}
 - ▶ $\Rightarrow z_i$ e z_j confrontati SSE il **primo** pivot in Z_{ij} è z_i o z_j
- Pivot scelti casualmente e indipendentemente $|Z_{ij}| = j - i + 1$
 $\Rightarrow$ probabilità z_i o z_j scelto per primo in Z_{ij} è $\frac{1}{j-i+1}$

$$\begin{aligned}
 \Pr\{z_i \text{ e' confrontato con } z_j\} &= \Pr\{z_i \text{ o } z_j \text{ primo pivot scelto da } Z_{ij}\} \\
 &= \Pr\{z_i \text{ primo pivot scelto da } Z_{ij}\} \\
 &\quad + \Pr\{z_j \text{ primo pivot scelto da } Z_{ij}\} \\
 &= \frac{1}{j-i+1} + \frac{1}{j-i+1} \\
 &= \frac{2}{j-i+1}
 \end{aligned}$$

Sostituiamo in $E[X]$

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ e' confrontato con } z_j\} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

Cambiamo variabili ($k = j - i$)

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^{n-1} O(\lg n) \\ &= O(n \lg n) \end{aligned}$$

⇒ tempo di esecuzione atteso di RANDOMIZED-QUICKSORT

- Era $O(n + X)$
- Quindi $O(n \lg n)$

Ordinamento in tempo lineare

Ordinamento per confronti

- Usa **confronto** di elementi per ordinare

Limiti inferiori per ordinamento

Algoritmo	Tempo di esecuzione caso peggiore	Tempo di esecuzione atteso/ caso medio
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$ (atteso)
...	...	...

Nessun ordinamento può essere meglio di $n \lg n$?

di sicuro $\Omega(n)$ per esaminare tutti gli input

Tempo esecuzione ordinamento

Albero di decisione

- Albero binario che rappresenta i possibili confronti fatti da:
 - ▶ uno specifico algoritmo di ordinamento
 - ▶ su input di una data dimensione
- Conto solo **confronti**
- **Astrae** da tutto il resto: controllo di flusso, spostamento di dati...

Per ogni algoritmo di ordinamento basato su **confronti**

- Un albero per ogni n
- Albero modella tutti i possibili percorsi di esecuzione

Lunghezza del cammino più lungo dalla radice alle foglie?

Caso peggiore

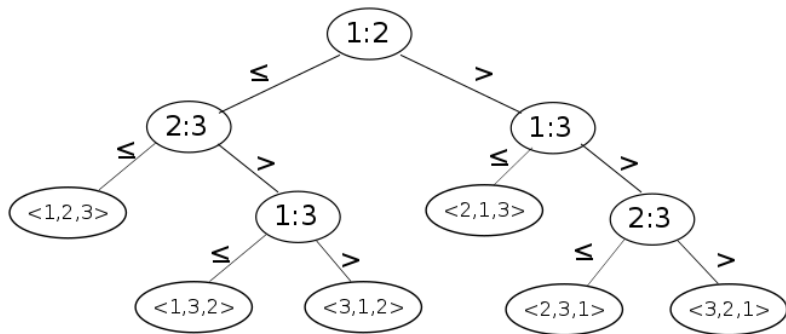
- Dipende dall'algoritmo: Insertion sort: $\Theta(n^2)$ Merge sort: $\Theta(n \lg n)$

INSERTION-SORT(A)

```
1  for  $j \leftarrow 2$  to  $A.length$ 
2       $key \leftarrow A[j]$ 
3       $i \leftarrow j - 1$ 
4      while  $i > 0$  and  $A[i] > key$ 
5           $A[i + 1] \leftarrow A[i]$ 
6           $i \leftarrow i - 1$ 
7       $A[i + 1] \leftarrow key$ 
```

Insertion sort su 3 elementi

- radice corrisponde a ciclo per $j = 2$
- altri nodi $j = 3$
- nodi interni: secondo valore è la chiave
- le foglie sono permutazioni dei valori in ingresso



Lemma

Ogni albero binario di altezza h ha al massimo 2^h foglie

h = altezza l_h = # di foglie,

Allora $l_h \leq 2^h$

Dim

Induzione su h

- **Base:** $h = 0$ Albero con un solo nodo (foglia) $\Rightarrow 2^h = 1$

- **Passo induttivo:**

Supponiamo **vero** per $h - 1$

Ogni **foglia** dell'albero di altezza $h - 1$ ha al massimo 2 nuove **foglie**

$$l_h \leq 2 \cdot l_{h-1} = 2 \cdot 2^{h-1} \quad (\text{ipotesi induttiva})$$

$$= 2^h$$

Teorema

Ogni albero di decisione che ordina n elementi ha altezza $\Omega(n \lg n)$

Dim

- Almeno una foglia per ogni permutazione: $l \geq n!$

Altrimenti non sarebbe corretto!

- Lemma:

$$2^h \geq l \geq n! \Rightarrow 2^h \geq n!$$

- $\lg$ di entrambi i membri:

$$h \geq \lg(n!)$$

- Approssimazione di Stirling: $n! > \left(\frac{n}{e}\right)^n$

$$h \geq \lg(n!) \geq \lg\left(\frac{n}{e}\right)^n = n \lg\left(\frac{n}{e}\right)$$

$$= n \lg n - n \lg e = \Omega(n \lg n)$$

- Nota:** Ω perché c'è $\geq$

⇒ Merge sort

E' un **ordinamento per confronto asintoticamente ottimo**

Ordinamento in tempo lineare

E' possibile ordinare in meno di $\Omega(n)$?

- Ordinamento **non** basato su confronti
- Niente è gratis: abbiamo bisogno di più spazio ...

Counting sort

- *Ipotesi essenziale* :
numeri da ordinare sono **numeri naturali** $\in \{0, 1, \dots, k\}$
- **Input**: $A[1..n]$, dove $A[j] \in \{0, 1, \dots, k\}$ per $j = 1, 2, \dots, n$
- **Output**: $B[1..n]$, ordinati
 B è già allocato
- **Memoria aggiuntiva**: $C[0..k]$

COUNTING-SORT(A, B, k)

```
1  sia  $C[0..k]$  un nuovo array
2  for  $i \leftarrow 0$  to  $k$ 
3       $C[i] \leftarrow 0$ 
4  for  $j \leftarrow 1$  to  $A.length$ 
5       $C[A[j]] \leftarrow C[A[j]] + 1$ 
6  //  $C[i]$  ora contiene il numero di elementi uguali a  $i$ 
7  for  $i \leftarrow 1$  to  $k$ 
8       $C[i] \leftarrow C[i] + C[i - 1]$ 
9  //  $C[i]$  ora contiene il numero di elementi minori o uguali a  $i$ 
10 for  $j \leftarrow A.length$  downto 1
11      $B[C[A[j]]] \leftarrow A[j]$ 
12      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Nota

C è riempito a partire dalla posizione 0 (primo valore della chiave)

Esempio: $A = [2_1, 5_1, 3_1, 0_1, 2_2, 3_2, 0_2, 3_3]$

- $k = 5$
- 1° **for** : $C[0, 0, 0, 0, 0, 0]$
- 2° **for** : $C[2, 0, 2, 3, 0, 1]$
- 3° **for** : $C[2, 2, 4, 7, 7, 8]$
- $\Rightarrow$ posizione di 5 è 8; quelle di 3 sono 7, 6, 5
- 1^a iterazione ultimo **for**
 $B[C[A[8]]] = B[C[3]] = B[7] = A[8] = 3_3$
 $B[-, -, -, -, -, -, 3_3, -]$
 $C[2, 2, 4, 6, 7, 8]$
- 2^a iterazione ultimo **for**
 $B[C[A[7]]] = B[C[0]] = B[2] = A[7] = 0_2$
 $B[-, 0_2, -, -, -, -, 3_3, -]$
 $C[1, 2, 4, 6, 7, 8]$

... Esempio: $A = [2_1, 5_1, 3_1, 0_1, 2_2, 3_2, 0_2, 3_3]$

- 3^a iterazione ultimo **for**

$$B[C[A[6]]] = B[C[3]] = B[6] = A[6] = 3_2$$

$$B[-, 0_2, -, -, -, 3_2, 3_3, -]$$

$$C[1, 2, 4, 5, 7, 8]$$

- 4^a iterazione ultimo **for**

$$B[C[A[5]]] = B[C[2]] = B[4] = A[5] = 2_2$$

$$B[-, 0_2, -, 2_2, -, 3_2, 3_3, -]$$

$$C[1, 2, 3, 5, 7, 8]$$

- 5^a iterazione ultimo **for**

$$B[C[A[4]]] = B[C[0]] = B[1] = A[4] = 0_1$$

$$B[0_1, 0_2, -, 2_2, -, 3_2, 3_3, -]$$

$$C[0, 2, 3, 5, 7, 8]$$

- ...

- $B[0_1, 0_2, 2_1, 2_2, 3_1, 3_2, 3_3, 5_1]$

Counting sort è **stabile**

- Chiavi con lo stesso valore appaiono nello stesso ordine in uscita di quello che avevano in input
- Vero per come funziona l'ultimo ciclo

Correttezza

Tramite invarianti di ciclo!

Analisi

- $\Theta(n + k) \Rightarrow \Theta(n)$ se $k = O(n)$
- Ordinamento in **tempo lineare!!**
- Quanto grande può essere k ?
 - ▶ OK per valori a 32-bit? $k = 2^{32} = 4G$ memoria
 - ▶ 16-bit? Forse no
 - ▶ 4-bit? Probabilmente sì
- Counting sort usato in radix sort

So ordinare su una cifra ...

Come ordino su n cifre?

- Posso ordinare dalla cifra più significativa, ma...
dovrei mettere da parte più dati intermedi
- **Esempio:** ordinare i fogli di un libro di 1000 pagine mescolate:
10 mucchietti per le centinaia,
poi ognuna ha 10 mucchietti per le decine ecc.

Radix sort

Idea (controintuitiva): ordino le cifre **meno significative** per prime

RADIX-SORT(A, d)

for $i \leftarrow 1$ **to** d

 Usa ordinamento **stabile** per ordinare array A sulla cifra i

Radix sort

Idea (controintuitiva): ordino le cifre **meno significative** per prime

$\text{RADIX-SORT}(A, d)$

for $i \leftarrow 1$ **to** d

 Usa ordinamento **stabile** per ordinare array A sulla cifra i

326

453

608

835

751

435

704

690

Radix sort

Idea (controintuitiva): Ordino le cifre **meno significative** per prime

RADIX-SORT(A, d)

for $i \leftarrow 1$ **to** d

Usa ordinamento stabile per ordinare array A sulla cifra i

	S
326	690
453	751
608	453
835	704
751	835
704	326
690	608

→

Radix sort

Idea (controintuitiva): Ordino le cifre **meno significative** per prime

RADIX-SORT(A, d)

for $i \leftarrow 1$ **to** d

Usa ordinamento stabile per ordinare array A sulla cifra i

	S	S
326	690	704
453	751	608
608	453	326
835	704	835
751	835	435
435	435	751
704	326	453
690	608	690

Radix sort

Idea (controintuitiva): Ordino le cifre **meno significative** per prime

RADIX-SORT(A, d)

for $i \leftarrow 1$ **to** d

Usa ordinamento stabile per ordinare array A sulla cifra i

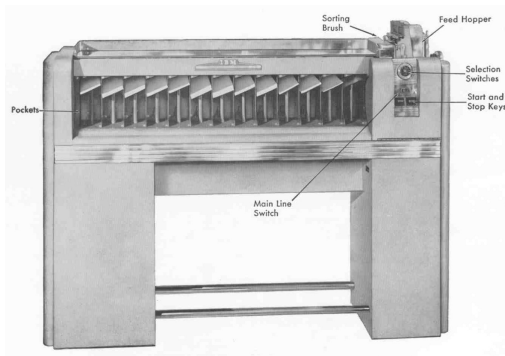
		S		S		S
326		690		704		326
453		751		608		435
608		453		326		453
835		704		835		608
751	→	835	→	435	→	690
435		435		751		704
704		326		453		751
690		608		690		835

Correttezza

- **Induzione** su i (colonna)
- Suppongo cifre $1, 2, \dots, i - 1$ ordinate
 - ▶ Se 2 cifre in posizione i sono **diverse**
⇒ ordinamento su posizione i è corretto
(posizioni $1, \dots, i - 1$ irrilevanti)
 - ▶ Se 2 cifre in posizione i sono **uguali** (es. 23 e 25)
⇒ numeri sono già nell'ordine giusto (ipotesi induttiva)
Ordinamento **stabile** sulla cifra i li lascia nell'ordine giusto
- ⇒ ordinamento **stabile** sulla cifra i lascia le cifre $1, \dots, i$ ordinate

Uso di radix sort

- Usato da IBM negli anni 1930 per ordinare schede perforate (per censimento)
- Macchine ordinatrici di schede lavoravano con una colonna alla volta
- L'uomo "a mano" realizzava radix-sort!



Analisi di radix-sort

Se utilizzo counting sort come ordinamento intermedio:

- $\Theta(n + k)$ per passo (cifre nell'intervallo $0, \dots, k$)
- d passi
- $\Rightarrow \Theta(d(n + k))$
- Se $k = O(n)$
 $\Rightarrow \Theta(d n)$

Suddivisione chiavi

- Parole di b bit divise in "pezzi" di r bit
- $d = \lceil b/r \rceil$ cifre di r bit ciascuna
- **Counting sort** con $k = 2^r - 1$

Esempio

- Parole di 32 bit, cifre di 8 bit
 - ▶ $b = 32$
 - ▶ $r = 8$
 - ▶ $d = \lceil b/r \rceil = \lceil 32/8 \rceil = 4$
 - ▶ $k = 2^r - 1 = 2^8 - 1 = 255$

Analisi

- n parole
- Tempo = $\Theta(\frac{b}{r}(n + 2^r))$

Scegliere r

Bilanciare b/r e $n + 2^r$

- Scegliendo $r \approx \lg n$ si ha

$$\Theta\left(\frac{b}{r}(n + 2^r)\right) = \Theta\left(\frac{b}{\lg n}(n + n)\right) = \Theta\left(\frac{b}{\lg n}n\right)$$
- Meglio di $\Theta(n)$
- Se fosse $r < \lg n \Rightarrow \frac{b}{r} > \frac{b}{\lg n} \Rightarrow n + 2^r$ non migliora e resta a $\Theta(n)$
- Se fosse $r > \lg n \Rightarrow n + 2^r$ diventa più grande di n
 Esempio se $r = 2 \lg n \Rightarrow 2^r = 2^{2 \lg n} = (2^{\lg n})^2 = n^2$

Esempio

- Per ordinare 2^{16} numeri a 32 bit
 usare $r = \lg 2^{16} = 16$ bit
 $\lceil b/r \rceil = 2$ passi

Radix sort vs merge sort e quicksort

- 1 milione ($n = 2^{20}$) di interi a 32-bit
- **Radix** sort: $r \approx \lg n$
- $\lceil \frac{32}{20} \rceil = 2$ passi (su n dati)
 - ▶ Nota: in radix sort un “passo” è $2n$:
 - ★ uno per counting sort
 - ★ uno per muovere i dati da B ad A (counting sort non ordina sul posto)
- **Merge sort/quicksort**: $\lg n = 20$ passi (su n dati)
- $\Rightarrow 4n$ vs $20n$

Limiti inferiori per ordinamento

Algoritmo	Tempo di esecuzione caso peggiore	Tempo di esecuzione atteso/ caso medio
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$ (atteso)
Counting sort	$\Theta(k + n)$	$\Theta(k + n)$
Radix sort	$\Theta(d(k + n))$	$\Theta(d(k + n))$

Hashing

Insiemi dinamici

- Operazioni su **dizionari**:
INSERT, SEARCH, DELETE
- **Tabelle hash**: implementazione efficiente di dizionario
 - ▶ tempo atteso di ricerca: $O(1)$
 - ▶ tempo di ricerca, caso peggiore: $\Theta(n)$
- Generalizzazione di **indirizzamento diretto**:
 - ▶ Elemento con chiave k nella posizione k

Tabelle ad indirizzamento diretto

- Insieme dinamico
- Universo delle chiavi: $U = \{0, 1, \dots, m - 1\}$
- Se m non è troppo grande $\Rightarrow$
 - ▶ tabella ad indirizzamento diretto: $T[0 \dots m - 1]$
 - ▶ con chiavi **duplicate**: lista collegata

DIRECT-ADDRESS-SEARCH(T, k)

return $T[k]$

DIRECT-ADDRESS-INSERT(T, x)

$T[x.key] \leftarrow x$

DIRECT-ADDRESS-DELETE(T, x)

$T[x.key] \leftarrow \text{NIL}$

Tempo?

Tabelle ad indirizzamento diretto

Tabelle hash

- Spesso ... tabella con $|U|$ elementi è troppo grande
- Spesso ... $|K| \ll |U|$ (K chiavi memorizzate)
- Usiamo **funzione hash**
Memorizzo k in $T[h(k)]$ (non in $T[k]$)
- $h : U \rightarrow \{0, 1, \dots, m-1\}$
- Funzione hash è **non iniettiva**

Si hanno **collisioni**:

$$h(k) = h(k') \text{ con } k \neq k'$$

- ▶ Può capitare se $|K| \leq m$
- ▶ Capita sicuramente se $|K| > m$
- Due metodi per gestire collisioni:
 - ▶ Concatenamento
 - ▶ Indirizzamento aperto

Funzioni hash

$$h : U \rightarrow \underbrace{\{0, 1, \dots, m - 1\}}_{\text{numero cella}}$$

- $h(k)$ dovrebbe realizzare un

Hash uniforme semplice:

ogni elemento ha la stessa probabilità di andare in ogni cella indipendentemente da dove vanno gli altri elementi

- In pratica non è possibile
 - ▶ non conosco la distribuzione di probabilità da cui sono prese le chiavi
 - ▶ le chiavi potrebbero non essere prese indipendentemente una dall'altra
- Si usano informazioni sul dominio delle chiavi per creare una buona funzione hash

Metodo delle divisioni

- $h(k) = k \bmod m$
- $a \bmod m = a - \left\lfloor \frac{a}{m} \right\rfloor m$
- **Esempio:**
 $m = 20$ e $k = 91 \Rightarrow$
 $h(k) = 11$
- **Vantaggio:** Veloce
- **Svantaggio:** Evitare alcuni valori di m :
 - ▶ Evitare potenze intere di 2
 Perché ?
 Se $m = 2^p \Rightarrow h(k)$ sono le p cifre meno significative di k
 - ▶ Critico per rappresentazioni di stringhe!
- Buona scelta per m :
*numero **primo** non troppo vicino ad una potenza di 2*

Chiavi: **numeri naturali**

- $h(k)$ t.c. $k \in \mathbb{N}$
- Se $k \notin \mathbb{N}$? Convertire k
- **Stringa** di caratteri?
- Notazione posizionale con opportuna base
 - ▶ Esempio: *CLRS*
 - ▶ ASCII: C = 67, L = 76, R = 82, S = 83
 - ▶ 7 bit (ASCII di base)
 - ▶ $\Rightarrow$
$$\text{CLRS} \rightarrow (67 \cdot 128^3) + (76 \cdot 128^2) + (82 \cdot 128^1) + (83 \cdot 128^0)$$
$$= 141,764,947$$

Risoluzione collisioni con concatenamento

Tutti gli elementi con lo stesso hash sono in una **lista collegata**

Inserimento

CHAINED-HASH-INSERT(T, x)

inserisce x in testa alla lista $T[h(x.key)]$

- Tempo nel caso **peggiore?** $O(1)$
- Se l'elemento non è già nella lista
- Altrimenti: un'altra ricerca

Risoluzione collisioni con concatenamento

Ricerca

CHAINED-HASH-SEARCH(T, k)

return ricerca un elemento con chiave k nella lista $T[h(k)]$

- **Tempo** ? proporzionale alla **lunghezza** della lista in $h(k)$

Cancellazione

CHAINED-HASH-DELETE(T, x)

cancella x dalla lista $T[h(x.key)]$

- Tempo nel caso **peggiore** è $O(1)$
con liste doppiamente collegate
(non considera il tempo per trovare l'elemento da cancellare)
- Con liste semplicemente collegate, costa quanto la ricerca

- **Non** è una struttura dati **molto dinamica**

- ▶ si può *raddoppiare* / *dimezzare* la tabella via via
- ▶ quanto costa?

Esercizio su hash

- Si consideri una tabella hash con $m = 13$ e con funzione hash calcolata con il metodo delle divisioni
- Si consideri la seguente successione di chiavi:
 $\langle 5, 21, 10, 34, 47, 3, 18, 35, 11, 23 \rangle$
 - 1 Indicare la configurazione di una tabella hash a indirizzamento diretto (con concatenamento) dopo l'inserimento di tutte le chiavi precedenti.
 - 2 Si cancellino nell'ordine le seguenti chiavi: $\langle 18, 21, 34, 11 \rangle$ e si disegni la configurazione ottenuta dopo aver eseguito tutte le cancellazioni.

Analisi di hash con concatenamento

- Tempo richiesto per:
 - ▶ Ricerca con successo
 - ▶ Ricerca senza successo
- **Fattore di caricamento** $\alpha = \frac{n}{m}$
 - ▶ $n = \#$ di elementi memorizzati nella tabella
 - ▶ $m = \#$ di slot nella tabella ($\#$ di liste collegate)
 - ▶ α : numero medio di elementi in ogni lista collegata
 - ▶ si può avere $\alpha < 1$, $\alpha = 1$, o $\alpha > 1$
- Caso **peggiore**: tutte le n chiavi hanno hash nello stesso slot
 Una singola lista di lunghezza n
 $\Rightarrow$ tempo: $\Theta(1) + \Theta(n) = \Theta(n)$
- Caso **medio** dipende dalla funzione hash

Caso medio di hash con concatenamento

- Ipotesi: **hash uniforme semplice**: per ogni elemento ognuno degli m è ugualmente probabile come hash
- n_j = lunghezza lista $T[j]$ ($j = 0, 1, \dots, m - 1$)
- $\Rightarrow n = n_0 + n_1 + \dots + n_{m-1}$
- Valor atteso di n_j : $E[n_j] = \alpha = \frac{n}{m}$
- Calcolo la funzione hash in $O(1)$
 $\Rightarrow$ tempo per cercare l'elemento con chiave k dipende dalla lunghezza $n_{h(k)}$ della lista $T[h(k)]$

Due casi:

- ▶ Ricerca **senza successo**
- ▶ Ricerca **con successo**

Ricerca senza successo

Teorema

Una ricerca senza successo richiede un tempo atteso di $\Theta(1 + \alpha)$

Dimostrazione

- Hash uniforme semplice $\Rightarrow$ l'hash di ogni chiave non già nella tabella finisce in ognuno degli m slot con la stessa probabilità
- Per ricercare **senza successo** ogni chiave k bisogna cercare **fino alla fine della lista** $T[h(k)]$
- $T[h(k)]$ ha **lunghezza attesa** $E[n_{h(k)}] = \alpha$
- $\Rightarrow$ con calcolo della funzione hash:
 $\Theta(1 + \alpha)$

Ricerca con successo:

- Diverso da ricerca senza successo
- Ogni lista **non ha** la stessa probabilità di essere oggetto delle ricerche
- Questa è proporzionale al numero di elementi nella lista

Teorema

Una ricerca con successo richiede un tempo atteso $\Theta(1 + \alpha)$

Dimostrazione

- Cerco x (può essere **qualunque** degli n elementi memorizzati)
- Gli n_x elementi **prima** di x nella lista $h(x.key)$ sono stati **inseriti dopo** x in $h(x.key)$
 $\Rightarrow$ Cercando x esamino $n_x + 1$ elementi
- Calcolo numero atteso di elementi inseriti in $h(x.key)$ dopo x
- x_i : i -mo elemento inserito nella tabella ($i = 1, 2, \dots, n$)
- $k_i = x_i.key$
- Variabile casuale indicatrice
 $X_{ij} = I\{h(k_i) = h(k_j)\}$
- Hash uniforme semplice $\Rightarrow Pr\{h(k_i) = h(k_j)\} = \frac{1}{m} \Rightarrow$
 $E[X_{ij}] = \frac{1}{m}$

- Numero atteso di elementi esaminati

(ricerca **con successo** $\Rightarrow$ valore atteso sugli n elementi inseriti)

$$\begin{aligned}
 E\left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij}\right)\right] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}]\right) = \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m}\right) \\
 &= 1 + \frac{1}{nm} \sum_{i=1}^n (n - i) = 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i\right) \\
 &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2}\right) = 1 + \frac{n-1}{2m} \\
 &= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}
 \end{aligned}$$

- Tempo totale (con calcolo funzione hash)

$$\Theta\left(2 + \frac{\alpha}{2} - \frac{\alpha}{2n}\right) = \Theta(1 + \alpha)$$

- Se $n = O(m) \Rightarrow \alpha = O(1) \Rightarrow \Theta(1)$

Indirizzamento aperto

Idea

- Memorizza tutte le chiavi nella tabella hash
- Ogni slot contiene una chiave o NIL

Cercare chiave k

1) Ispeziona slot $h(k)$:

- ▶ Se slot $h(k)$ contiene la chiave $k \Rightarrow$ ricerca ha successo
- ▶ Se contiene NIL $\Rightarrow$ ricerca non ha successo
- ▶ Se $h(k)$ contiene $k' \neq k$
 - ★ Calcola l'indice di qualche altro slot da ispezionare e vai a 1)
- La successione di slot ispezionati deve essere una **permutazione** dei numeri di slot $< 0, 1, \dots, m - 1 >$
 - ▶ esaminiamo tutti gli slot se devo
 - ▶ non esaminiamo nessuno slot più di una volta

Funzione hash, con indirizzamento aperto

$$h : U \times \underbrace{\{0, 1, \dots, m-1\}}_{\text{numero ispezione}} \rightarrow \underbrace{\{0, 1, \dots, m-1\}}_{\text{numero cella}}$$

- $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ (**sequenza di ispezione**)
deve essere una **permutazione** di $\langle 0, 1, \dots, m-1 \rangle$
- Esempio: $h(k, i) = (h'(k) + i) \bmod m$ ($h'(k) = k \bmod m$)

HASH-SEARCH(T, k)

```

1   $i \leftarrow 0$ 
2  repeat
3       $j \leftarrow h(k, i)$ 
4      if  $T[j] = k$ 
5          return  $j$ 
6       $i \leftarrow i + 1$ 
7  until  $T[j] = \text{NIL}$  or  $i = m$ 
8  return NIL
  
```

Inserimento

- Cerco il valore e lo inserisco nel primo NIL trovato

HASH-INSERT(T, k)

```

1   $i \leftarrow 0$ 
2  repeat
3       $j \leftarrow h(k, i)$ 
4      if  $T[j] = \text{NIL}$ 
5           $T[j] \leftarrow k$ 
6          return  $j$ 
7      else  $i \leftarrow i + 1$ 
8  until  $i = m$ 
9  error ""hash table overflow""

```

- Ritorna il numero dello slot che prende la chiave k o indica un errore se non c'è uno slot vuoto dove mettere k

Cancellazione

- Posso semplicemente mettere NIL nello slot contenente la chiave k da cancellare?
- No! Potrei interrompere la catena di ricerca per una chiave k' inserita dopo k
- **Soluzione:** Uso un valore speciale DEL (deleted) invece di NIL quando cancello
- **Ricerca** tratta DEL come se lo slot contenesse una chiave che non corrisponde a quella cercata
- **Inserimento** tratta DEL come se lo slot fosse vuoto per riusarlo (piccola modifica del codice)
- **Svantaggio:** ora il tempo di ricerca non dipende più da α

Esercizio su hash

- Si consideri una tabella hash con $m = 13$
- Ispezione più semplice (non l'unica!): ispezione lineare
 - ▶ $h(k, i) = (h'(k) + c i) \bmod m$
 - ▶ $h'(k) = k \bmod m$
- Si consideri la seguente successione di chiavi:
< 5, 21, 10, 34, 47, 3, 18, 35, 11, 23 >
 - 1 Indicare la configurazione di una tabella hash a indirizzamento aperto (con esplorazione lineare) dopo l'inserimento di tutte le chiavi precedenti.
 - 2 Si cancellino nell'ordine le seguenti chiavi: < 18, 21, 34, 11 > e si disegni la configurazione ottenuta dopo aver eseguito tutte le cancellazioni.

Calcolare la sequenza di esplorazioni

Hash uniforme

- Ciascuna chiave è ugualmente probabile che generi ciascuna delle $m!$ permutazioni di $\langle 0, 1, \dots, m-1 \rangle$ come sequenza di esplorazione
- Generalizza l'**hash uniforme semplice**
- Difficile implementare un vero hash uniforme $\Rightarrow$
- Si approssima garantendo che almeno la sequenza di esplorazione sia una **permutazione** di $\langle 0, 1, \dots, m-1 \rangle$
- **Nessuna di queste tecniche può produrre tutte le $m!$ sequenze di esplorazione**
- Si usano funzioni hash ausiliarie
$$h'() : U \rightarrow \{0, 1, \dots, m-1\}$$

Esplorazione lineare

- $h(k, i) = (h'(k) + c \cdot i) \bmod m$
Data la chiave k e il numero di esplorazione i ($0 \leq i < m$)
- L'*esplorazione iniziale determina* l'intera sequenza
- Si hanno **solo m possibili sequenze** (non $m!$)
- Si ha **clustering** (addensamento) **primario**:
lunghe sequenze (run) di slot occupati
- *Run lunghi tendono a diventare più lunghi*:
uno slot vuoto preceduto da i slot pieni verrà riempito
successivamente con probabilità $(i + 1)/m$

Esplorazione quadratica

- $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$
 $c_1, c_2 \neq 0$ costanti
- Bisogna scegliere c_1, c_2 , e m per assicurare che si abbia una permutazione completa di $\langle 0, 1, \dots, m-1 \rangle$
- Ad esempio: $c_1 = \frac{1}{2}, c_2 = \frac{1}{2}$ e $m = 2^p$
- Può esserci **clustering secondario**:
se $h'(k) = h'(k')$ ($k \neq k'$) allora k e k' avranno la stessa sequenza di esplorazione

Doppio hash

- Usa due funzioni hash ausiliarie:
 - ▶ h_1 per prima esplorazione
 - ▶ h_2 per esplorazioni restanti
- $h(k, i) = (h_1(k) + h_2(k) i) \bmod m$.
- (*) Scegliere $h_2(k)$: primo relativo con m (no fattori comuni) per garantire che la sequenza di esplorazione sia una permutazione completa di $\langle 0, 1, \dots, m-1 \rangle$
 - ▶ Scegliere m come una potenza di 2 e h_2 che produca sempre un numero dispari > 1
 - ▶ Scegliere m primo e:
 - $h_1(k) = k \bmod m$
 - $h_2(k) = 1 + (k \bmod m')$. Con m' più piccolo di m (es. $m' = m-1$)
- Si hanno $\Theta(m^2)$ ($\ll m!$) diverse sequenze di esplorazione (m per $h_1(k)$ e m per $h_2(k)$)
- Comunque migliore di m (per hash lineare)

Analisi di indirizzamento aperto

• Ipotesi

- ▶ Tabella non piena $0 \leq n < m \Rightarrow 0 \leq \alpha < 1$
- ▶ Supponiamo **hash uniforme**
- ▶ No cancellazioni

Ricerca senza successo

Numero atteso di esplorazioni è al più $\frac{1}{1-\alpha}$

Inserimento

Numero atteso di esplorazioni è al più $\frac{1}{1-\alpha}$

Ricerca con successo

Numero atteso di esplorazioni è al più $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$

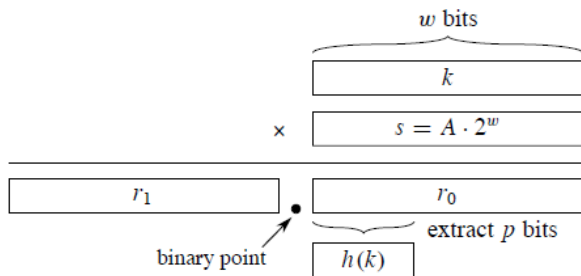
Funzioni hash (cont.) $h : U \rightarrow \underbrace{\{0, 1, \dots, m-1\}}_{\text{numero cella}}$

Metodo delle moltiplicazioni

- Scegliere una costante A : $0 < A < 1$
 $h(k) = \lfloor m(kA \bmod 1) \rfloor$
- $kA \bmod 1 = kA - \lfloor kA \rfloor =$ parte frazionaria di kA
- **Svantaggio**: Più lento del metodo delle divisioni
- **Vantaggio**: Il valore di m non è critico

Implementazione facile

- $m = 2^p$ (p intero)
- w : numero di bit della parola della macchina
- Supponiamo che k stia in una sola parola (k richiede w bit)
- Scelgo s intero $0 < s < 2^w$ (s richiede w bit)
- $A = \frac{s}{2^w}$ ($0 < A < 1$)



$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

- $k \cdot s = r_1 2^w + r_0$ (prodotto di due parole di w bit)
- r_1 : parte intera di kA : ($r_1 = \lfloor kA \rfloor$)
- r_0 : parte frazionaria di kA : ($r_0 = kA \bmod 1 = kA - \lfloor kA \rfloor$)
- $\lfloor m(kA \bmod 1) \rfloor \Rightarrow$
 - ▶ shift a sinistra r_0 di $p = \lg m$ bit
 - ▶ prendo i p bit a sinistra del punto binario
- Non serve shiftare prendo i p bit più significativi di r_0

Come scegliere A

- Il metodo delle moltiplicazioni funziona con ogni valore $0 < A < 1$
- Lavora meglio con alcuni valori rispetto ad altri sulla base della chiave su cui fare l'hash
- Knuth suggerisce di usare $A \approx \frac{\sqrt{5}-1}{2}$
- Quindi **dato** w **si sceglie** s **intero t.c.** $\frac{s}{2^w} \approx (\sqrt{5} - 1)/2$

(*) Hash randomizzato

- Conoscendo la funzione hash usata un “avversario sleale” potrebbe inviare tutte le chiavi nello stesso slot
- Potrei randomizzare l'hash come con quicksort randomizzato?
- Scelgo casualmente la funzione *all'inizio* dell'utilizzo di una tabella (non per ogni calcolo della funzione)
- Come per quicksort randomizzato: **nessun input** può provocare sistematicamente il comportamento nel caso peggiore dell'algoritmo

(*) Hash universale

- p è un numero primo t.c. $\forall k$ si ha $0 \leq k \leq p - 1$
- scelgo a e b naturali t.c. $0 \leq a \leq p - 1$ e $0 \leq b \leq p$
- funzione hash: $h_{a,b} = ((a \cdot k + b) \bmod p) \bmod m$
- esempio: $h_{3,4}(8) = 5$
- Scegliendo a e b casualmente ho una famiglia con $p(p - 1)$ funzioni hash diverse per un certo m

(*) Hash perfetto

- Scenario: insieme di chiavi **statico**
- **Hash perfetto**: complessità è $O(1)$ nel caso peggiore!
 - ▶ **Idea**: provo con varie funzioni hash fino a trovarne una che non genera collisioni
 - ▶ **Problemi**:
 - ★ m dovrebbe essere troppo grande
 - ★ potrei dover provare tante (troppe) volte
- **Soluzione**: hash a **due livelli** con hash universale in ogni livello
 - ▶ **Primo livello** è come hash con concatenamento
 - ▶ **Secondo livello**: tabelle hash con dimensioni diverse (n_i^2 dove n_i sono i valori in $T[i]$)
 - ▶ Si basa su un **teorema**: *Se memorizzo n chiavi in una tabella hash con $m = n^2$ utilizzando una funzione h da una classe universale di funzioni allora la probabilità che si verifichi una collisione è $< \frac{1}{2}$*
 - ▶ **Nota**: probabilità $< \frac{1}{2}$ (**non certezza!**) quindi devo provare varie funzioni per ogni $T[i]$ per non avere collisioni nelle tabelle di secondo livello

Tabelle dinamiche

- Tabella hash di dimensione m
- n varia
- Se $\alpha = \frac{n}{m} = 1$
rialloco con $m' > m$ e copio gli oggetti
- Se $\alpha \leq 0.5$
alloco tabella più piccola e copio gli oggetti
- Costo principale?
 - ▶ copia degli oggetti dopo riallocazione
- **Obiettivi**
 - 1 Tempo ammortizzato per operazione $O(1)$
 - 2 Poco spazio inutilizzato

(*) TABLE-INSERT

- Inizialmente $T.num = T.size = 0$

TABLE-INSERT(T, x)

if $T.size = 0$

 alloca tabella $T.table$ con 1 elemento

$T.size \leftarrow 1$

if $T.num = T.size$

 alloca NT con $2 \cdot T.size$ elementi

 inserisci gli elementi di $T.table$ in NT

 free $T.table$

$T.table \leftarrow NT$

$T.size \leftarrow 2 \cdot T.size$

inserisci x in $T.table$

$T.num \leftarrow T.num + 1$

(*) Analisi ammortizzata

- Studia **tempo** per eseguire una **sequenza** di operazioni (**diverse**) su una struttura dati
- Mostra che anche se alcune operazioni sono costose in **media** il costo per operazione è piccolo

Non si fa una media su una distribuzione di input

- Non c'è probabilità
- Considero **costo medio nel caso peggiore** per una sequenza di n operazioni
- Posso ottenere il costo medio per operazione (**costo ammortizzato**)

(*) Tempo di esecuzione

- Conto solo inserimento
Altri costi costanti per ogni chiamata
- Costa ($O(1)$) per ogni inserimento
- Supponiamo di fare solo inserimenti
- c_i = costo operazione i -ma
 - ▶ tabella **non piena**: $c_i = 1$
 - ▶ tabella **piena**: $c_i = i$ (copio i elementi)
 - ▶ $\Rightarrow c_i = O(i)$ in generale
- **Analisi superficiale** (caso peggiore)
 - ▶ n operazioni, $c_i = O(i)$
 - ▶ $\Rightarrow$ tempo per n operazioni $O(n^2)$ (serie aritmetica)
- Analisi non corretta
 - ▶ espando solo se $i - 1$ è una potenza esatta di 2 (tabella piena)!

(*) Analisi **corretta**

- Non si espande sempre:

$$c_i = \begin{cases} i & \text{se } i - 1 \text{ e' una potenza esatta di } 2 \\ 1 & \text{altrimenti} \end{cases}$$

$$\text{Costo totale} = \sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j$$

$$= n + 2^{\lfloor \lg n \rfloor + 1 - 1} = n + 2 \cdot 2^{\lfloor \lg n \rfloor} - 1 = n + 2n - 1$$

$$< n + 2n = 3n$$

- Per **analisi aggregata** il costo ammortizzato per operazione è 3

- Serie geometrica: $\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$

Alberi binari di ricerca

Alberi di ricerca

- Supportano operazioni dinamiche
- Usati spesso come dizionario
- Operazioni di base in $O(h)$ ($h =$ altezza albero)
- Vari tipi di alberi di ricerca:
 - ▶ alberi binari di ricerca (ABR)
 - ▶ alberi Rosso-Neri (ARN)
 - ▶ Alberi AVL
 - ▶ alberi B

Alberi Binari di Ricerca

Alberi binari di ricerca

- Rappresentato con struttura dati collegata (ogni nodo è un oggetto)
 - ▶ $T.root$ punta a radice albero T
 - ▶ Campi:
 - ★ *chiave* (ed eventualmente dati satellite)
 - ★ *left*: punta a figlio sx
 - ★ *right*: punta a figlio dx
 - ★ *p*: punta a padre $T.root.p = NIL$

Proprietà albero binario di ricerca

- Se y è nel sottoalbero sx di x , allora $y.key \leq x.key$
- Se y è nel sottoalbero dx di x , allora $y.key \geq x.key$

Inserimento

TREE-INSERT(T, z)

$y \leftarrow \text{NIL}$ // Tiene traccia del padre

$x \leftarrow T.\text{root}$

while $x \neq \text{NIL}$

$y \leftarrow x$

if $z.\text{key} < x.\text{key}$

$x \leftarrow x.\text{left}$

else $x \leftarrow x.\text{right}$

$z.p \leftarrow y$

if $y = \text{NIL}$

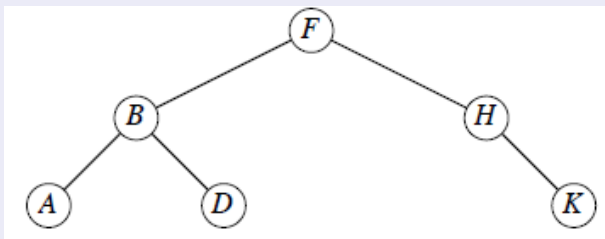
$T.\text{root} \leftarrow z$ // L'albero T era vuoto

elseif $z.\text{key} < y.\text{key}$

$y.\text{left} \leftarrow z$

else $y.\text{right} \leftarrow z$

Inserimento



Inserire **C**

Tempo?

- $O(h)$

Quanto vale h ?

- Nel caso migliore $h = \lg n$
- Nel caso peggiore $h = n$
- Da cosa dipende?
- Dall'**ordine** di inserimento

Interrogare ABR

Chiamata: $\text{TREE-SEARCH}(\text{root}[T], k)$

$\text{TREE-SEARCH}(x, k)$

if $x = \text{NIL}$ or $k = x.\text{key}$

return x

if $k < x.\text{key}$

return $\text{TREE-SEARCH}(x.\text{left}, k)$

else return $\text{TREE-SEARCH}(x.\text{right}, k)$

Tempo ?

$O(h)$

Versione iterativa

- iterativa più veloce
- ricorsiva più intuitiva

```
ITERATIVE-TREE-SEARCH( $x, k$ )  
  while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$   
    if  $k < x.\text{key}$   
       $x \leftarrow x.\text{left}$   
    else  $x \leftarrow x.\text{right}$   
  return  $x$ 
```

Correttezza?

Invariante di ciclo $\Rightarrow$ vediamola insieme...

Minimo e Massimo

per proprietà ABR

- **Chiave minima** è nel nodo più a sinistra
- **Chiave massima** è nel nodo più a destra

TREE-MINIMUM(x)

```
while  $x.left \neq \text{NIL}$ 
     $x \leftarrow x.left$ 
return  $x$ 
```

TREE-MAXIMUM(x)

```
while  $x.right \neq \text{NIL}$ 
     $x \leftarrow x.right$ 
return  $x$ 
```

Tempo ?

- $O(h)$

Attraversamento simmetrico di un albero (*inorder*)

INORDER-TREE-WALK(x)

if $x \neq \text{NIL}$

INORDER-TREE-WALK($x.\text{left}$)

stampa $x.\text{key}$

INORDER-TREE-WALK($x.\text{right}$)

- Anche:
 - ▶ **attraversamento anticipato di un albero** (*preorder*)
 - ▶ **attraversamento posticipato di un albero** (*postorder*)
- Con *inorder*
elementi sono stampati in ordine monotonamente crescente

Correttezza

Per induzione dalla proprietà dell'albero binario di ricerca

Tempo

- *Intuitivamente* $\Theta(n)$ (albero con n nodi) perché visita ogni nodo una volta sola
- $T(n) = \Omega(n)$ (visita **tutti** gli n nodi)
- E' possibile mostrare che $T(n) = O(n)$

$$T(n) = O(n)$$

- c : costo per sottoalbero vuoto ($T(0)$)
- Per $n > 0$ un nodo ha:
 k nodi in sottoalbero sx e
 $n - k - 1$ nodi in sottoalbero dx
- $\Rightarrow T(n) \leq T(k) + T(n - k - 1) + d$
 $(d = \text{costo per eseguire la funzione})$
- Ipotesi di sostituzione: $T(n) \leq (c + d)n + c$
 Per $n = 0$: $(c + d) \cdot 0 + c = c = T(0)$
 Per $n > 0$:

$$T(n) \leq T(k) + T(n - k - 1) + d$$

$$= ((c + d)k + c) + ((c + d)(n - k - 1) + c) + d$$

$$= (c + d)n + c - (c + d) + c + d = (c + d)n + c$$

Successore e predecessore

- Successore di x :
 - ▶ y t.c. $y.key$ è la più piccola chiave $> x.key$
- Se x ha la chiave più grande
 - ▶ $\Rightarrow$ successore di x è NIL

Due casi

- 1 x ha un sottoalbero **destro non vuoto**
 $\Rightarrow$ Successore di x è il **minimo** nel **sottoalbero destro** di x
- 2 x ha un sottoalbero **destro vuoto**
 $\Rightarrow$ Successore (y) di x è
 antenato più prossimo di x il cui figlio sinistro è anche antenato di x

TREE-PREDECESSOR è simmetrico di TREE-SUCCESSOR

TREE-SUCCESSOR(x)

```

if  $x.right \neq \text{NIL}$ 
    return TREE-MINIMUM( $x.right$ )
 $y \leftarrow x.p$ 
while  $y \neq \text{NIL}$  and  $x = y.right$ 
     $x \leftarrow y$ 
     $y \leftarrow y.p$ 
return  $y$ 

```

Tempo ?

- $O(h)$
- TREE-SUCCESSOR e TREE-PREDECESSOR visitano nodi su un cammino verso il basso o verso l'alto dell'albero

Cancellazione di z

Tre casi a seconda del numero di figli

Caso 1: z non ha figli

- ▶ Cancella z aggiustando il puntatore del padre

Caso 2: z ha un figlio

- ▶ Cancella z
- ▶ Il padre ($z.p$) punta al figlio di z invece che a z

Caso 3: z ha due figli

- ▶ Chi è $y = succ(z)$?
 - ★ **Minimo** del sottoalbero dx
- ▶ Quanti figli ha $y = succ(z)$?
 - ★ 0 o 1 (altrimenti non sarebbe il successore!)
- ▶ $\Rightarrow$ Cerco $y = min(z.right)$
- ▶ **Cancello** y (Caso 1 o 2)
- ▶ Sostituisco la chiave di z e dati satellite con quelli di y

Alberi binari di ricerca **costruiti in modo casuale**

- Date n chiavi inserirle in ordine casuale in un ABR vuoto
- Ciascuna delle $n!$ permutazioni è ugualmente probabile
- Non consideriamo cancellazioni
- **Altezza attesa:** $O(\lg n)$
- Non lo dimostriamo!

Esercizio 12.4.3

- Ogni ABR con n chiavi **non** è ugualmente probabile
- Provare con $n = 3$
 - ▶ 6 possibili permutazioni dei dati
 - ▶ 5 ABR diversi
 - ▶ 4 alberi appaiono solo una volta
 - ▶ 1 albero appare due volte

Ordinamento tramite ABR

Esercizio 12.3-3

- Usare TREE-INSERT e INORDER-TREE-WALK per ordinare un insieme di numeri
- Posso ordinare n numeri:
 - ▶ costruendo prima un ABR inserendo questi numeri
 - ▶ stampando i numeri con un attraversamento simmetrico
- Quali sono i tempi di esecuzione nel caso peggiore e nel caso migliore?
- E' un ordinamento stabile?
- Come posso fare a randomizzare questo algoritmo?

Alberi binari di ricerca con chiavi uguali (*)

Le chiavi uguali sono un problema per l'implementazione di ABR

Problema 12-1

- Quali sono le prestazioni asintotiche inserendo n chiavi identiche in un ABR inizialmente vuoto?
- E' possibile modificare TREE-INSERT verificando se $z.key = x.key$ prima di $z.key < x.key$ e se $z.key = y.key$ prima di $z.key < y.key$. Quando valgono le uguaglianze:
 - ① mantenere una lista concatenata con chiavi uguali
 - ② mantenere un flag booleano $b[x]$ nel nodo x e porre x a $x.left$ o $x.right$ a seconda del valore del flag (che si alterna ad ogni visita di x con valore uguale)
 - ③ Scegliere casualmente se assegnare $x.left$ o $x.right$ a x
- Per ogni strategia trovare le prestazioni asintotiche inserendo n chiavi identiche in un ABR inizialmente vuoto
- Cosa comportano queste strategie per la ricerca dei valori?

Inserimento

TREE-INSERT(T, z)

$y \leftarrow \text{NIL}$ // Tiene traccia del padre

$x \leftarrow T.\text{root}$

while $x \neq \text{NIL}$

$y \leftarrow x$

if $z.\text{key} < x.\text{key}$

$x \leftarrow x.\text{left}$

else $x \leftarrow x.\text{right}$

$z.p \leftarrow y$

if $y = \text{NIL}$

$T.\text{root} \leftarrow z$ // L'albero T era vuoto

elseif $z.\text{key} < y.\text{key}$

$y.\text{left} \leftarrow z$

else $y.\text{right} \leftarrow z$

AVL

Alberi AVL

- Il nome deriva dagli inventori: G.M. Adelson-Velsky e E.M. Landis
- Un AVL è un ABR "quasi" bilanciato: per ogni nodo le altezze dei due sottoalberi differiscono al massimo di 1
- **Altezza** del nodo $h(x)$:
numero di archi nel cammino più lungo fino ad una foglia
- **Foglie vuote** (NIL)
 - ▶ Una sentinella, $T.NIL$ per tutte le foglie di T
 - ▶ $T.NIL.h = 0$
 - ▶ Non interessa key in $T.NIL$
- Un **AVL** eredita gli attributi di un ABR, aggiunge ad ogni nodo x l'attributo $x.h$ (altezza del sottoalbero con radice in x)
- Proprietà AVL: per ogni nodo x dell'albero $|BF(x)| \leq 1$
 $BF(x) = x.left.h - x.right.h$
 $BF(x)$ *Balancy Factor* del nodo

Altezza **massima** di AVL con n nodi interni è $2 \lg(n + 1)$

- $n(h)$: numero minimo di nodi in un AVL di altezza h

Teorema: per ogni $h > 1$ si ha $n(h) \geq 2^{h/2} - 1$

Dimostrazione: induzione su h

- **Casi base:** $h = 1$: $n(1) = 1 > 2^{1/2} - 1 = \sqrt{2} - 1$
 $h = 2$: $n(2) = 2 > 2^{1/1} - 1 = 1$

- **Passo induttivo (per $h > 2$):**

da definizione di AVL (un sottoalbero ha altezza $h - 1$ l'altro almeno $h - 2$):

$n(h) \geq 1 + n(h - 1) + n(h - 2)$ **ipotesi induttiva:**

$$n(h) \geq 1 + 2^{\frac{h-1}{2}} - 1 + 2^{\frac{h-2}{2}} - 1 = (2^{-1/2} + 2^{-1})2^{h/2} - 1 > 2^{h/2} - 1$$

$$\Rightarrow n(h) \geq 2^{h/2} - 1$$

- $\Rightarrow \lg(n + 1) \geq h/2 \Rightarrow h \leq 2\lg(n + 1)$

Operazioni su AVL

Operazioni che non modificano ABR

- MIN
- MAX
- SUCCESSOR
- PREDECESSOR
- SEARCH

Si eseguono in tempo $O(h)$

Impiegano $O(\lg n)$ con alberi AVL

Inserimento e cancellazione non facili

Inserimento

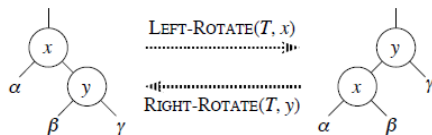
- Inserisco normalmente
- Potrei violare la regola AVL per qualche nodo (inserisco la nuova chiave nel sottoalbero più alto)
- Eventuali nodi con sbilanciamento possono essere solo nel cammino dalla radice al nodo inserito
- Devo aggiustare l'albero riorganizzando le chiavi

Cancellazione

- Considerazioni simili all'inserimento

Rotazioni

- **Ristrutturazione** dell'albero usata per mantenere AVL **bilanciati**
- Cambio solo i puntatori e non cambio proprietà ABR
- Si ha **rotazione sinistra** e **destra**



- Mantiene ordinamento delle chiavi
- Dopo la rotazione l'albero è **più bilanciato**

Tempo ?

- $O(1)$ LEFT-ROTATE e RIGHT-ROTATE: modifica un numero costante di puntatori anche aggiornando attributi $x.h$ e $y.h$

LEFT-ROTATE(T, x)

```

 $y \leftarrow x.right$  // Imposta  $y$ 
 $x.right \leftarrow y.left$  // Sposta sottoalbero sx di  $y$  in dx di  $x$ 
 $x.h \leftarrow \max(x.left.h, x.right.h) + 1$ 
if  $y.left \neq T.NIL$ 
     $y.left.p \leftarrow x$ 
 $y.p \leftarrow x.p$  // Collega il padre di  $x$  a  $y$ 
if  $x.p = T.NIL$ 
     $T.root \leftarrow y$ 
elseif  $x = x.p.left$ 
     $x.p.left \leftarrow y$ 
else  $x.p.right \leftarrow y$ 
 $y.left \leftarrow x$  // Pone  $x$  a sx di  $y$ 
 $y.h \leftarrow \max(y.left.h, y.right.h) + 1$ 
 $x.p \leftarrow y$ 

```

- Prima dell'esecuzione $x.right \neq T.NIL$

RVL-INSERT(T, z)

```
1   $y \leftarrow T.NIL$ 
2   $x \leftarrow T.root$ 
3  while  $x \neq T.NIL$ 
4       $y \leftarrow x$ 
5      if  $z.key < x.key$ 
6           $x \leftarrow x.left$ 
7      else  $x \leftarrow x.right$ 
8   $z.p \leftarrow y$ 
9  if  $y = T.NIL$ 
10      $T.root \leftarrow z$ 
11  elseif  $z.key < y.key$ 
12      $y.left \leftarrow z$ 
13  else  $y.right \leftarrow z$ 
14   $z.left \leftarrow T.NIL$ 
15   $z.right \leftarrow T.NIL$ 
16   $z.h \leftarrow 1$ 
17  AVL-INSERT-FIXUP( $T, z$ )
```

Inserimento

Al termine di RVL-INSERT

- Solo gli antenati del nuovo nodo possono essere sbilanciati
 - ▶ unici nodi con sottoalberi modificati
- $\forall x$ antenato di z :
 $-2 \leq BF(x) \leq 2$
(prima dell'inserimento era AVL quindi $-1 \leq BF(x) \leq 1$)
- E' un problema se $BF(x) = 2$ o $BF(x) = -2$
 - ▶ Se $BF(x) = 2$ il sottoalbero sinistro è più alto di due livelli del destro
 - ▶ Viceversa se $BF(x) = -2$

Quattro casi

$$BF(x) = 2$$

- Il nuovo nodo è stato inserito in $x.left.left$
 - ▶ sarà: $BF(x.left) = 1$
- Il nuovo nodo è stato inserito in $x.left.right$
 - ▶ sarà: $BF(x.left) = -1$

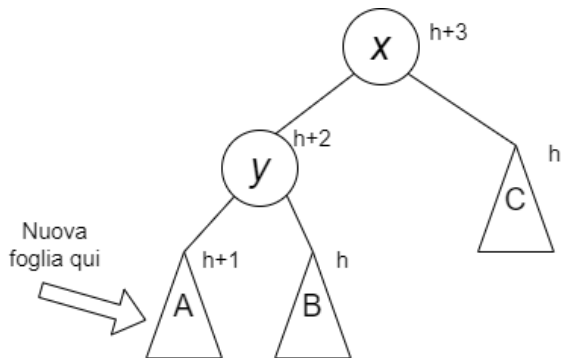
$$BF(x) = -2$$

- Il nuovo nodo è stato inserito in $x.right.left$
 - ▶ sarà: $BF(x.right) = 1$
- Il nuovo nodo è stato inserito in $x.right.right$
 - ▶ sarà: $BF(x.right) = -1$

- $BF(X) = -2$ simmetrico rispetto a $BF(x) = 2$
- vediamo i primi due

$$BF(x) = 2$$

nuovo nodo in $x.left.left$ ($BF(x.left) = 1$)

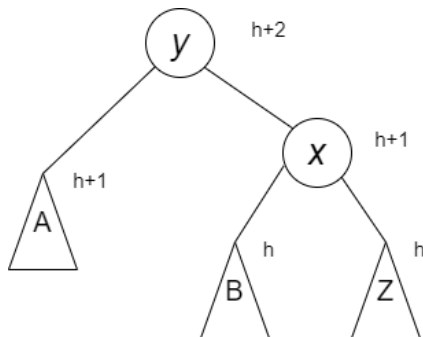


- Rotazione a destra su nodo x

$$BF(x) = 2$$

nuovo nodo in $x.left.left$ ($BF(x.left) = 1$)

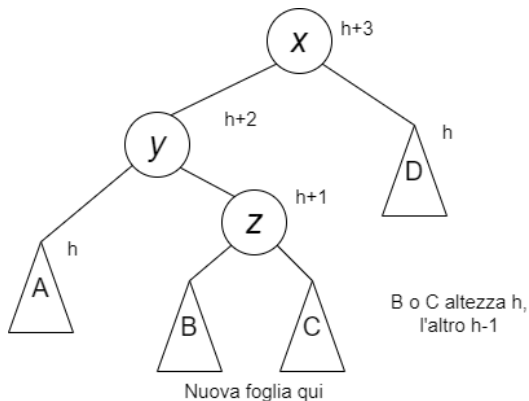
- Rotazione a destra su nodo x



- Sistemato

$$BF(x) = 2$$

nuovo nodo in $x.left.right$ ($BF(x.left) = -1$)

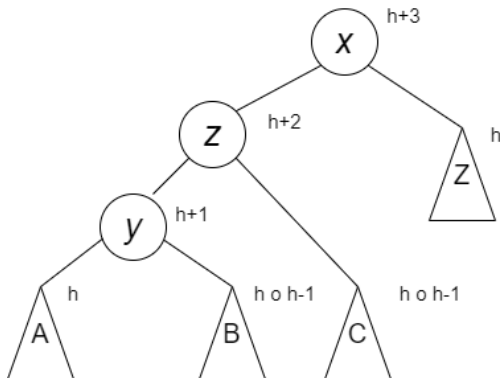


- Una rotazione non basta
 - ▶ Rotazione a sinistra su nodo y
 - ▶ Rotazione a destra su nodo x

$$BF(x) = 2$$

nuovo nodo in $x.left.right$ ($BF(x.left) = -1$)

- Rotazione a sinistra su nodo y

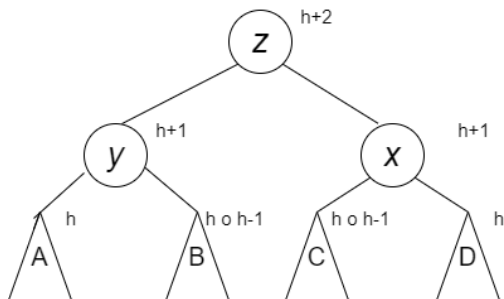


- Rotazione a destra su nodo x

$$BF(x) = 2$$

nuovo nodo in $x.left.right$ ($BF(x.left) = -1$)

- Rotazione a destra su nodo x



- Sistemato!

AVL-INSERT-FIXUP(T, x)

```

1   $x \leftarrow x.p$ 
2  while  $x \neq T.NIL$ 
3       $x.h \leftarrow \max(x.left.h, x.right.h) + 1$ 
4      if  $(x.left.h - x.right.h) = 2$  //  $BH(x)$ 
5          if  $(x.left.left.h - x.left.right.h) = -1$  //  $BH(x.left)$ 
6              LEFT-ROTATE( $T, x.left$ )
7              RIGHT-ROTATE( $T, x$ )
8               $x \leftarrow x.p$  // è ancora radice del sottoalbero
9              // 9-12 simmetrico con 4-7
10         elseif  $(x.left.h - x.right.h) = -2$  //  $BH(x)$ 
11             if  $(x.right.left.h - x.right.right.h) = 1$  //  $BH(x.right)$ 
12                 RIGHT-ROTATE( $T, x.right$ )
13                 LEFT-ROTATE( $T, x$ )
14              $x \leftarrow x.p$  // è ancora radice del sottoalbero
15   $x \leftarrow x.p$ 

```

Correttezza?

- Invariante di ciclo
 - ▶ all'inizio di ogni iterazione in AVL c'è al massimo una violazione del bilanciamento di AVL
 - ▶ ... non vediamo i dettagli

Tempo?

- AVL-INSERT: $O(\lg n)$ fino a AVL-INSERT-FIXUP
- In AVL-INSERT-FIXUP:
 - ▶ Ogni iterazione: $O(1)$ e sale di 1 livello
 - ▶ Si hanno $O(\lg n)$ livelli $\Rightarrow O(\lg n)$
 - ▶ Al massimo 2 rotazioni in totale
 - ▶ Dobbiamo comunque arrivare alla radice per aggiustare $x.h$
- Inserimento in albero AVL: $O(\lg n)$

Cancellazione

Non la trattiamo

Strutture dati aumentate

Progettare algoritmi

- Raramente progetto una struttura dati da zero
- Spesso prendo una struttura dati nota e **aggiungo** informazioni
- La struttura dati può permettere **nuove operazioni**
- Gestire correttamente la nuova informazione **senza perdita di efficienza**

Statistiche d'ordine dinamiche

Albero AVL, più :

- $\text{OS-SELECT}(x, i)$: ritorna il puntatore al nodo che contiene l'*i*-ma **chiave più piccola** del sotto-albero con radice in x
- $\text{OS-RANK}(T, x)$: ritorna la posizione (**rango**) di x nell'ordine lineare determinato da un attraversamento **inorder** di T (indicato con $\text{rank}(x, T)$)

Implementazione semplice?

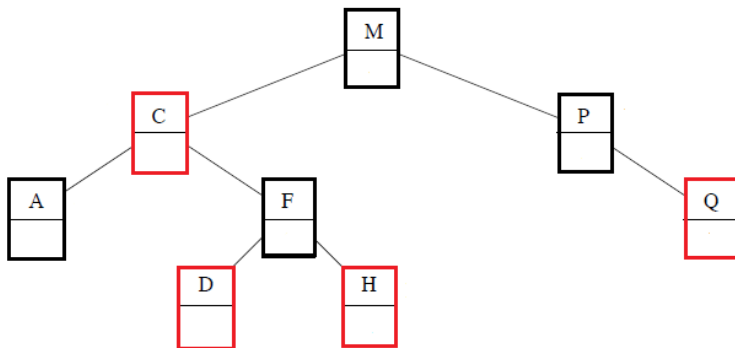
- Eseguo un attraversamento in-order e conto quanti nodi trovo
 - ▶ Per $\text{OS-SELECT}(x, i)$ mi fermo dopo i nodi
 - ▶ Per $\text{OS-RANK}(T, x)$ mi fermo quando trovo x
- Costo? $O(n)$

Aumento AVL (o RN)

- Aggiungo $x.size = \# \text{ nodi}$ nel sotto-albero con radice in x
 - ▶ Incluso x
 - ▶ Foglie (sentinelle) escluse
- Def $T.NIL.size = 0 \Rightarrow$

$$x.size = x.left.size + x.right.size + 1$$
- Struttura dati **aumentata**:
 $\langle x.key, x.p, x.left, x.right, x.color, x.size \rangle$

Esempio



Chiavi duplicate

- Rango non definito per chiavi uguali
- Lo definisco rispetto ad attraversamento inorder
(Se cambio $C = D \Rightarrow \text{rank}(C, T) = 2$ e $\text{rank}(D, T) = 3$)

OS-SELECT

- Ritorna il puntatore al nodo con l' i -ma chiave più piccola del sotto-albero con radice in x
- Chiamata iniziale: OS-SELECT($T.root, i$)

OS-SELECT(x, i)

```

1   $r \leftarrow x.left.size + 1$ 
2  if  $i = r$ 
3      return  $x$ 
4  elseif  $i < r$ 
5      return OS-SELECT( $x.left, i$ )
6  else return OS-SELECT( $x.right, i - r$ )

```

Esempio: OS-SELECT($T.root, 5$)

Correttezza

- Ricorsivo in coda
 - $\Rightarrow$ posso vederlo come iterativo
 - $\Rightarrow$ invariante di ciclo
- Prima di ogni iterazione l' i -mo valore si trova nel sottoalbero con radice in x
- $r =$ rango di x nel sottoalbero con radice x
 - ▶ Se $i = r \Rightarrow$ OK ritorna x
 - ▶ Se $i < r \Rightarrow i$ -mo elemento più piccolo è nel sottoalbero s_x
 - ▶ Se $i > r \Rightarrow i$ -mo elemento più piccolo è nel sottoalbero d_x
 tolgo gli r elementi nel sottoalbero di x
 che **precedono** quelli nel sottoalbero destro di x

Costo

- Ogni chiamata ricorsiva scende di un livello
- L'albero RN ha $O(\lg n)$ livelli $\Rightarrow O(\lg n)$ chiamate
- **Tempo** $O(\lg n)$

OS-RANK(T, x)

- Ritorna il rango di x nell'albero T

OS-RANK(T, x)

```

1   $r \leftarrow x.\text{left.size} + 1$ 
2   $y \leftarrow x$ 
3  while  $y \neq T.\text{root}$ 
4      if  $y = y.p.\text{right}$ 
5           $r \leftarrow r + y.p.\text{left.size} + 1$ 
6       $y \leftarrow y.p$ 
7  return  $r$ 
```

Correttezza

Invariante di ciclo

- All'inizio di ogni iterazione del ciclo
 $r = \text{rank}(x.\text{key}, y)$

Inizializzazione

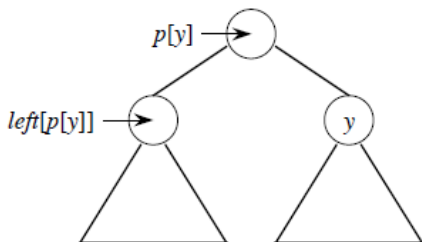
- $r = \text{rank}(x.\text{key}, x)$ e $y = x$

Terminazione

- Ciclo termina quando $y = T.\text{root}$
- Sottoalbero con radice in y è l'intero albero
- $r = \text{rank}(x.\text{key}, T)$
(proprietà utile)

Conservazione

- se $r = \text{rank}(x.\text{key}, y)$ all'inizio dell'iterazione,
 allora $r = \text{rank}(x.\text{key}, y.p)$ alla fine dell'iterazione:
 - ▶ se y è un figlio s_x , sottoalbero del fratello ha nodi che seguono x
 $\Rightarrow r$ non cambia
 - ▶ se y è un figlio d_x , tutti i nodi nel sottoalbero del fratello
 precedono tutti i nodi nel sottoalbero di y
 $\Rightarrow r \leftarrow r + y.p.\text{left.size} + 1$



Analisi

- y sale di un livello ad ogni iterazione
- $O(\lg n)$

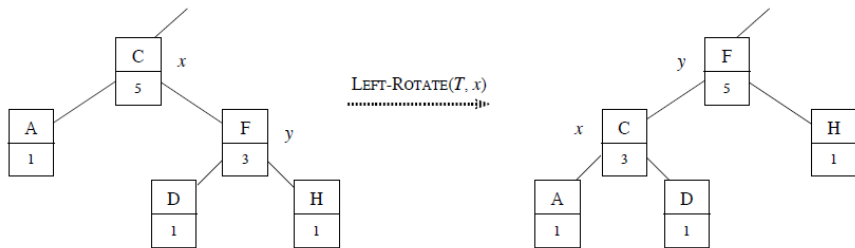
Gestione attributo *size*

- Gestire $x.size$ in **inserimento** e **cancellazione**
- Gestione **efficiente** per non doverli ricalcolare tutti ($\Omega(n)$)
 - ▶ Come farei in $\Omega(n)$?
 - ▶ Devo risalire sull'albero e aggiornare tutti i size.
Come?
 - ▶ Attraversamento differito
 - ▶ Scrivere algoritmo
- Ci riusciamo senza aumentare costo inserimento e cancellazione ($O(\lg n)$)?

Inserimento

Visita per inserimento

- Il nuovo nodo sarà un **discendente** di ciascun nodo visitato (solo di questi nodi)
- Incrementa *size* di ogni nodo **visitato**
- In **fixup**:
 - ▶ Sale nell'albero
 - ▶ Cambia $x.h$ $O(\lg n)$ volte
 - ▶ Esegue ≤ 2 rotazioni
- Rotazioni cambiano *size* sottoalberi
 - ▶ Nuove dimensioni sulla base delle vecchie e delle dimensioni dei figli



Rotazioni

- Aggiungo in fondo a LEFT-ROTATE :
 - ▶ $y.\text{size} \leftarrow x.\text{size}$ (sottoalbero totale **non cambia**)
 - ▶ $x.\text{size} \leftarrow x.\text{left.size} + x.\text{right.size} + 1$
 - ▶ Altri nodi **non cambiano**
- Analogamente per rotazione **destra**
- Aggiorno $x.\text{size}$ in $O(1)$ per rotazione
- Aggiornamento campi durante fixup: $O(1)$
- $\Rightarrow O(\lg n)$ per **inserire**

Cancellazione

- In 2 passi (no dettagli):
 - ① Divide un nodo y
 - ② Fixup
 - ★ Percorso $y \rightarrow \text{root}$, decrementa *size* in ogni nodo trovato ($O(\lg n)$)
 - ★ Solo cambiamenti di $x.h$ e (≤ 3) rotazioni
 - ★ $O(1)$ per aggiornamento campi *size*
- $O(\lg n)$ per **cancellare**

Programmazione dinamica

Programmazione dinamica

- **Divide-et-impera**

- ▶ scompone il problema in sottoproblemi **indipendenti**
- ▶ risolve i sottoproblemi
- ▶ li ricombina

- **Programmazione dinamica**

- ▶ sottoproblemi **non indipendenti**
 - ★ condividono dei sottoproblemi
- ▶ quando risolvo un **sottoproblema comune**, salvo la soluzione in una **tabella**, per riutilizzarla dopo
- ▶ “programmazione” si riferisce all'uso di una **tecnica tabulare**

- Usata spesso per problemi di **ottimizzazione**

- ▶ problemi di **massimizzazione, minimizzazione**
- ▶ un problema potrebbe avere più soluzioni ottime
 - ★ voglio trovare **una** soluzione con **il** valore ottimo

Bellman - Eye of the Hurricane: An Autobiography (1984)

I spent the Fall quarter (of 1950) at RAND. ... Where did the name, dynamic programming, come from?

The 1950s were not good years for mathematical research. We had a very interesting gentleman in Washington named Wilson. He was Secretary of Defense, and he actually had a pathological fear and hatred of the word **research**.

I'm not using the term lightly; I'm using it precisely. His face would suffuse, he would turn red, and he would get violent if people used the term research in his presence. You can imagine how he felt, then, about the term **mathematical**.

The RAND Corporation was employed by the Air Force, and the Air Force had Wilson as its boss, essentially. Hence, I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics inside the RAND Corporation.

What title, what name, could I choose?

...

Bellman - Eye of the Hurricane: An Autobiography (1984)

...

In the first place I was interested in planning, in decision making, in thinking. But planning, is not a good word for various reasons.

I decided therefore to use the word **programming**. I wanted to get across the idea that this was dynamic, this was multistage, this was time-varying I thought, lets kill two birds with one stone. Lets take a word that has an absolutely precise meaning, namely **dynamic**, in the classical physical sense.

It also has a very interesting property as an adjective, and that is it's impossible to use the word dynamic in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It's impossible.

Thus, I thought **dynamic programming** was a good name. It was something not even a Congressman could object to. So I used it as an umbrella for my activities.

Passi principali

Per sviluppare un algoritmo di programmazione dinamica

- 1 Caratterizzare la struttura di una soluzione ottima
- 2 Definire ricorsivamente il valore di una soluzione ottima del problema
- 3 Calcolare una soluzione ottima in modo bottom-up
 - ▶ da sottoproblemi semplici a quelli più difficili, fino al problema originario
- 4 Costruire una soluzione ottima del problema richiesto

Problema: taglio delle aste

- Prezzo di un'asta di acciaio dipende dalla sua lunghezza
- Un'asta lunga n può essere tagliata in pezzi più corti
- Trovare il taglio **ottimo** che **massimizza** il ricavo della vendita
 - potrei anche venderla intera

Tabella dei prezzi

lunghezza i	1	2	3	4	5	6	7	8	9	10
prezzo p_i	1	5	8	9	10	17	17	20	24	30

lunghezza l_i	1	2	3	4
prezzo p_i	1	5	8	9

Possibili tagli ($n = 4$)

- 8 possibili modi di taglio

[4] (9)

[1, 3] (9)

[1, 1, 2] (7)

[1, 1, 1, 1] (4)

[2, 2] **(10)**

[1, 2, 1] (7)

[3, 1] (9)

[2, 1, 1] (7)

- 2^{n-1} **modi** per tagliare un'asta lunga n

▶ $n - 1$ punti di taglio

▶ 2^{n-1} combinazioni di *taglio* - *non taglio*

▶ come un numero binario con $n - 1$ cifre

- Taglio ottimo: [2, 2]

- Ricavo massimo (r_n): $r_4 = 10$

Sottostruttura ottima

Esprimo r_n in funzione di r_i ($i < n$)

- $\forall$ taglio i ho: $r_n = r_i + r_{n-i}$
 - ▶ se non taglio: $r_n = p_n$ (prezzo asta intera)
 - ▶ $\Rightarrow$

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1)$$

- **Ottimo** è la somma dei ricavi **ottimi** delle 2 **semiaste**
- Ottimo **incorpora** i 2 ottimi delle soluzioni dei 2 sottoproblemi
- Dim (per assurdo):
 - ▶ Supponiamo che r_i (o r_{n-i}) **non** sia l'ottimo del sottoproblema
 - ▶ sostituisco a r_i una soluzione ottima $r'_i > r_i$ ho $r'_n > r_n$
 - ▶ $\Rightarrow r_n$ non sarebbe un ottimo
- Un problema ha una **sottostruttura ottima** se la sua soluzione incorpora le soluzioni ottime dei suoi sottoproblemi (che posso risolvere indipendentemente)

Altra formulazione

r_n dipende dall'ottimo di **un solo** sottoproblema:

- r_n = prezzo del primo pezzo + **taglio ottimo** della restante asta:

$$r_n = p_i + r_{n-i}$$

- vale anche se non taglio l'asta ($i = n$):

$$r_n = p_n + r_0 \text{ con } r_0 = 0$$

- Quindi: $r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$

Passi principali

Per sviluppare un algoritmo di programmazione dinamica

- 1 Caratterizzare la struttura di una soluzione ottima
- 2 **Definire ricorsivamente il valore di una soluzione ottima del problema**
- 3 Calcolare una soluzione ottima in modo bottom-up
 - ▶ dai sottoproblemi più semplici a quelli più difficili, fino al problema originario
- 4 Costruire una soluzione ottima del problema richiesto

Algoritmo ricorsivo

- Ottenuto da:

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

- dove

p : tabella prezzi

n : lunghezza asta

CUT-ROD(p, n)

if $n = 0$

return 0

$q \leftarrow -\infty$

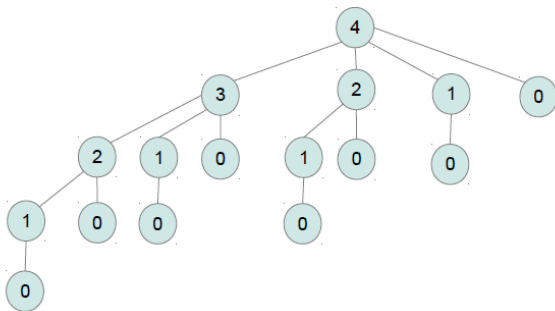
for $i \leftarrow 1$ **to** n

$q \leftarrow \max(q, p[i] + \text{CUT-ROD}(p, n - i))$

return q

Costo

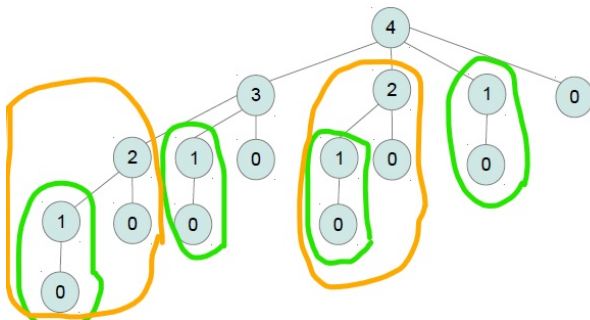
- $T(n) = \text{costante} + \sum_{j=0}^{n-1} T(j)$
- Si può mostrare che $T(n) = \Theta(2^n)$
- In pratica considero tutti i possibili modi di taglio dell'asta (2^{n-1})
- Lo stesso problema è risolto più e più volte!!



Albero di ricorsione (nel nodo indico n)

Costo

- $T(n) = \text{costante} + \sum_{j=0}^{n-1} T(j)$
- Si può mostrare che $T(n) = \Theta(2^n)$
- In pratica considero tutti i possibili modi di taglio dell'asta (2^{n-1})
- Lo stesso problema è risolto più e più volte!!



Albero di ricorsione (nel nodo indico n)

Algoritmo di programmazione dinamica

- Usando memoria aggiuntiva miglioro il tempo di esecuzione
 - ▶ diventa polinomiale!!
 - ▶ aumento la complessità spaziale riducendo quella temporale
- **Idea:**
 - ▶ memorizzo il risultato dei sottoproblemi appena calcolati
 - ▶ se trovo di nuovo un sottoproblema considero il risultato memorizzato
- **Quindi:**
 - ▶ risolvo ogni sottoproblema **una volta sola**
 - ▶ il costo diventa **polinomiale** se:
 - ★ il numero di problemi *distinti* da risolvere è polinomiale e
 - ★ la risoluzione dei singoli problemi richiede tempo polinomiale
- Due tecniche per implementare la programmazione dinamica:
 - ▶ top-down
 - ▶ bottom-up

Metodo **top-down**

- risolvo il **problema di dimensione** n , e ricorsivamente risolvo **sottoproblemi** più piccoli
- memorizzo in una tabella i risultati già calcolati
- **prima** di lanciare la **ricorsione** sul problema più piccolo, **controllo** nella tabella se non ho già calcolato la soluzione

Metodo **bottom-up**

- parto dai problemi più piccoli e li risolvo in ordine crescente di dimensione
- quando arrivo al problema di dimensione i , ho già risolto tutti i problemi di dimensioni $< i$

Stesso tempo di esecuzione asintotico

- **Top-down** a volte non esamina tutti i possibili sottoproblemi
- **Bottom-up** ha costi inferiori per le chiamate ricorsive (le costanti)

Memoization (annotazione)

MEMOIZED-CUT-ROD(p, n)

Sia $r[0..n]$ un nuovo array

for $i \leftarrow 0$ **to** n

$r[i] \leftarrow -\infty$

return MEMOIZED-CUT-ROD-AUX(p, n, r)

MEMOIZED-CUT-ROD-AUX(p, n, r)

if $r[n] \geq 0$

return $r[n]$

if $n = 0$

$q \leftarrow 0$

else $q \leftarrow -\infty$

for $i \leftarrow 1$ **to** n

$q \leftarrow \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$

$r[n] \leftarrow q$

return q

Algoritmo ricorsivo (visto prima)

- Differente rispetto a programmazione dinamica!

CUT-ROD(p, n)

if $n = 0$

return 0

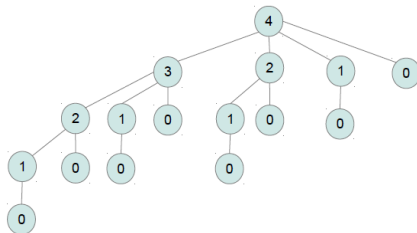
$q \leftarrow -\infty$

for $i \leftarrow 1$ **to** n

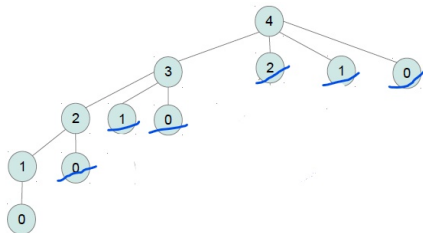
$q \leftarrow \max(q, p[i] + \text{CUT-ROD}(p, n - i))$

return q

- Algoritmo ricorsivo



- Memoization



- $2^n \gg n^2$ quando n cresce, ma $2^4 = 4^2 \dots$

Passi principali

Per sviluppare un algoritmo di programmazione dinamica

- ➊ Caratterizzare la struttura di una soluzione ottima
- ➋ Definire ricorsivamente il valore di una soluzione ottima del problema
- ➌ **Calcolare una soluzione ottima in modo bottom-up**
 - ▶ dai sottoproblemi più semplici a quelli più difficili, fino al problema originario
- ➍ Costruire una soluzione ottima del problema richiesto

Calcolo bottom-up di soluzione ottima

Metodo bottom-up

- parto dai problemi più piccoli e li risolvo in ordine crescente di dimensione
- quando arrivo al problema di dimensione i , ho già risolto tutti i problemi di dimensioni $< i$

BOTTOM-UP-CUT-ROD(p, n)

Sia $r[0..n]$ un nuovo array

$r[0] \leftarrow 0$

for $j \leftarrow 1$ **to** n

$q \leftarrow -\infty$

for $i \leftarrow 1$ **to** j

$q \leftarrow \max(q, p[i] + r[j - i])$

$r[j] \leftarrow q$

return $r[n]$

Complessità

- MEMOIZED-CUT-ROD: $T(n) = \Theta(n^2)$
 - ▶ ogni sottoproblema è risolto una volta sola
nel ciclo n iterazioni per risolvere un problema di dimensione n
 - ▶ in tutto $n + (n - 1) + (n - 2) + \dots + 1 = \frac{n(n+1)}{2}$ iterazioni
- BOTTOM-UP-CUT-ROD: $T(n) = \Theta(n^2)$
 - ▶ 2 cicli annidati
 - ▶ ancora $n + (n - 1) + (n - 2) + \dots + 1 = \frac{n(n+1)}{2}$ iterazioni
- Top-down e bottom-up hanno lo stesso tempo di esecuzione asintotico

Passi principali

Per sviluppare un algoritmo di programmazione dinamica

- 1 Caratterizzare la struttura di una soluzione ottima
- 2 Definire ricorsivamente il valore di una soluzione ottima del problema
- 3 Calcolare una soluzione ottima in modo bottom-up
 - ▶ dai sottoproblemi più semplici a quelli più difficili, fino al problema originario
- 4 **Costruire una soluzione ottima del problema richiesto**

Ricostruire una soluzione

- Algoritmi CUT-ROD restituiscono il **massimo ricavo**, non il **modo** di taglio per ottenerlo
- uso array $s[]$ in cui tengo traccia del modo di effettuare il taglio:
 - ▶ $s[j]$: lunghezza del primo pezzo nel taglio ottimale di un'asta lunga j

EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

$r[0 \dots n]$ ed $s[0 \dots n]$ due nuovi array

$r[0] \leftarrow 0$

for $j \leftarrow 1$ **to** n

$q \leftarrow -\infty$

for $i \leftarrow 1$ **to** j

if $q < p[i] + r[j - i]$

$q \leftarrow p[i] + r[j - i]$

$s[j] \leftarrow i$

$r[j] \leftarrow q$

return (r, s)

PRINT-CUT-ROD-SOLUTION(p, n)

$(r, s) \leftarrow \text{EXTENDED-BOTTOM-UP-CUT-ROD}(p, n)$

while $n > 0$

 print $s[n]$

$n \leftarrow n - s[n]$

Esempio di esecuzione

Eseguire

- PRINT-CUT-ROD-SOLUTION($p, 7$)

lunghezza i	1	2	3	4	5	6	7	8	9	10
prezzo p_i	1	5	8	9	10	17	17	20	24	30

i	0	1	2	3	4	5	6	7	8	9	10
p_i	0	1	5	8	9	10	17	17	20	24	30
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

Longest Common Subsequence

(sottosequenza comune più lunga)

Problema

Date 2 sequenze, $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$
trovare una sottosequenza comune ad X e Y di lunghezza massima

Sottosequenza

- Sottosequenza di una sequenza:
sequenza a cui tolgo 0 o più elementi
- - ▶ $X = \langle x_1, x_2, \dots, x_m \rangle$
 - ▶ $Z = \langle z_1, z_2, \dots, z_k \rangle$ è una **sottosequenza** di X se
 $\exists$ una sequenza di indici $I = \langle i_1, i_2, \dots, i_k \rangle$ t.c.
 $i_j > i_h$ se $j > h \quad \forall j, h = 1, 2, \dots, k$ e
 $x_{i_j} = z_j \quad \forall j = 1, 2, \dots, k$
- $\langle b, c, f \rangle$ sottosequenza di $\langle a, b, c, d, e, f, g \rangle$ con $I = \langle 2, 3, 6 \rangle$

Esempi sottosequenze

- calibro
 - ▶ Sottosequenze: a, li, ab, cio
 - ▶ **Non** sono sottosequenze: ba, oa, ...

Esempi LCS

- panca $\leftrightarrow$ camp
→ *pa*, ma anche *aa*
- angelo $\leftrightarrow$ demone
→ *ne*
- CDADDCC $\leftrightarrow$ BABADBC
→ *ADC*

Algoritmo forza-bruta

- Per ogni sotto-sequenza di X
 - ▶ controllare se è una sotto-sequenza di Y

Tempo

- 2^m sottosequenze di X da controllare
- $\Theta(n)$ per controllare ogni sottosequenza:
 - ▶ scorrere Y per la prima lettera, poi per la seconda e così via
- $\Theta(n \cdot 2^m)$

Programmazione dinamica: passi principali

- 1 Caratterizzare la struttura di una soluzione ottima
- 2 Definire ricorsivamente il valore di una soluzione ottima del problema
- 3 Calcolare una soluzione ottima in modo bottom-up
 - ▶ dai sottoproblemi più semplici a quelli più difficili, fino al problema originario
- 4 Costruire una soluzione ottima del problema richiesto

Passo 1: Sotto-struttura ottima (sottosequenza comune)

Notazione

- X_i = prefisso di X con i elementi: $\langle x_1, \dots, x_i \rangle$
- Y_j = prefisso di Y con j elementi: $\langle y_1, \dots, y_j \rangle$
- **Esempio:**
se $X = \langle A, B, C, D \rangle$ si ha
 $X_1 = \langle A \rangle$ e $X_3 = \langle A, B, C \rangle$

Teorema

Date le sequenze $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$
sia $Z = \langle z_1, \dots, z_k \rangle$ una qualsiasi LCS di X e Y

- 1 Se $x_m = y_n$, allora $z_k = x_m = y_n$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1}
- 2 Se $x_m \neq y_n$, allora $z_k \neq x_m$ implica che Z è una LCS di X_{m-1} e Y
- 3 Se $x_m \neq y_n$, allora $z_k \neq y_n$ implica che Z è una LCS di X e Y_{n-1}

Dimostrazione 1/3

① Se $x_m = y_n$ allora $z_k = x_m = y_n$ e Z_{k-1} è LCS di X_{m-1} e Y_{n-1}

① Mostro che se $x_m = y_n$ allora $z_k = x_m = y_n$

- ▶ Suppongo $z_k \neq x_m$
- ▶ **Costruisco** $Z' = \langle z_1, \dots, z_k, x_m \rangle$
 - ★ E' una CS di X e Y e ha lunghezza $k + 1 \Rightarrow$
 - ★ Z' è una CS più lunga di $Z \Rightarrow$
 - ★ **contraddice** ipotesi che Z sia una LCS

② Mostro che Z_{k-1} è una **LCS** di X_{m-1} e Y_{n-1}

- ▶ E' chiaramente una **CS**
- ▶ Mostro che è **LCS**
 - ★ **supponiamo** che esista una CS W di X_{m-1} e Y_{n-1} che sia più lunga di Z_{k-1} (lunghezza di $W \geq k$)
 - ★ Costruisco W' appendendo x_m a W
 - ★ W' è una CS di X e Y , ha lunghezza $\geq k + 1 \Rightarrow$ contraddice il fatto che Z sia una LCS di X e Y

Dimostrazione 2-3

① visto

- ② Se $x_m \neq y_n$, allora $z_k \neq x_m$ implica che Z è una LCS di X_{m-1} e Y
- ③ Se $x_m \neq y_n$, allora $z_k \neq y_n$ implica che Z è una LCS di X e Y_{n-1}

- ② Se $z_k \neq x_m$, allora Z è una **CS** di X_{m-1} e Y . Mostro che è **LCS**
Suppongo $\exists$ una sottosequenza W di X_{m-1} e Y con lunghezza $> k$
 Allora W è una sottosequenza comune di X e $Y \Rightarrow$
 contraddice il fatto che Z sia una LCS

- ③ Simmetrico di 2

Quindi

una LCS di due sequenze contiene al suo interno (come prefisso) una LCS di prefissi delle sequenze:

sottostruttura ottima!

Passo 2: Formulazione ricorsiva

- Esamino uno o due sottoproblemi per trovare LCS di X e Y
 - ▶ Se $x_m = y_n$ devo **risolvere un sottoproblema**
 - ★ trovare una LCS di X_{m-1} e Y_{n-1}
 - accodo $x_m = y_n$ a questa LCS e ottengo una LCS di X e Y
 - ▶ Se $x_m \neq y_n$ devo **risolvere due sottoproblemi**:
 - ★ trovare una LCS di X_{m-1} e Y
 - ★ trovare una LCS di X e Y_{n-1}

La più lunga di queste due LCS è una LCS di X e Y

- Soluzione ricorsiva di LCS con una ricorrenza:

- ▶ $c[i, j]$ = lunghezza di LCS di X_i e Y_j
- ▶ voglio $c[m, n]$

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \text{ (X o Y e' vuota)} \\ c[i - 1, j - 1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ \max(c[i, j - 1], c[i - 1, j]) & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

- Se $x_i \neq y_j$ devo risolvere **solo due** sottoproblemi

- ▶ LCS di X_i e Y_{j-1}
- ▶ LCS di X_{i-1} e Y_j

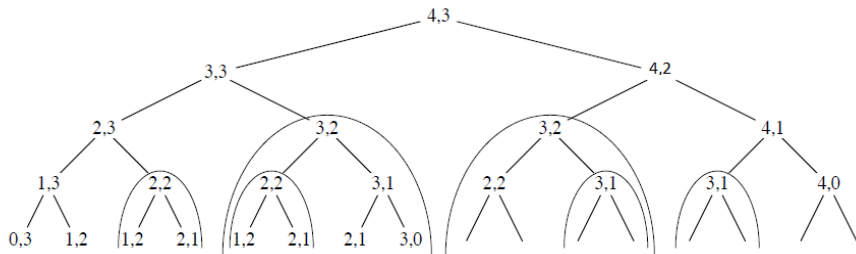
- Per taglio delle aste li consideravo tutti

- Per il teorema non abbiamo escluso soluzioni!

Posso scrivere un algoritmo ricorsivo basato su questa formulazione

Esempio: $LCS(casa, cut)$

- ad ogni livello si ri-chiama a sx con $i - 1$, a dx con $j - 1$
- dovrei arrivare a 0,0 in ogni cammino



Costo

- Altezza: $n + m$ (qui $4 + 3 = 7$)
- Tempo è potenzialmente **esponenziale**
- Molti sottoproblemi ripetuti
- Invece di ricalcolare memorizzo in una tabella (memoization)

Passo 3: Calcolare la lunghezza della soluzione ottima

LCS-LENGTH(X, Y)

$m \leftarrow X.length$

$n \leftarrow Y.length$

Siano $b[1..m, 1..n]$ e $c[0..m, 0..n]$ due nuove tabelle

for $i \leftarrow 1$ **to** m

$c[i, 0] \leftarrow 0$

for $j \leftarrow 0$ **to** n

$c[0, j] \leftarrow 0$

for $i \leftarrow 1$ **to** m

for $j \leftarrow 1$ **to** n

if $x_i = y_j$

$c[i, j] \leftarrow c[i - 1, j - 1] + 1$

$b[i, j] \leftarrow " \nwarrow "$

elseif $c[i - 1, j] \geq c[i, j - 1]$

$c[i, j] \leftarrow c[i - 1, j]$

$b[i, j] \leftarrow " \uparrow "$

else $c[i, j] \leftarrow c[i, j - 1]$

$b[i, j] \leftarrow " \leftarrow "$

return c e b

Passo 3: Calcolare la lunghezza della soluzione ottima

Tabelle ausiliari

- $c[i, j]$ contiene la lunghezza ottima di LCS per X_i e Y_j
- $b[i, j]$ indica la strada seguita (il sottoproblema) per risolvere LCS di X_i e Y_j
- Se $b[i, j] = \nwarrow$, ho esteso LCS di un carattere ($x_i = y_j$)
- LCS contiene x_i (e y_j) in cui $b[i, j] = \nwarrow$

Esempio: ABCBTA $\leftrightarrow$ ABRACA

		A	B	R	A	C	A
		0	0	0	0	0	0
A	0	1	1	1	1	1	1
B	0	1	2	2	2	2	2
C	0						
B	0						
T	0						
A	0						

Esempio: ABCBTA $\leftrightarrow$ ABRACA

		j	0	1	2	3	4	5	6
i		y	A	B	R	A	C	A	
		x	0	0	0	0	0	0	0
0	x	0	0	0	0	0	0	0	
1	A	0	↖ 1	← 1	← 1	↖ 1	← 1	↖ 1	
2	B	0	↑ 1	↖ 2	← 2	← 2	← 2	← 2	
3	C	0	↑ 1	↑ 2	↑ 2	↑ 2	↖ 3	← 3	
4	B	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3	
5	T	0	↑ 1	↑ 2	↑ 2	↑ 2	↑ 3	↑ 3	
6	A	0	↖ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4	

Passo 4: Costruire soluzione ottima

- Chiamata: PRINT-LCS(b, X, m, n)

PRINT-LCS(b, X, i, j)

if $i = 0$ or $j = 0$

return

if $b[i, j] = "↖"$

PRINT-LCS($b, X, i - 1, j - 1$)

print x_i

elseif $b[i, j] = "↑"$

PRINT-LCS($b, X, i - 1, j$)

else PRINT-LCS($b, X, i, j - 1$)

Distanza di editing

Edit Distance ($X \rightarrow Y$)

- Trasformare la stringa X nella stringa Y
- Posso usare solo operazioni elementari:
 - ▶ **Inserimento** di un carattere
 - ▶ **Cancellazione** di un carattere
 - ▶ **Sostituzione** di un carattere
 - ▶ ...
- Definiamo (come per LCS):
 $X_i = \langle x_1, x_2 \dots x_i \rangle$ e $Y_j = \langle y_1, y_2 \dots y_j \rangle$
- **Sottoproblemi**: trova sequenza minima di operazioni che converta X_i in Y_j ($0 \leq i \leq m$ e $0 \leq j \leq n$)
- **Minimizzo** le operazioni:
 il loro numero è la distanza tra stringhe
- Esempio: *Carta* $\rightarrow$ *Parla* è a distanza 2

Idea generale

- $c[i, j]$ è il costo di una soluzione ottima al problema $X_i \rightarrow Y_j$
- Suppongo di **conoscere** l'ultima operazione usata per $X_i \rightarrow Y_j$
- Sei possibilità
- Posso calcolare $c[i, j]$ in funzione di valori precedenti

Se l'ultima operazione era una **copia**

- Doveva essere $x_i = y_j$
- Resta il sottoproblema $X_{i-1} \rightarrow Y_{j-1}$
- **Sottostruttura ottima:**
Una soluzione ottima al problema $X_i \rightarrow Y_j$
deve includere una soluzione ottima a $X_{i-1} \rightarrow Y_{j-1}$
- Quindi, supponendo che l'ultima operazione fosse una copia:

$$c[i, j] = c[i - 1, j - 1] + \text{costo(copia)}$$

Se era una **sostituzione**

- Doveva essere $x_i \neq y_j$
- Sottostruttura ottima (come copia)
 - ▶ **Sottoproblema** $X_{i-1} \rightarrow Y_{j-1}$.
- Supponendo che l'ultima operazione fosse una sostituzione:

$$c[i, j] = c[i - 1, j - 1] + \text{costo(sostituzione)}$$

Se era uno **scambio**

- Doveva essere $x_i = y_{j-1}$ e $x_{i-1} = y_j$
(con $i, j \geq 2$)
- **Sottoproblema** $X_{i-2} \rightarrow Y_{j-2}$ e
- Supponendo che l'ultima operazione fosse uno scambio:

$$c[i, j] = c[i - 2, j - 2] + \text{costo(scambio)}$$

Se era una **cancellazione**

- Non ho restrizioni su X e Y
- Cancellazione: rimozione di un carattere da X_i lasciando Y_j invariato
- **Sottoproblema** $X_{i-1} \rightarrow Y_j$
- Supponendo che l'ultima operazione fosse una cancellazione:

$$c[i, j] = c[i - 1, j] + \text{costo(cancellazione)}$$

Se era una **inserimento**

- Non ho restrizioni su X e Y
- Inserimento: rimozione di un carattere da Y_j lasciando X_i invariato
- **Sottoproblema** $X_i \rightarrow Y_{j-1}$
- Supponendo che l'ultima operazione fosse un inserimento si ha

$$c[i, j] = c[i, j - 1] + \text{costo(inserimento)}$$

Casi base

- Se $i = 0$ o $j = 0$
- X_0 e Y_0 sono stringhe vuote
- Converti la **stringa vuota** in Y_j con j **inserimenti**
 - ▶ $c[0, j] = j \cdot \text{costo}(\text{inserimento})$
- Converti X_i in Y_0 con i **cancellazioni**
 - ▶ $c[i, 0] = i \cdot \text{costo}(\text{cancellazione})$
- Con $i = j = 0$ ho $c[0, 0] = 0$
 - ▶ Non c'è costo per convertire la stringa vuota nella stringa vuota

Formulazione ricorsiva

$c[i, j]$ per $i, j > 0$ applicando le formule precedenti

$$c[i, j] = \min \left\{ \begin{array}{ll} c[i-1, j-1] + \text{cost}(\text{copy}) & \text{se } x[i] = y[j] \\ c[i-1, j-1] + \text{cost}(\text{replace}) & \text{se } x[i] \neq y[j] \\ c[i-2, j-2] + \text{cost}(\text{twiddle}) & \text{se } i, j \geq 2, \\ & x[i] = y[j-1], \\ & \text{e } x[i-1] = y[j] \\ c[i-1, j] + \text{cost}(\text{delete}) & \text{sempre,} \\ c[i, j-1] + \text{cost}(\text{insert}) & \text{sempre.} \end{array} \right.$$

Algoritmo

- Come LCS lo pseudocodice riempie la tabella “per righe”
- Funzionerebbe anche “per colonne”
- $op[i, j]$ memorizza l'operazione usata

EDIT-DISTANCE(x, y)

$m \leftarrow x.length$

$n \leftarrow y.length$

Siano $c[0..m, 0..n]$ e $op[0..m, 0..n]$ due nuove tabelle

for $i \leftarrow 0$ **to** m

$c[i, 0] \leftarrow i \cdot cost(delete)$

$op[i, 0] \leftarrow DELETE$

for $j \leftarrow 0$ **to** n

$c[0, j] \leftarrow j \cdot cost(insert)$

$op[0, j] \leftarrow INSERT$

...

```

...
for  $i \leftarrow 1$  to  $m$ 
  for  $j \leftarrow 1$  to  $n$ 
     $c[i, j] \leftarrow \infty$ 
    if  $x_i = y_j$ 
       $c[i, j] \leftarrow c[i - 1, j - 1] + \text{cost}(\text{copy})$ 
       $op[i, j] \leftarrow \text{COPY}$ 
    if  $x_i \neq y_j$  and  $c[i - 1, j - 1] + \text{cost}(\text{replace}) < c[i, j]$ 
       $c[i, j] \leftarrow c[i - 1, j - 1] + \text{cost}(\text{replace})$ 
       $op[i, j] \leftarrow \text{REPLACE}(\text{by } y_j)$ 
    if  $i \geq 2$  and  $j \geq 2$  and  $x_i = y_{j-1}$  and
       $x_{i-1} = y_j$  and  $c[i - 2, j - 2] + \text{cost}(\text{twiddle}) < c[i, j]$ 
       $c[i, j] \leftarrow c[i - 2, j - 2] + \text{cost}(\text{twiddle})$ 
       $op[i, j] \leftarrow \text{TWIDDLE}$ 
    if  $c[i - 1, j] + \text{cost}(\text{delete}) < c[i, j]$ 
       $c[i, j] \leftarrow c[i - 1, j] + \text{cost}(\text{delete})$ 
       $op[i, j] \leftarrow \text{DELETE}$ 
    if  $c[i, j - 1] + \text{cost}(\text{insert}) < c[i, j]$ 
       $c[i, j] \leftarrow c[i, j - 1] + \text{cost}(\text{insert})$ 
       $op[i, j] \leftarrow \text{INSERT}(y_j)$ 
return  $c$  and  $op$ 

```

Analisi

- **Tempo** $\Theta(m \cdot n)$
- **Spazio** $\Theta(m \cdot n)$

Ricostruire la sequenza ottima di operazioni

- Si usa la tabella *op* restituita da EDIT-DISTANCE
- Chiama OP-SEQUENCE(*op*, *m*, *n*)

OP-SEQUENCE(op, i, j)

if $i = 0$ and $j = 0$

return

if $op[i, j] = \text{COPY}$ or $op[i, j] = \text{REPLACE}$

$i' \leftarrow i - 1$

$j' \leftarrow j - 1$

elseif $op[i, j] = \text{TWIDDLE}$

$i' \leftarrow i - 2$

$j' \leftarrow j - 2$

elseif $op[i, j] = \text{DELETE}$

$i' \leftarrow i - 1$

$j' \leftarrow j$

else // Deve essere $op[i, j] = \text{INSERT}$

$i' \leftarrow i$

$j' \leftarrow j - 1$

OP-SEQUENCE(op, i', j')

PRINT $op[i, j]$

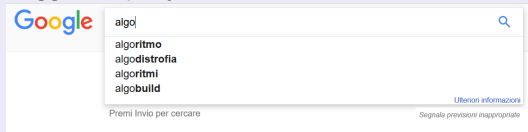
Esempio: ALABARDA e BANDANA

/	/	B	A	N	D	A	N	A
/	0	1	2	3	4	5	6	7
A	1	1	1	2	3	4	5	6
L	2	2	2	2	3	4	5	6
A	3	3	2	3	3	3	4	5
B	4	3	3	3	4	4	4	5
A	5	4	3	4	4	4	5	4
R	6	5	4	4	5	5	5	5
D	7	6	5	5	4	5	6	6
A	8	7	6	6	5	4	5	6

Utilizzi di Edit Distance

Correzione ortografica

- Due usi:
 - ▶ Correggere documenti con parole scritte in modo errato
 - ▶ Suggestire query all'utente



- Due approcci principali:
 - ▶ Parole isolate
 - ★ Controllo ogni parola indipendentemente dalle altre
 - ★ Non trovo errori di parole scambiate (es: *pino* ↔ *lino*)
 - ▶ Context-sensitive
 - ★ Guardo il contesto della frase (es: Ho tagliano un lino)

Google Books Ngram Viewer

Graph these comma-separated phrases: ☐ case-insensitivebetween and from the corpus with smoothing of [Search lots of books](#)

Correzione di documenti

- Necessario per documenti "letti" con OCR
 - ▶ Considero errori specifici
 - ★ esiste modem in documenti del XIX secolo?
 - ★ Confondo *rn* con *m*
 - ▶ Può usare conoscenza specifica del dominio
 - ★ Es., OCR può confondere O e D
- Anche documenti elettronici (ed pagine Web) hanno errori
 - ▶ Si confondono O e I (adiacenti in una tastiera QWERTY)
- Spesso non modifico il documento, ma ottimizzo il match query-documenti memorizzati

Correzione di parole isolate

- Ipotesi: esiste un lessico con lo spelling corretto delle parole
- Due approcci:
 - ▶ Usa lessico standard
 - ★ Webster's English Dictionary
 - ★ Un lessico specifico per documenti in domini specifici
 - ▶ Usa le parole della collezione
 - ★ Tutte le parole nel Web
 - ★ Tutti i nomi, acronimi ecc.
 - ★ (Inclusi gli errori !?!)
- Dato lessico L e la stringa Q , trovare le parole in L più vicine a Q
- Cosa vuol dire “più vicino”?
- Alternative:
 - ▶ Edit distance (già vista)
 - ▶ Weighted edit distance
 - ▶ Intersezione di n-gram

Weighted edit distance

- Edit Distance dove il costo di una operazione dipende dai caratteri coinvolti non solo dal tipo di operazione
 - ▶ Per gestire errori di OCR o di battitura
 - ★ Esempio: m confuso più spesso con n che con q
 - ★ Sostituendo m con n dovrei avere edit distance inferiore
- Serve una matrice di pesi W che dipende dall'applicazione

Modificare Edit distance tenendo conto di W

- ...

Usare edit distance

- Posso confrontare Q con tutte le parole in L ?
- Troppo costoso
- Una possibilità :
 - ▶ Da Q **elenco** tutte le sequenze di caratteri con edit distance inferiore ad una soglia (es. 2)
 - ▶ Interseco questo insieme con L
 - ▶ Mostro i termini trovati all'utente

Edit distance con tutto il dizionario?

- Come riduco l'insieme dei termini candidati nel dizionario?
- Una possibilità : intersezione di n-gram

Intersezione di n-gram

- Enumero tutti gli n-gram in Q e in L
- Uso un indice di n-gram per trovare tutti i termini in L che contengono **qualunque** n-gram di Q
- O cerco tutti i termini che contengono **abbastanza** n-gram di Q

Esempio con 3-gram

- $L = \text{november}$
 - ▶ 3-gram: nov, ove, vem, emb, mbe, ber
- $Q = \text{december}$
 - ▶ 3-gram: dec, ece, cem, emb, mbe, ber
- Tre 3-gram di sovrapposizione (su 6 in ogni parola)
- Come avere una misura di sovrapposizione normalizzata?

Coefficiente di Jaccard

- Dati gli insiemi X e Y
il coefficiente di Jaccard è :

$$JC = \frac{|X \cap Y|}{|X \cup Y|}$$

- X e Y non devono avere la stessa dimensione
- $0 \leq JC \leq 1$
 - ▶ 1 se X e Y hanno gli stessi elementi
 - ▶ 0 se sono disgiunti
- Nel nostro caso X e Y sono insiemi di n -gram
- Per trovare i termini considero una soglia
 - ▶ Es., se $JC > 0.8$, corrispondono

Algoritmi elementari per grafi

Grafo orientato (diretto)

- $G = (V, E)$:
 V è un insieme finito ed E è una relazione binaria in V
- V : **insieme dei vertici** di G
- E : **insieme degli archi** di G
- Arco: insieme $\{u, v\}$ dove $u, v \in V$ e $u \neq v$
- Indico l'arco con (u, v) anziché con $\{u, v\}$
- **Grado** di un vertice: numero di archi che incidono nel vertice

Grafo non orientato (indiretto)

- $G = (V, E)$ in cui E è composto da coppie di vertici *non ordinate* anziché da coppie *ordinate*

Cammini

- **Cammino** di lunghezza k dal vertice u a u' in $G = (V, E)$: sequenza $\langle v_0, v_1, v_2, \dots, v_k \rangle$ di vertici t.c. $u = v_0$, $u' = v_k$ e $(v_{i-1}, v_i) \in E$ per $i = 2, \dots, k$
Lunghezza del cammino: numero di archi nel cammino
- In G **orientato** un cammino $\langle v_0, v_1, v_2, \dots, v_k \rangle$ forma un **ciclo** se $v_0 = v_k$ e il cammino contiene almeno un arco
 Un grafo senza cicli è **aciclico**
- G non orientato è **connesso** se ogni coppia di vertici è collegata attraverso un cammino
- **Componenti connesse** di G : classi di equivalenza dei vertici secondo la relazione “è raggiungibile da”
- Un grafo orientato è **fortemente connesso** se due vertici qualsiasi sono raggiungibili l'uno dall'altro

Rappresentazione grafi

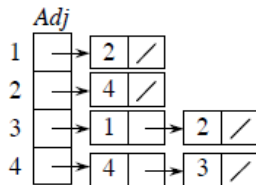
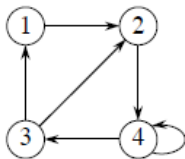
- $G = (V, E)$ può essere **diretto** o **indiretto**
- Due rappresentazioni comuni:
 - 1 Liste di adiacenza
 - 2 Matrice di adiacenza
- **Tempo** spesso espresso considerando sia $|V|$ che $|E|$
- Anche **spazio**
- In notazione asintotica si omette la cardinalità
Esempio: $O(V + E)$

Lista di adiacenza

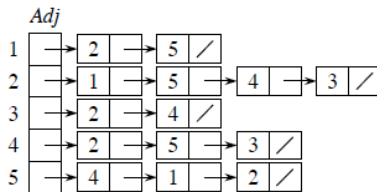
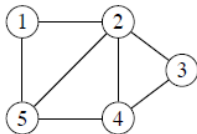
- Vettore Adj di $|V|$ **liste**, una per nodo
- $Adj[u]$ contiene tutti i nodi v t.c. $(u, v) \in E$
(per grafi indiretti considero sia (u, v) che (v, u))
- **Pesi** degli archi ($w : E \rightarrow R$) si memorizzano nelle liste
(in generale attributi)

Esempi

● Grafo **diretto**



● Grafo **indiretto**



Lista di adiacenza

- Vettore Adj di $|V|$ **liste**, una per nodo
- $Adj[u]$ contiene tutti i nodi v t.c. $(u, v) \in E$
(per grafi indiretti considero sia (u, v) che (v, u))
- **Pesi** degli archi ($w : E \rightarrow R$) si memorizzano nelle liste
(in generale attributi)

Spazio

- $\Theta(V + E)$

Tempo

- **elencare tutti** i nodi adiacenti ad u :
 $\Theta(u.degree)$
- **determinare** se $(u, v) \in E$:
 $O(u.degree)$

Matrice di adiacenza

- Matrice $|V| \times |V|$ $A = (a_{ij})$

$$a_{ij} = \begin{cases} 1 & \text{se } (i,j) \in E \\ 0 & \text{altrimenti} \end{cases}$$

Esempi visti prima

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

	1	2	3	4
1	0	1	0	0
2	0	0	0	1
3	1	1	0	0
4	0	0	1	1

Matrice di adiacenza

- Matrice $|V| \times |V|$ $A = (a_{ij})$

$$a_{ij} = \begin{cases} 1 & \text{se } (i, j) \in E \\ 0 & \text{altrimenti} \end{cases}$$

Spazio

- $\Theta(V^2)$

Tempo

- **elencare tutti** i vertici adiacenti a u :
 $\Theta(V)$
- **determinare** se $(u, v) \in E$:
 $\Theta(1)$
- Uno meglio e uno peggio rispetto a liste di adiacenza...
- Per grafi pesati memorizzo pesi invece di bit

Visita in Ampiezza (Breadth-First search)

- Scopre vertici raggiungibili da nodo sorgente $s \in V$
- **Input:** Grafo $G = (V, E)$ e nodo sorgente $s \in V$
- **Output:** $\forall v \in V$
 - ▶ $v.d$ = distanza (numero minimo di archi) da s a v
 - ▶ $v.\pi = u$ t.c. (u, v) è l'ultimo arco nel cammino minimo $s \rightsquigarrow v$
 - ★ $u (= v.\pi)$: **predecessore** di v
 - ★ l'insieme di archi $\{(v.\pi, v) : v \neq s\}$ **forma un albero**
 - ▶ Vedremo poi una generalizzazione con pesi sugli archi

Coda FIFO

- ENQUEUE inserisce in coda FIFO
- DEQUEUE rimuove da coda FIFO
- Come si potrebbe implementare?
- Quanto costano ENQUEUE e DEQUEUE?

BFS(G, s)

for ogni vertice $u \in G.V - \{s\}$

$u.color \leftarrow WHITE$

$u.d \leftarrow \infty$

$u.\pi \leftarrow NIL$

$s.color \leftarrow GRAY$

$s.d \leftarrow 0$

$s.\pi \leftarrow NIL$

$Q \leftarrow \emptyset$

ENQUEUE(Q, s)

while $Q \neq \emptyset$

$u \leftarrow DEQUEUE(Q)$

for ogni $v \in G.Adj[u]$

if $v.color = WHITE$

$v.color \leftarrow GRAY$

$v.d \leftarrow u.d + 1$

$v.\pi \leftarrow u$

 ENQUEUE(Q, v)

$u.color \leftarrow BLACK$

Idea

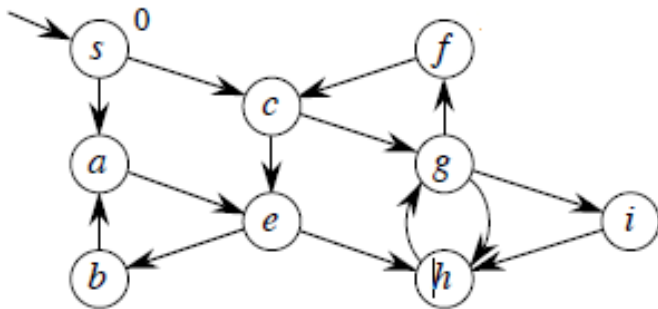
- Manda un'onda da s
 - ▶ **Prima** colpisce tutti i nodi a **distanza** 1 da s
 - ▶ **Poi** colpisce tutti i nodi a **distanza** 2 da s
 - ▶ ...
- Memorizza il fronte dell'onda nella coda Q
 $v \in Q$ sse l'onda ha colpito v , ma non è ancora uscita da v

Colore nodi

- Mentre BFS progredisce ogni nodo ha un colore:
 - ▶ *WHITE* = **non scoperto**
 - ▶ *GRAY* = **scoperto**, ma **non terminato**
 - ▶ *BLACK* = **terminato**

Esempio

Grafo diretto



Esercizio

Si consideri il seguente grafo orientato costituito da 7 vertici e da 7 archi (il grafo è rappresentato tramite liste di adiacenza)

$A \rightarrow B$

$C \rightarrow A, E, F$

$D \rightarrow C$

$E \rightarrow D$

$F \rightarrow G$

- Si disegni il grafo
- Si rappresenti tramite matrice di adiacenza
- Si esegua un attraversamento BFS partendo dal nodo A e indicando sui nodi la distanza (tramite lista di adiacenza)
- Si esegua un attraversamento BFS partendo dal nodo D e indicando sui nodi la distanza (tramite lista di adiacenza)

Esercizio

BFS(G, s)

for ogni vertice $u \in G.V - \{s\}$

$u.color \leftarrow WHITE$

$u.d \leftarrow \infty$

$u.\pi \leftarrow NIL$

$s.color \leftarrow GRAY$

$s.d \leftarrow 0$

$s.\pi \leftarrow NIL$

$Q \leftarrow \emptyset$

ENQUEUE(Q, s)

while $Q \neq \emptyset$

$u \leftarrow DEQUEUE(Q)$

for ogni $v \in G.Adj[u]$

if $v.color = WHITE$

$v.color \leftarrow GRAY$

$v.d \leftarrow u.d + 1$

$v.\pi \leftarrow u$

 ENQUEUE(Q, v)

$u.color \leftarrow BLACK$

Valori di d in Q

- Si può mostrare che Q contiene nodi con valori d nel seguente modo:

$$i, i, i, \dots, i, i+1, i+1, \dots, i+1$$

- ▶ ci sono solo 1 o 2 valori
 - ▶ se ci sono 2 valori questi differiscono di 1 e quelli minori sono prima
- Ogni nodo prende un valore d finito almeno una volta $\rightarrow$ i valori assegnati ai nodi crescono monotonamente nel tempo
- La **dimostrazione di correttezza è complessa** ...

Complessità

- **Tempo** = ??
 - ▶ Ciclo sui vertici $\Rightarrow$ **Tempo** = $\Theta(V)$?
 - ▶ Doppio ciclo sui vertici $\Rightarrow$ **Tempo** = $\Theta(V^2)$?
- BFS può non raggiungere tutti i nodi $\Rightarrow O$ e non Θ
- **Tempo** = $O(V + E)$
- Si può usare aggregazione:
 - ▶ ENQUEUE e DEQUEUE richiedono $O(1)$
 - ▶ $O(V)$ perché ogni nodo è **messo nella coda** al massimo una volta
 - ▶ $O(E)$ perché ogni nodo è **tolto dalla coda** al massimo una volta e si esamina (u, v) solo quando u è tolto dalla coda
 - ▶ Ogni arco è analizzato
 - ★ al massimo **una volta se diretto**
 - ★ al massimo **due volte se non diretto**

Correttezza

- La dimostrazione di correttezza è complessa

Visita in profondità (Depth-First search)

- **Input:** $G = (V, E)$ (diretto o indiretto), nessun nodo sorgente
- **Output** per ogni nodo v :
 - ▶ $v.d$ = tempo di **scoperta**
 - ▶ $v.f$ = tempo di **fine**
 - ▶ $v.\pi$
- $v.d$ e $v.f$ utili in seguito
- Esplora sistematicamente ogni arco
 - ▶ Ri-inizia da diversi nodi se necessario
 - ▶ Appena scopre un arco esplora da questo
 - ▶ *BFS* invece mette il nodo scoperto in coda FIFO

Colore nodi

- Ogni nodo ha un colore che cambia:
 - ▶ *WHITE* = **non scoperto**
 - ▶ *GRAY* = scoperto, ma **non terminato**
 - ▶ *BLACK* = **terminato**

Tempi di scoperta e di terminazione

- Interi unici: $1 \leq v.d < v.f \leq 2|V|$
- $\forall v: v.d < v.f$

DFS

DFS(G)

for ogni vertice $u \in G.V$

$u.color \leftarrow WHITE$

$u.\pi \leftarrow NIL$

$time \leftarrow 0$

for ogni vertice $u \in G.V$

if $u.color = WHITE$

 DFS-VISIT(G, u)

DFS-Visit

DFS-VISIT(G, u)

$time \leftarrow time + 1$

$u.d \leftarrow time$

$u.color \leftarrow GRAY$

for ogni $v \in G.Adj[u]$

if $v.color = WHITE$

$v.\pi \leftarrow u$

 DFS-VISIT(G, v)

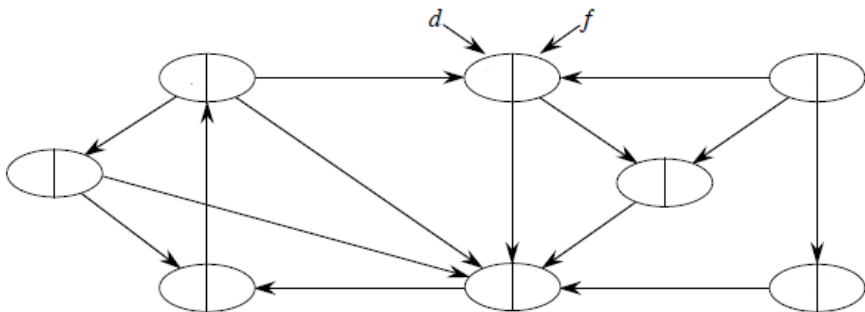
$u.color \leftarrow BLACK$

$time \leftarrow time + 1$

$u.f \leftarrow time$

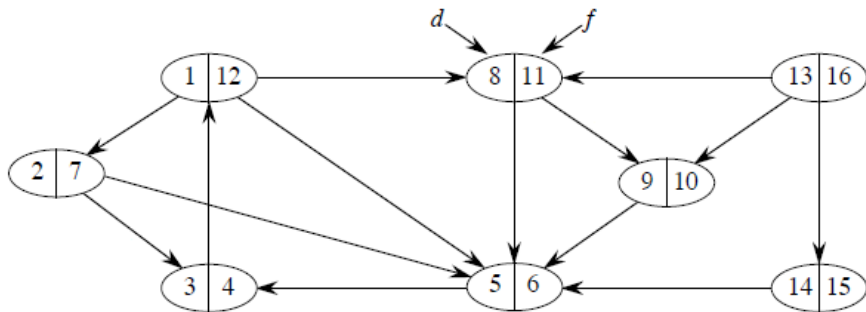
Esempio

- In ogni nodo si indica $(v.d|v.f)$



Esempio

- In ogni nodo si indica $(v.d|v.f)$



Esempio

Tempo

- $\Theta(V + E)$
 - ▶ Analisi simile a *BFS*
 - ▶ Θ , **non solo** O , perché esplora ogni nodo e arco

Foresta depth-first (DF)

- Quando scopro vertice v durante l'ispezione di $Adj[u]$ ho $v.\pi = u$
- Il sotto-grafo dei predecessori forma una **foresta depth-first** contenente almeno un albero DF
- Ogni albero contiene archi (u, v) t.c.
 - u è **gray** e
 - v è **bianco**
 quando esploro (u, v)

Teorema delle parentesi (no dim)

In una **visita in profondità** di $G(V, E) \forall$ coppia di vertici u, v è soddisfatta **una sola** delle seguenti condizioni:

- ① Gli intervalli $[u.d, u.f]$ e $[v.d, v.f]$ sono **completamente disgiunti**:
 $u.d < u.f < v.d < v.f$ **oppure** $v.d < v.f < u.d < u.f$
 e **inoltre** ne u ne v sono **un discendente** dell'altro in un albero DF
- ② L'intervallo di v è **completamente contenuto** in quello di u
 $u.d < v.d < v.f < u.f$ e
 v è un discendente di u in un albero DF
- ③ L'intervallo di u è **completamente contenuto** in quello di v
 $v.d < u.d < u.f < v.f$ e
 u è un discendente di v in un albero DF

Non può succedere che $u.d < v.d < u.f < v.f$

Come le parentesi:

- OK: $() \ [] \ (()) \ [(())]$
- Non OK: $([]) \ [()]$

Corollario (annidamento degli intervalli dei discendenti (no dim)

v è un **discendente** di u nella foresta DF sse $u.d < v.d < v.f < u.f$

Teorema del cammino bianco (no dimostrazione)

In una foresta DF di un grafo $G(V, E)$

v è un **discendente** di u

sse al tempo $u.d$, c'è un cammino $u \rightsquigarrow v$ in G fatto di soli nodi bianchi
(ancora da scoprire)

(ad eccezione di u che è stato appena colorato di grigio)

Classificazione di archi

- Durante la visita in profondità classifico gli archi del grafo:
 - ▶ **albero** (**T**, Tree): archi in foresta DF trovati esplorando (u, v)
 - ▶ **indietro** (**B**, Back): (u, v) t.c. u è un **discendente** di v
 - ▶ **avanti** (**F**, Forward): (u, v) t.c. v è un **discendente** di u , ma non è T
 - ▶ **trasversale** (**C**, Cross): ogni altro arco
 - ★ tra nodi dello stesso albero DF
 -
 - ★ di diversi alberi DF

Classificare l'arco (u, v) durante la visita

- Se $v.color = WHITE \Rightarrow T$
- Se $v.color = GRAY \Rightarrow B$
- Se $v.color = BLACK \Rightarrow F$ o C
 - ▶ Se $u.d < v.d \Rightarrow F$ v è un discendente di u (t. parentesi)
 - ▶ Se $u.d > v.d \Rightarrow C$

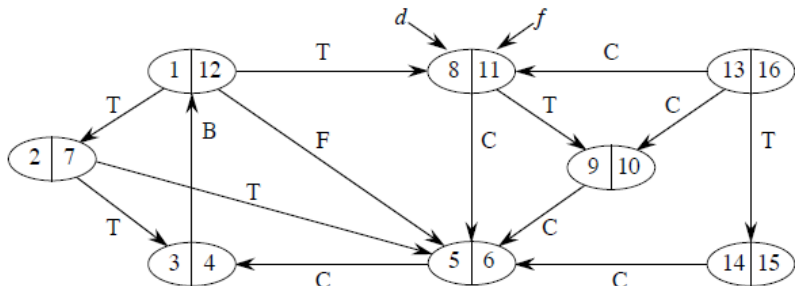
Teorema (no dim)

Con DFS di un grafo non diretto ho solo archi T e B

No archi F o C

Esempio completo

- In ogni nodo si indica $(v.d|v.f)$
- Archi etichettati (T, B, F, C)

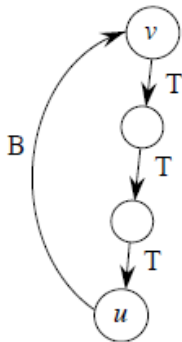


Esempio

	A	B	C	D	E	F	G	H
A	0	1	0	1	1	0	0	0
B	0	0	1	0	1	0	0	0
C	1	0	0	0	0	0	0	0
D	0	0	0	0	1	1	0	0
E	0	0	1	0	0	0	0	0
F	0	0	0	0	1	0	0	0
G	0	0	0	1	0	1	0	1
H	0	0	0	0	1	0	0	0

Lemma

Un grafo **diretto** G è **aciclico** sse
una visita DFS di G non genera archi back



Lemma

Un grafo **diretto** G è **aciclico** sse
una visita DFS di G non genera archi back

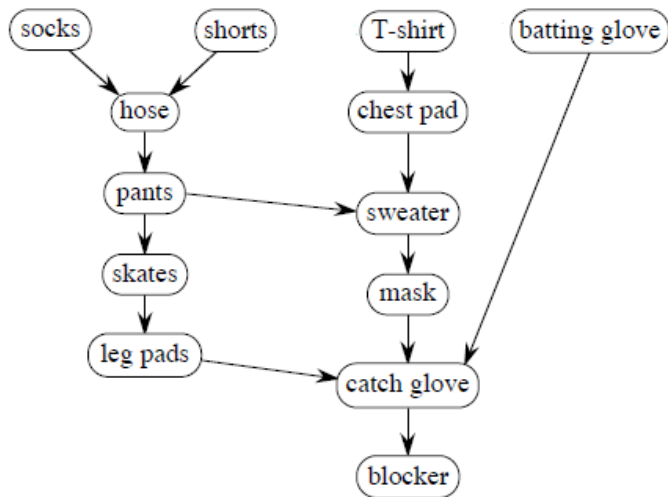
Dimostrazione

- Se c è un arco back $\Rightarrow$ ciclo
 - ▶ Supponiamo $\exists$ un arco back (u, v)
 - ▶ Allora v è un antenato di u nella foresta DF
 - ▶ Quindi c è un percorso $v \rightsquigarrow u$, insieme a $u \rightarrow v$ c è il ciclo:
 $v \rightsquigarrow u \rightarrow v$
- Se c è un ciclo $\Rightarrow$ arco back
 - ▶ Supponiamo che G contenga il ciclo c
 - ▶ Sia v il primo nodo scoperto in c e sia (u, v) l'arco precedente in c
 - ▶ Al tempo $v.d$, i vertici di c formano un cammino bianco $v \rightsquigarrow u$ (poiché v è il primo vertice scoperto in c)
 - ▶ Da teorema del cammino bianco u è discendente in v nella foresta DF
 - ▶ Quindi, (u, v) è un arco back.

Ordinamento topologico

- Un **ordinamento topologico** di un grafo aciclico (DAG: Directed Acyclic Graph) $G = (V, E)$ è un
 - ▶ **ordinamento lineare** di tutti i suoi vertici t.c.
 - ▶ se G **contiene un arco** (u, v) allora u **appare prima di** v nell'ordinamento (da qualche parte)
 - ▶ $\neq$ ordinamento di numeri
- Per grafi ciclici **non è possibile** effettuare ordinamento lineare
- Utile per gestire oggetti che hanno un **ordine parziale**:
 - ▶ $a > b$ e $b > c \Rightarrow a > c$
 - ▶ Potrei avere a e b t.c. non si sa se $a > b$ o $b > a$ (esempio, so che $a > c$ e $b > c$)
- Si può sempre avere un ordinamento **totale** ($a > b$ o $b > a \forall a \neq b$) da un ordinamento **parziale**

Esempio: vestire un portiere di pattinaggio su ghiaccio



socks (calzini) - shorts (pantaloncini) - hose (calze)- pants (pantaloni) - skates (pattini)
 - leg pads (paragambe) - t-shirt - chest pad (sternale) - sweater (maglione) - mask
 (maschera) - batting glove (mano dx) - catch glove (mano sx) - blocker (mano dx)

Ordinamento topologico: algoritmo

TOPOLOGICAL-SORT(G)

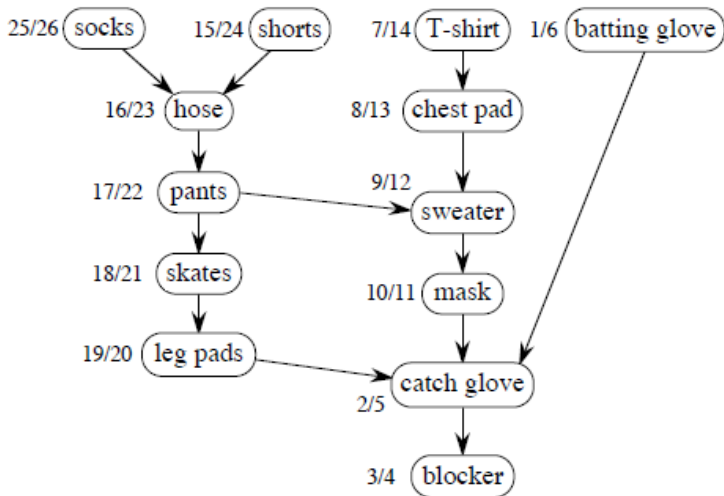
- 1 Chiama DFS(G) per calcolare i tempi di completamento $v.f$ per ogni vertice v
- 2 Completata l'ispezione del vertice, inserisce il vertice in testa a una lista concatenata
- 3 **return** la lista concatenata di vertici

- 3 equivale a:
"Emetti i vertici in ordine di tempo di terminazione decrescente"
- Non serve ordinare per tempo di terminazione

Tempo

$\Theta(V + E)$

Esempio: vestire un portiere di pattinaggio su ghiaccio



Esempio

- 26 socks (calzini)
- 24 shorts (pantaloncini)
- 23 hose (calze)
- 22 pants (pantaloni)
- 21 skates (pattini)
- 20 leg pads (paragambe)
- 14 t-shirt
- 13 chest pad (sternale)
- 12 sweater (maglione)
- 11 mask (maschera)
- 6 batting glove (guanto per la mano dx)
- 5 catch glove (per la mano sinistra)
- 4 blocker (per la mano destra)

Correttezza

Mostriamo che se $(u, v) \in E$, allora $u.f > v.f$

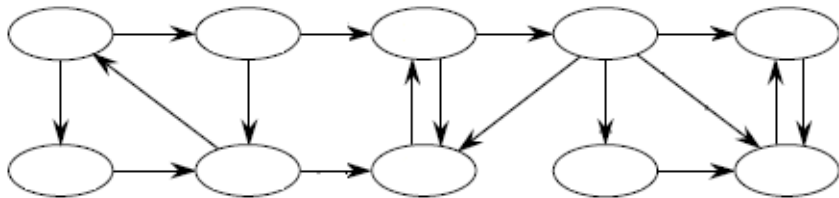
Quando **esploro** (u, v) , che colori hanno u e v ?

- u è **grigio**
- Anche v è **grigio**?
 - ▶ No, altrimenti v sarebbe un antenato di u
 - ★ $\Rightarrow (u, v)$ è un arco back
 - ★ $\Rightarrow$ contraddizione del lemma precedente (DAG non ha archi back)
- E' v **bianco**?
 - ▶ Allora diventa un discendente di u
 Dal teorema delle parentesi si ha $u.f > v.f > v.d > u.d$
 $\Rightarrow u.f > v.f$
- E' v **nero**?
 - ▶ Allora v è già terminato
 Esploriamo (u, v) quindi u non ancora terminato
 $\Rightarrow u.f > v.f$

Componenti fortemente connesse

- Dato il grafo diretto $G = (V, E)$
- Una componente fortemente connessa (Strongly Connected Component SCC) di G è un **insieme massimale di vertici** $C \subseteq V$ t.c. per ogni $u, v \in C$, esistono entrambi i cammini $u \rightsquigarrow v$ e $v \rightsquigarrow u$

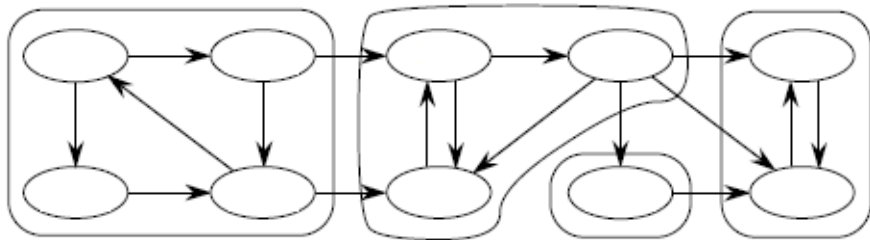
Esempio



Componenti fortemente connesse

- Dato il grafo diretto $G = (V, E)$
- Una componente fortemente connessa (Strongly Connected Component SCC) A di G è un **insieme massimale di vertici** $C \subseteq V$ t.c. per ogni $u, v \in C$, esistono entrambi i cammini $u \rightsquigarrow v$ e $v \rightsquigarrow u$

Esempio



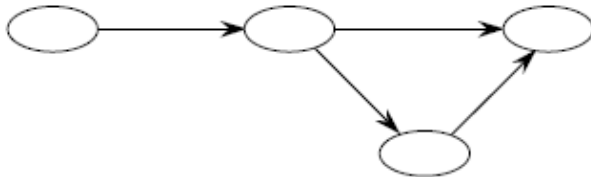
G^T = **trasposta** di G

- $G^T = (V, E^T)$
 - ▶ $E^T = \{(u, v) : (v, u) \in E\}$
- G^T è G con tutti gli archi invertiti
- Creazione di G^T
 - ▶ semplice con matrice di adiacenza
 - ▶ in tempo $\Theta(V + E)$ con liste di adiacenza
- G e G^T hanno le **stesse** SCC
 - ▶ u e v sono raggiungibili reciprocamente in G sse lo sono in G^T

Grafo delle componenti

- $G^{SCC} = (V^{SCC}, E^{SCC})$
 - ▶ V^{SCC} ha un vertice per ogni SCC in G
 - ▶ E^{SCC} ha un arco se c'è un arco fra le corrispondenti SCC in G

Per esempio di prima?



Lemma 22.13: G^{SCC} è un DAG

Formalmente:

- siano C e C' due SCC distinte in G
siano $u, v \in C$, $u', v' \in C'$, e
supponiamo che ci sia un cammino $u \rightsquigarrow u'$ in G
- **allora** non ci può anche essere un cammino $v' \rightsquigarrow v$ in G

Dimostrazione

- Supponiamo che ci sia un cammino $v' \rightsquigarrow v$ in G
- allora ci sono cammini
 $u \rightsquigarrow u' \rightsquigarrow v'$ e
 $v' \rightsquigarrow v \rightsquigarrow u$ in G
- u e v' sono raggiungibili l'uno dall'altro e quindi non sono in SCC separate (contraddizione ipotesi)

Componenti fortemente connesse: algoritmo

STRONGLY-CONNECTED-COMPONENTS(G)

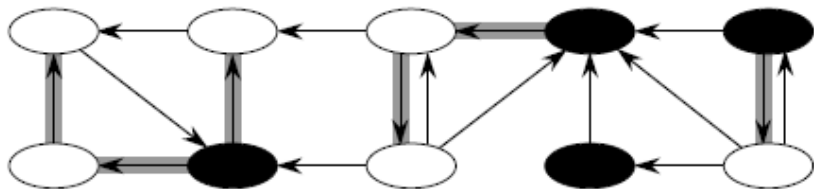
- 1 Chiama DFS(G) per calcolare $v.f$ per ogni vertice v
- 2 Calcola G^T
- 3 Chiama DFS(G^T), nel ciclo in DFS considera i vertici in ordine decrescente rispetto ai tempi $u.f$ (calcolati nella riga 1)
- 4 Genera l'output dei vertici di ciascun albero della foresta DF prodotta alla riga 3 come una singola componente fortemente connessa

Tempo

$$\Theta(V + E)$$

Esempio di prima

- 1 Fare DFS
- 2 G^T
- 3 DFS



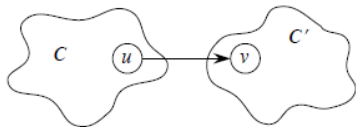
Grafo di G^T quindi archi girati rispetto a prima (radici foresta DF annerite)

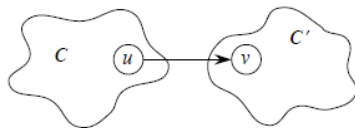
Notazione per lemma

- $u.d$ e $u.f$ si riferiscono al primo DFS
- Estendo notazione per d e f ad insiemi di vertici $U \subseteq V$:
 - ▶ $d(U) = \min_{u \in U} \{u.d\}$ (primo tempo di scoperta)
 - ▶ $f(U) = \max_{u \in U} \{u.f\}$ (ultimo tempo di fine)

Lemma 22.14

- Siano C e C' due SCC distinte in $G = (V, E)$ e
- supponiamo esista un arco $(u, v) \in E$ t.c. $u \in C$ e $v \in C'$
- **allora** $f(C) > f(C')$

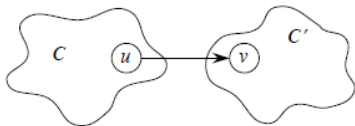




Dimostrazione 1/2

Due casi a seconda del primo tra $d(C)$ e $d(C')$

- Se $d(C) < d(C')$
 - ▶ x : **primo** vertice scoperto in C
 - ▶ al tempo $x.d$, tutti i vertici in C e C' sono bianchi
 - ▶ cammini di vertici bianchi da x a tutti i vertici in C e C'
 - ▶ teorema **cammino bianco**: tutti i vertici in C e C' sono discendenti di x nell'albero DF
 - ▶ teorema delle **parentesi**: $x.f = f(C) > f(C')$



Dimostrazione 2/2

- Se $d(C) > d(C')$

- ▶ y : primo vertex scoperto in C'
- ▶ al tempo $y.d$
 - ★ tutti i vertici in C' sono bianchi e
c'è un cammino bianco da y ad ogni vertice in C'
 - ★ $\Rightarrow$ tutti i vertici in C' sono discendenti di y
 $\Rightarrow y.f = f(C')$
- ▶ al tempo $y.d$
 - ★ tutti i vertici in C sono bianchi ($d(C) > d(C')$)
 - ★ G^{SCC} è un DAG (lemma) c'è un arco $(u, v) \Rightarrow \nexists$ arco da C' a C
 - ★ Nessun vertice in C è raggiungibile da y
 - ★ Al tempo $y.f$ tutti i vertici in C **sono ancora bianchi**
 - ★ $\Rightarrow \forall w \in C, w.f > y.f = f(C')$
- ▶ $\Rightarrow f(C) > f(C')$

Lemma 22.14 (appena dimostrato)

- Siano C e C' due SCC distinte in $G = (V, E)$ e
- supponiamo esista un arco $(u, v) \in E$ t.c. $u \in C$ e $v \in C'$
- **allora** $f(C) > f(C')$

Corollario 22.15

- Siano C e C' due SCC distinte in $G = (V, E)$ e
- **supponiamo** che ci sia un arco $(u, v) \in E^T$, dove $u \in C$ e $v \in C'$
- **allora** $f(C) < f(C')$

Dimostrazione

$$(u, v) \in E^T \Rightarrow (v, u) \in E$$

Le SCC di G e G^T sono le stesse, $f(C') > f(C)$

Corollario 22.15 (appena dimostrato)

- Siano C e C' due SCC distinte in $G = (V, E)$ e
- **supponiamo** che ci sia un arco $(u, v) \in E^T$, dove $u \in C$ e $v \in C'$
- **allora** $f(C) < f(C')$

Corollario

- Siano C e C' due SCC distinte in $G = (V, E)$, e
- **supponiamo** che $f(C) > f(C')$
- **allora** non ci può essere un arco tra C e C' in G^T

Dimostrazione

Supponiamo per assurdo che esista l'arco
allora si avrebbe $f(C) < f(C')$ (Corollario 22.15)
contraddicendo l'ipotesi $f(C) > f(C')$

Intuizione su correttezza di SCC

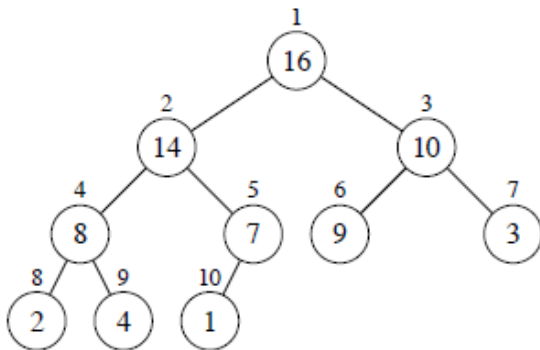
- DFS su G^T inizia con SCC C t.c. $f(C)$ è massimo
 - ▶ Inizia da $x \in C$ e visita tutti i vertici in C
 - ▶ Siccome $f(C) > f(C')$ da corollario $\forall C' \neq C$
 $\Rightarrow$ no archi da C a C' in G^T
 - ▶ $\Rightarrow$ DFS visita solo vertici in C
- Vertice successivo: $y \in C'$ t.c. $f(C')$ è massimo in tutte le $\text{SCC} \neq C$
 - ▶ DFS visita tutti i vertici in C'
i soli archi che escono da C' vanno in C già visitato
- ...
- Si continua
 - ▶ Quando scelgo un vertice posso raggiungere solo:
 - ★ vertici nella sua SCC
 - ★ vertici in SCC **già visitati** nel secondo DFS

Heap

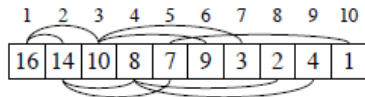
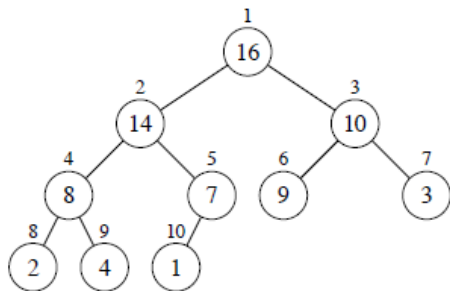
Heap

- Albero binario **quasi completo**

- ▶ **Altezza** di un nodo =
di archi nel cammino semplice più lungo dal nodo ad una foglia
- ▶ **Altezza** di heap =
altezza della radice = $\Theta(\lg n)$



- Heap **può** essere memorizzato in un array A
 - ▶ Radice dell'albero: $A[1]$
 - ▶ Padre di $A[i] \rightarrow A[\lfloor i/2 \rfloor]$
 - ▶ Figlio sx di $A[i] \rightarrow A[2i]$
 - ▶ Figlio dx di $A[i] \rightarrow A[2i + 1]$
- Due tipi: **max-heap** e **min-heap**



Proprietà heap

- **max-heap:**

$\forall$ nodo i , eccetto la radice

$$A[\text{PARENT}(i)] \geq A[i]$$

- **min-heap:**

$\forall$ nodo i , eccetto la radice

$$A[\text{PARENT}(i)] \leq A[i]$$

- Per induzione e transitività di $\geq$ proprietà max-heap garantisce che il **massimo** di un max-heap è la **radice**
- Analogamente per min-heap
- HEAPSORT usa max-heap

MAX-HEAPIFY conserva proprietà max-heap

- Prima di MAX-HEAPIFY, $A[i]$ può essere più piccolo dei suoi figli
- I sottoalberi sx e dx di i sono max-heap
- Dopo MAX-HEAPIFY, il sottoalbero con radice i è un max-heap

MAX-HEAPIFY(A, i)

$l \leftarrow \text{LEFT}(i)$

$r \leftarrow \text{RIGHT}(i)$

if $l \leq A.\text{heap-size}$ and $A[l] > A[i]$

$\text{massimo} \leftarrow l$

else $\text{massimo} \leftarrow i$

if $r \leq A.\text{heap-size}$ and $A[r] > A[\text{massimo}]$

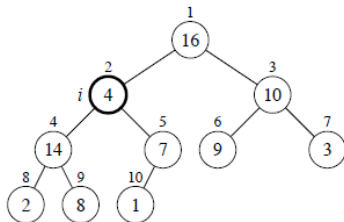
$\text{massimo} \leftarrow r$

if $\text{massimo} \neq i$

scambia $A[i]$ con $A[\text{massimo}]$

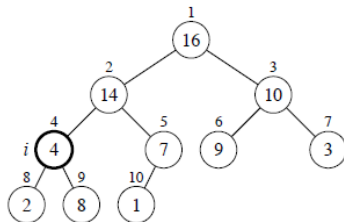
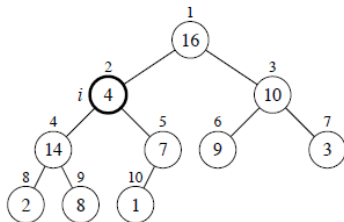
MAX-HEAPIFY($A, \text{massimo}$)

Esempio



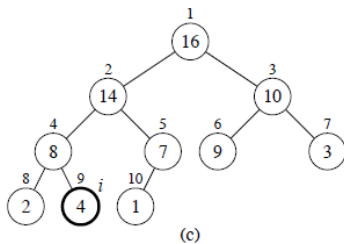
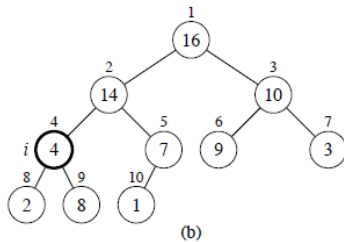
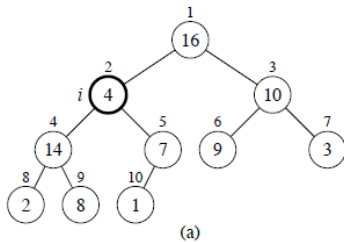
- Nodo 2 viola inizialmente la proprietà max-heap

Esempio



- Nodo 2 viola inizialmente la proprietà max-heap

Esempio



- Nodo 2 viola inizialmente la proprietà max-heap

Analisi

Tempo

- $O(\lg n)$
- libro: analisi formale con la ricorrenza
- intuitivamente
 - ▶ Heap è un albero binario quasi completo
 - ▶ $O(\lg n)$ livelli; ad ogni livello
 - ★ confronto 3 elementi e al max ne scambio 2

Correttezza

- Con **invariante di ciclo**
- Non lo vediamo...

Costruire un heap

Dato array non ordinato produce max-heap bottom-up

BUILD-MAX-HEAP(A)

$A.heap-size \leftarrow A.length$

for $i \leftarrow \lfloor A.length/2 \rfloor$ **downto** 1

 MAX-HEAPIFY(A, i)

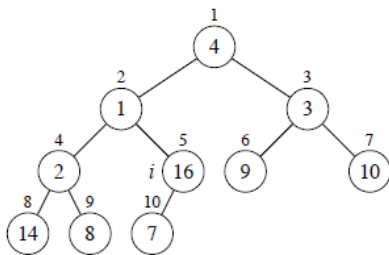
Nota

da $\lfloor A.length/2 \rfloor$ in poi sono foglie, quindi max-heap!

Esempio

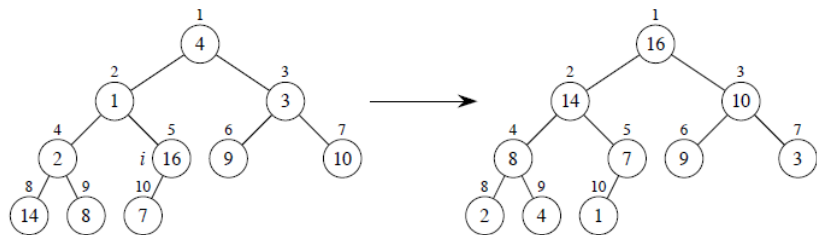
Costruire un max-heap da:

	1	2	3	4	5	6	7	8	9	10
A	4	1	3	2	16	9	10	14	8	7



- i inizia da 5
- MAX-HEAPIFY applicato a sottoalberi con radici 16, 2, 3, 1, 4

Esempio



Code con priorità

- Gestiscono un **insieme dinamico** S
- Ogni elemento nell'insieme ha una **chiave**
- Operazioni con Max-priority:
 - ▶ $\text{INSERT}(S, x)$: inserisce l'elemento x in S
 - ▶ $\text{MAXIMUM}(S)$: restituisce l'elemento di S con massima chiave
 - ▶ $\text{EXTRACT-MAX}(S)$: restituisce e rimuove l'elemento di S con massima chiave
 - ▶ $\text{INCREASE-KEY}(S, x, k)$: incrementa il valore dell'elemento x a k ($k \geq$ valore corrente della chiave di x)

Esempio di applicazione di max-priority

- Schedulazione job

Implementazione di coda con priorit  tramite heap

max-priority queue implementata con max-heap

- con Heap **compromesso** tra
 - ▶ **coda**: rapido inserimento, lenta estrazione
inserisco subito, ma la devo scorrere (o ordinare) per trovare max
 - ▶ **coda ordinata** lento inserimento, rapida estrazione
la devo scorrere per inserire, ma prendo subito l'elemento buono
- Con heap **entrambe** le operazioni prendono tempo $O(\lg n)$
- Min-priority queue implementata con min-heap

Trovare il massimo

E' semplice:   la radice

HEAP-MAXIMUM(A)

return $A[1]$

Tempo

$\Theta(1)$

Estrarre il massimo

HEAP-EXTRACT-MAX(A)

```

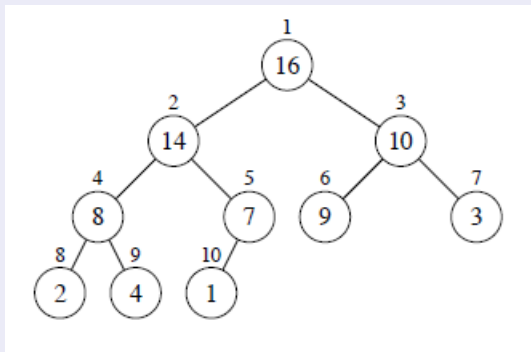
if  $A.heap-size < 1$ 
    error "underflow dell'heap"
 $max \leftarrow A[1]$ 
 $A[1] \leftarrow A[A.heap-size]$ 
 $A.heap-size \leftarrow A.heap-size - 1$ 
MAX-HEAPIFY( $A, 1$ )
return  $max$ 

```

Analisi

- Assegnamenti di tempo costante più tempo per MAX-HEAPIFY
- **Tempo:** $O(\lg n)$

Esempio: eseguire HEAP-EXTRACT-MAX



- Prende 16 da nodo 1
- Muove 1 dal nodo 10 al nodo 1
- Cancella nodo 10
- MAX-HEAPIFY dalla radice per preservare le proprietà max-heap

Incrementare valore chiave

Input

Insieme S , elemento i , nuovo valore per key

HEAP-INCREASE-KEY(A, i, key)

if $key < A[i]$

error “nuova chiave piu’ piccola della corrente”

$A[i] \leftarrow key$

while $i > 1$ and $A[Parent(i)] < A[i]$

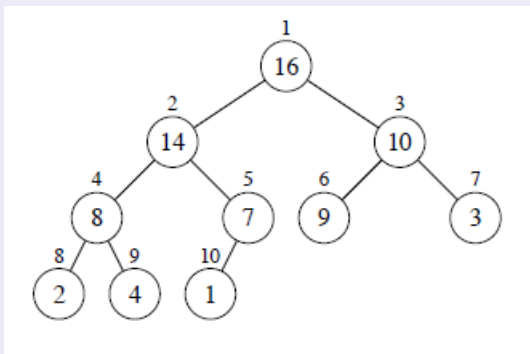
scambia $A[i] \leftrightarrow A[Parent(i)]$

$i \leftarrow Parent(i)$

Analisi

- Cammino verso l'alto da nodo i è lungo $O(\lg n)$
- **Tempo:** $O(\lg n)$

Esempio: incrementare *key* di 9 a 15



- Scambia le chiavi dei nodi 4 e 9
- poi dei nodi 2 e 4

Inserimento in heap

MAX-HEAP-INSERT(A, key)

$A.heap-size \leftarrow A.heap-size + 1$

$A[A.heap-size] \leftarrow -\infty$

HEAP-INCREASE-KEY($A, A.heap-size, key$)

- La sentinella serve per non far fallire il primo if in HEAP-INCREASE-KEY

Analisi

- Assegnamenti con tempo costante + tempo per HEAP-INCREASE-KEY
- **Tempo:** $O(\lg n)$

Strutture dati per insiemi disgiunti

Strutture dati per insiemi disgiunti

- Noto come: **union-find**
- Gestisce una collezione $S = \{S_1, \dots, S_k\}$ di **insiemi disgiunti dinamici**
- Ogni insieme è identificato da un **rappresentante** (un membro dell'insieme)
Non importa qual è il rappresentante, ma è **sempre lo stesso**

Strutture dati per insiemi disgiunti

Operazioni

- **MAKE-SET(x)**
crea un nuovo insieme $S_i = \{x\}$ e aggiunge S_i a S
- **UNION(x, y)**
se $x \in S_x, y \in S_y, \Rightarrow S \leftarrow S - S_x - S_y \cup \{S_x \cup S_y\}$
 - ▶ **Rappresentante** è qualunque elemento di $S_x \cup S_y$ (es. uno dei rappresentanti di S_x o S_y)
 - ▶ **Distrugge** S_x e S_y (insiemi disgiunti)
- **FIND-SET(x)**
ritorna il rappresentante dell'insieme che contiene x

Applicazione

Componenti connesse

In un grafo $G = (V, E)$, i vertici u e v sono nella stessa componente connessa sse c'è un cammino tra loro

- Le componenti connesse partizionano i vertici in classi di equivalenza
- Un grafo non orientato è **connesso** se ogni coppia di vertici è collegata tramite un cammino
- Un grafo orientato può essere **fortemente connesso**
- In ogni caso: classi di equivalenza secondo la relazione "è raggiungibile da"

Algoritmo

CONNECTED-COMPONENTS(G)

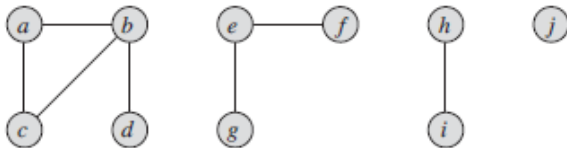
for ogni vertice $v \in G.V$

 MAKE-SET(v)

for ogni arco $(u, v) \in G.E$

if FIND-SET(u) $\neq$ FIND-SET(v)

 UNION(u, v)



Arco	Insiemi disgiunti									
MAKE-SET	{a}	{b}	{c}	{d}	{e}	{f}	{g}	{h}	{i}	{j}
(b,d)	{a}	{b,d}	{c}		{e}	{f}	{g}	{h}	{i}	{j}
(e,g)	{a}	{b,d}	{c}		{e,g}	{f}		{h}	{i}	{j}
(a,c)	{a,c}	{b,d}			{e,g}	{f}		{h}	{i}	{j}
(h,i)	{a,c}	{b,d}			{e,g}	{f}		{h,i}		{j}
(a,b)	{a,b,c,d}				{e,g}	{f}		{h,i}		{j}
(e,f)	{a,b,c,d}				{e,f,g}			{h,i}		{j}
(b,c)	{a,b,c,d}				{e,f,g}			{h,i}		{j}

Rappresentazione con lista concatenata

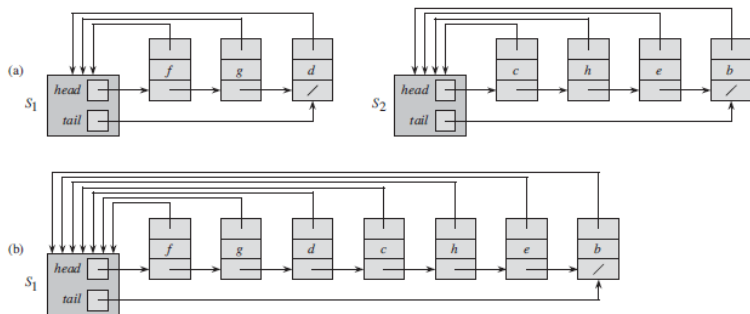
- Ogni insieme è una singola lista concatenata
- Ogni nodo nella lista ha campi per:
 - ▶ l'elemento dell'insieme
 - ▶ puntatore al rappresentante
 - ▶ puntatore al successore
- La lista ha testa (puntatore al rappresentante) e coda

Operazioni

MAKE-SET: crea una lista con un solo elemento

FIND-SET: restituisce il puntatore al rappresentante

UNION: è quello che costa. Ci sono un paio di modi per implementarlo (ne vediamo uno)



$UNION(X, Y)$: accoda la lista di x alla fine della lista di y

Usa il puntatore alla coda di y per trovare la fine

- Aggiornare il puntatore al rappresentante per ogni nodo nella lista di x
- Accoda una lista lunga ad una corta può prendere abbastanza tempo

Euristica dell'unione pesata

- accodare sempre la lista corta alla lunga
- una singola UNION può sempre richiedere tempo $\Omega(n)$
 - ▶ entrambi gli insiemi hanno $n/2$ membri

Teorema

Tempo con unione pesata per una sequenza di m operazioni su n elementi
 $O(m + n \lg n)$

Alberi di connessione minimi

Alberi di connessione minimi

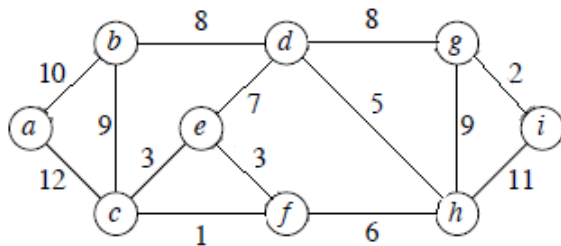
Una città ha un insieme di case e un insieme di strade

- La strada (u, v) collega la casa u con la casa v
- $w(u, v)$ costo per passare una linea ADSL su strada (u, v)
- **Obiettivo:** collegare tutte le case a Internet
 - 1 da ogni casa raggiungo ogni casa (ognuno è **connesso**) e
 - 2 costo totale di collegamento è **minimo**

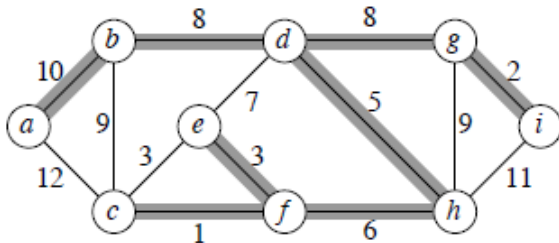
Modellato con un grafo non diretto $G = (V, E)$

- **Peso** $w(u, v) \quad \forall (u, v) \in E$
- Trovare $T \subseteq E$ t.c.
 - 1 T connette tutti i vertici (albero di connessione)
 - 2 $w(T) = \sum_{(u,v) \in T} w(u, v)$ (minimo)
- Minimum Spanning Tree: MST

Esempio di grafo



Esempio di grafo (archi in MST sono ombreggiati)



Proprietà di MST

- Quanti archi? $|V| - 1$
- Non ha cicli (è un albero ...)
- Può non essere unico
 - ▶ nell'esempio se $(e, f) \leftrightarrow (c, e)$
altro MST con **stesso** peso

Costruire la soluzione

- A è un insieme di archi
- Inizialmente $A = \emptyset$
- Aggiungo archi ad A conservando un
invariante di ciclo: A è un sottoinsieme di qualche MST
 - ▶ Se A è un sottoinsieme di qualche MST
 (u, v) è **sicuro** per A sse
 $A \cup \{(u, v)\}$ è un sottoinsieme di qualche MST
 - ▶ Aggiungo solo archi sicuri

Algoritmo generico MST

GENERIC-MST(G, w)

```
1   $A \leftarrow \emptyset$ 
2  while  $A$  non forma un albero di connessione
3      trova un arco  $(u, v)$  che e' sicuro per  $A$ 
4       $A \leftarrow A \cup \{(u, v)\}$ 
5  return  $A$ 
```

Correttezza con invariante di ciclo

- **Inizializzazione:** Insieme vuoto è sottoinsieme di MST
- **Conservazione:** Aggiungo solo archi sicuri $\Rightarrow$
 A rimane un sottoinsieme di qualche MST
- **Terminazione:** Tutti gli archi aggiunti ad A sono in un MST
 $\Rightarrow A$ è un albero di copertura che è anche un MST

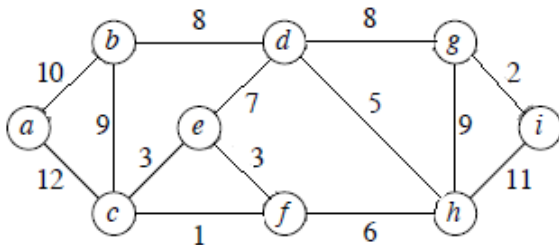
Trovare un arco sicuro

- Esempio: arco (c, f) ha il peso minore

E' sicuro per $A = \emptyset$?

Intuitivamente:

- ▶ $S \subset V$ un qualunque insieme di vertici t.c. $c \in S$ e $f \notin S$ ($f \in V - S$)
- ▶ Ogni MST deve avere almeno un arco che connette S con $V - S$
- ▶ Scegliamo l'arco con peso minimo (in questo caso (c, f))



Definizioni

$G = (V, E)$ un grafo non orientato

$S \subset V$ e $A \subseteq E$

- **taglio** $(S, V - S)$:
partizione di V in insiemi disgiunti S e $V - S$
- Un arco $(u, v) \in E$ **attraversa** il taglio $(S, V - S)$ se $u \in S$ e $v \in V - S$ (o viceversa)
- Un taglio **rispetta** un insieme di archi A sse nessun arco in A attraversa il taglio
- Un arco è **leggero** per un taglio se ha peso **minimo** tra tutti gli archi che attraversano il taglio

Teorema

Sia A un sottoinsieme di qualche MST,
 $(S, V - S)$ sia un taglio che rispetta A e
 (u, v) sia un arco leggero che attraversa $(S, V - S)$
Allora (u, v) è sicuro per A

Dimostrazione 1/3

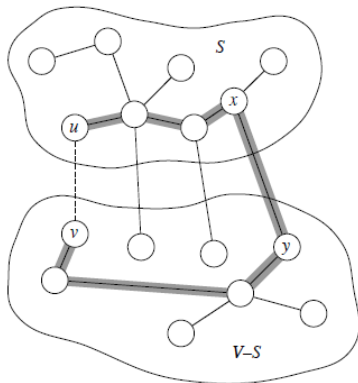
Sia T un MST che contiene A

- Se T contiene (u, v) : **OK** (u, v) è sicuro per A
- altrimenti costruisco un altro MST T' che include $A \cup \{(u, v)\}$

► ...

Dimostrazione 2/3

- altrimenti costruisco un altro MST T' che include $A \cup \{(u, v)\}$
 - ▶ T è MST $\Rightarrow$ ha un unico cammino p tra u e v
 - ▶ p attraversa il taglio $(S, V - S)$ almeno una volta in (x, y)
 - ▶ (u, v) è leggero $\Rightarrow w(u, v) \leq w(x, y)$
 - ▶ ...



In figura

- Archi (tranne (u, v)) sono in T
- A è qualche sottoinsieme di archi di T
- Archi ombreggiati: cammino p

Dimostrazione 3/3

- altrimenti costruisco un altro MST T' che include $A \cup \{(u, v)\}$
 - ▶ ...
 - ▶ Il taglio rispetta $A \Rightarrow (x, y) \notin A$
 - ▶ Costruisco $T' = T - \{(x, y)\} \cup \{(u, v)\}$
 - $w(T') = w(T) - w(x, y) + w(u, v)$
 - $\leq w(T)$ perché $w(u, v) \leq w(x, y)$
 - ▶ T' è un albero di copertura, $w(T') \leq w(T)$, e T è un MST
 - ▶ $\Rightarrow T'$ è MST
 - ▶ Quindi (u, v) è **sicuro** per A :
 - ★ $A \subseteq T$ e $(x, y) \notin A \Rightarrow A \subseteq T'$
 - ★ $A \cup \{(u, v)\} \subseteq T'$
 - ★ T' è un MST $\Rightarrow (u, v)$ è sicuro per A

GENERIC-MST(G, w)

```
1   $A \leftarrow \emptyset$ 
2  while  $A$  non forma un albero di connessione
3      trova un arco  $(u, v)$  che e' sicuro per  $A$ 
4       $A \leftarrow A \cup \{(u, v)\}$ 
5  return  $A$ 
```

In GENERIC-MST:

- $G_A = (V, A)$ è una foresta che contiene componenti connesse
 - ▶ Inizialmente ogni componente è un singolo vertice
- Ogni arco sicuro fonde due componenti
- MST ha $|V| - 1$ archi
 - $\Rightarrow$ *for* itera $|V| - 1$ volte
- Alla fine $|V| - 1$ archi sicuri $\Rightarrow$ una sola componente

Corollario identifico S

Se $C = (V_C, E_C)$ è una componente connessa nella foresta $G_A = (V, A)$ e (u, v) è un arco leggero che connette C ad un'altra componente in G_A (i.e. (u, v) è un arco leggero che attraversa il taglio $(V_C, V - V_C)$)
allora (u, v) è sicuro per A

Dimostrazione

Porre $S = V_C$ nel teorema

Questo porta all'algoritmo di **Kruskal**

Strutture dati per insiemi disgiunti

- **union-find**
- Gestisce una collezione $S = \{S_1, \dots, S_k\}$ di insiemi disgiunti dinamici
- Ogni insieme è identificato da un rappresentante (un elemento dell'insieme)
 - ▶ Non importa quale, ma sempre lo stesso

Operazioni

- **MAKE-SET(x)**: crea un nuovo insieme $S_i = \{x\}$ e aggiunge S_i a S
- **UNION(x, y)**: se $x \in S_x$, $y \in S_y$, allora

$$S \leftarrow S - S_x - S_y \cup \{S_x \cup S_y\}$$
 - ▶ distrugge S_x e S_y (insiemi devono essere disgiunti)
 - ▶ il rappresentante di S è qualunque elemento di $S_x \cup S_y$
- **FIND-SET(x)**: ritorna il rappresentante dell'insieme che contiene x

MST-KRUSKAL(G, w)

$A \leftarrow \emptyset$

for ogni vertice $v \in G.V$

 MAKE-SET(v)

ordina gli archi di $G.E$ in senso non decrescente rispetto al peso w

for ogni arco $(u, v) \in G.E$, preso in ordine di peso non decrescente

if FIND-SET(u) $\neq$ FIND-SET(v)

$A \leftarrow A \cup \{(u, v)\}$

 UNION(u, v)

return A

- Potrei aggiungere un test per fermarsi dopo $|V| - 1$ UNION

Analisi

Initializzare A : $O(1)$
Primo ciclo for : $|V|$ volte MAKE-SET
Ordinare E : $O(E \lg E)$
Secondo ciclo for : $O(E)$ volte (FIND-SET e UNION)

- Dipende da implementazione union-find!

Richiami

Algoritmo generico

GENERIC-MST(G, w)

```

1   $A \leftarrow \emptyset$ 
2  while  $A$  non forma un albero di connessione
3      trova un arco  $(u, v)$  che e' sicuro per  $A$ 
4       $A \leftarrow A \cup \{(u, v)\}$ 
5  return  $A$ 
  
```

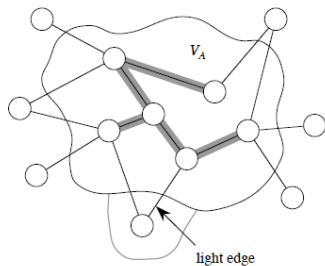
Teorema

Sia A un sottoinsieme di qualche MST,
 $(S, V - S)$ sia un taglio che rispetta A e
 (u, v) sia un arco leggero che attraversa $(S, V - S)$

Allora (u, v) è sicuro per A

Algoritmo di Prim

- Caso particolare di GENERIC-MST
- A è sempre un albero
- Inizia da “radice” arbitraria r
- Ad ogni passo, trova un **arco leggero** che attraversa il **taglio** $(V_A, V - V_A)$, dove $V_A = \{ \text{vertici toccati da archi di } A \}$
- Lo aggiunge ad A



Archi di A ombreggiati

MST-PRIM(G, w, r)

$Q \leftarrow \emptyset$

for ogni $u \in G.V$

$u.key \leftarrow \infty$

$u.\pi \leftarrow \text{NIL}$

INSERT(Q, u)

DECREASE-KEY($Q, r, 0$) // $r.key \leftarrow 0$

while $Q \neq \emptyset$

$u \leftarrow \text{EXTRACT-MIN}(Q)$

for ogni $v \in G.Adj[u]$

if $v \in Q$ and $w(u, v) < v.key$

$v.\pi \leftarrow u$

DECREASE-KEY($Q, v, w(u, v)$) // $v.key \leftarrow w(u, v)$

Come trovare arco leggero rapidamente?

Nella **coda a priorità** Q :

- Ogni elemento è un vertice in $V - V_A$ (ancora da aggiungere a V_A)
- $v.key$ è il peso minimo di ogni arco (u, v) con $u \in V_A$
- EXTRACT-MIN restituisce v t.c. esiste $u \in V_A$ e (u, v) è un arco leggero che attraversa $(V_A, V - V_A)$
- $v.key = \infty$ se v non è adiacente a nessun vertice in V_A

Gli archi di A formano un albero con radice r :

- Per ogni vertice v , $\pi = \text{padre di } v$
 $\pi = \text{NIL}$ se $v = r$

Invariante di ciclo

- Prima di ogni iterazione del while:
 - ① $A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$
 - ② I vertici già inseriti in MST sono quelli in $V - Q$
 - ③ Per ogni vertice $v \in Q$
 se $v.\pi \neq NIL$ allora $v.key (< \infty)$ è il peso di un arco **leggero** $(v, v.\pi)$
 che collega v a qualche vertice che si trova già in A
- Alla fine, $V_A = V$
 $\Rightarrow Q = \emptyset$
 e MST è $A = \{(v, v.\pi) : v \in V - \{r\}\}$

Code con priorità (richiamo...)

- Gestiscono insieme dinamico S
- Ogni elemento nell'insieme ha una **chiave**
- Operazioni con Min-priority:
 - ▶ $\text{INSERT}(S, x)$: inserisce l'elemento x in S
 - ▶ $\text{MINIMUM}(S)$: restituisce l'elemento di S con chiave minima
 - ▶ $\text{EXTRACT-MIN}(S)$: rimuove e restituisce l'elemento di S con chiave minima
 - ▶ $\text{DECREASE-KEY}(S, x, k)$: decrementa il valore dell'elemento x a k ($k \leq$ valore corrente della chiave di x)

Analisi

Dipende da come viene impletata la coda a priorità :

- Se Q è un heap binario

- ▶ INSERT $O(\lg n)$
- ▶ EXTRACT-MIN $O(\lg n)$
- ▶ DECREASE-KEY $O(\lg n)$

- **Analisi:**

Inizializzare Q e primo ciclo for: $O(V \lg V)$

Decrementare chiave di r : $O(\lg V)$

Ciclo while : $|V|$ chiamate a EXTRACT-MIN
 $\Rightarrow O(V \lg V)$
 $\leq |E|$ chiamate a DECREASE-KEY
 $\Rightarrow O(E \lg V)$

Totale: $O(E \lg V)$

- PRIM è asintoticamente uguale a KRUSKAL: $O(E \lg V)$